“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Curso

Curso integrador 1: Sistema software

Proyecto

CiviGo

Entrega

Avance 3

Docente

IVAN GUSTAVO HUAMAN TORES

Integrantes

Will Smith Ramos Llamocca – U22305797

Espejo Chacaltana Sebastian Imanol – U23266563

Zamora Zamudio Darick Hanssel – U22217326

Ica - Perú - 2026

Contenido

1 Introducción....4
2 Elaboración del plan de proyecto de desarrollo de software....5
2.1 Resumen ejecutivo....5
2.2 Antecedentes....5
2.3 Objetivo del proyecto....6
2.4 Alcance del proyecto....6
2.5 Requerimientos del proyecto....6
2.6 Estrategia de desarrollo....6
3 Descripción de la empresa....7
3.1 Misión....7
3.2 Visión....7
3.3 Rubro del negocio....7
4 Planteamiento de objetivos....8
4.1 Objetivo general....8
4.2 Objetivos específicos....8
5 Definición de los requerimientos del proyecto....9
6 Definición del alcance de la solución....12
6.1 Descripción del proyecto....12
6.2 Descripción de procesos de negocio....12
6.3 Aspectos dentro del alcance....12
6.4 Aspectos fuera del alcance....13
6.5 Restricciones....13
7 Elaboración del cronograma....14
7.1 Cronograma de documentación....14
7.2 Cronograma de desarrollo....15
8 Análisis y modelado de los casos de uso del negocio....17
8.1 Diagrama de casos de uso del negocio....17
8.2 Diagrama de actividades....18
9 Análisis y modelado de los casos de uso del sistema....19
9.1 Diagrama de caso de uso del sistema....19
9.2 Diagrama de actividades....21
10 Análisis de los requisitos del proyecto....23

Contenido

11 Matriz de trazabilidad de requisitos vs CUS.....25
12 Matriz de trazabilidad de requisitos vs HU.....26
12.1 Historias de usuario.....27
13 Análisis de la solución planteada.....31
13.1 Problemática de la empresa.....31
13.2 Tecnologías a emplear.....31
13.3 Descripción de la empresa.....32
13.4 Alcance de la solución del proyecto.....32
14 Especificaciones de los casos de uso.....33
15 Diagramas de análisis del sistema.....43
15.1 Arquitectura del sistema.....43
15.2 Realización de los casos de uso y diagramas de secuencia y comunicación.....44
16 Estructura del software.....50
16.1 Patrones de diseño.....51
17 Diagramas de modelo de datos.....52
17.1 Diagrama conceptual.....52
17.2 Diagrama lógico.....53
17.3 Diagrama físico.....55
18 Análisis de la calidad de la base de datos.....56
18.1 Normalización y entidades de la DB.....56
18.2 Diagrama entidad relación.....57
18.3 Diccionario de datos.....57
18.4 Scripts de creación de DB y tablas.....68
18.5 Scripts de inserción de data.....78
19 Gestión del desarrollo con Jira y GitHub.....82
19.1 Jira Épicas.....82
19.2 Jira HU del sprint 3 Prototipo.....82
19.3 Jira Backlog pendientes para otro sprint.....83
19.4 Jira Roadmap resumen del progreso.....83
19.5 Jira Sincronización con GitHub.....84
20 Diseño de mockups.....85

1 Introducción

CiviGo es una plataforma de información ciudadana para la provincia de Ica. Su propósito es relacionar cada incidente reportado con el lugar y el momento en que ocurrió, someterlo a las reglas de revisión del sistema y facilitar su consulta en un mapa. Esta información también interviene en la comparación de recorridos a pie, en bicicleta y en automóvil.

El tercer avance presenta el análisis y diseño de la plataforma web y de la aplicación de escritorio del administrador. Comprende el frontend desarrollado con Next.js, la API construida con Node.js y Express, la persistencia en PostgreSQL mediante Prisma y el cliente administrativo implementado con Electron. El análisis comprende también los servicios que participan en estos componentes, como la cartografía, la verificación del correo y la evaluación automatizada de reportes. [R01–R04]

Este Avance 3 corresponde a la semana 12 de un ciclo de 17 semanas. Presenta los requisitos, los modelos de negocio y del sistema, la arquitectura, el diseño de datos y los mockups hasta el punto 20 del índice del curso. Las cinco semanas restantes se destinan a la integración, la verificación de los flujos y la preparación de la entrega final.

El análisis se apoya en una revisión fija del repositorio, identificada en las referencias. La presencia de una función en el código permite describir su implementación; su aceptación requiere comprobar el comportamiento en el ambiente de entrega. Esta distinción guía el cronograma y los escenarios de validación del informe.

La información del mapa expresa el contenido registrado y su tratamiento dentro del sistema. Un tramo con nivel cero puede carecer de aportes vigentes o de puntuación suficiente. Ese resultado no acredita que la vía esté libre de incidentes. De la misma manera, una clasificación automática no sustituye la revisión de pruebas ni la intervención de los servicios de emergencia.

2 Elaboración del plan de proyecto de desarrollo de software

2.1 Resumen ejecutivo

CiviGo articula la recepción de aportes ciudadanos, su revisión y la consulta geográfica. El diseño distingue el reporte presentado por una persona del incidente que representa el evento. Cuando varios aportes describen un mismo hecho, pueden asociarse a ese incidente sin perder su autoría ni sus evidencias. [R04, R06]

La web permite explorar el mapa, reportar, consultar el estado de los aportes y planificar recorridos a pie, en bicicleta o en automóvil. Los agentes trabajan en un centro de revisión sujeto a permisos y ámbito territorial. El administrador utiliza un programa de escritorio para gestionar cuentas, catálogos, decisiones de revisión, anuncios y herramientas de control. Ambos clientes utilizan la misma API y las mismas reglas de negocio. [R01, R03, R10]

Al corte de la semana 12 se dispone de una base técnica identificable en los tres componentes. La prioridad para el cierre es demostrar que los contratos, los permisos y los cambios de estado se mantienen coherentes durante un flujo completo. El siguiente cuadro precisa el alcance de la evidencia utilizada en este avance.

Tabla 1 Situación del Avance 3 en la semana 12

Ámbito	Contenido al corte	Evidencia o condición
Implementación revisada	Web Next.js, API Express y escritorio Electron; modelo de 29 entidades y reglas de reportes y revisión.	Código de la revisión citada. No acredita una ejecución integrada.
Diseño documentado	25 requisitos funcionales, 20 casos de uso, 20 historias, arquitectura, modelo de datos y siete mockups.	Contenido del Avance 3 hasta el punto 20.
Cierre pendiente	Pruebas de contratos, permisos, archivos, rutas e integraciones; actualización de evidencias y sustentación.	Programación de las semanas 13 a 17.

2.2 Antecedentes

El problema inicial consiste en organizar información que, para ser útil en un recorrido, necesita ubicación, fecha y contexto de revisión. Una lista de publicaciones sin esos elementos dificulta distinguir un hecho vigente de un antecedente. También resulta insuficiente representar una avenida completa con una sola calificación cuando los aportes se concentran en un sector de ella.

CiviGo responde a esa necesidad mediante un catálogo de incidentes, coordenadas geográficas y un modelo de calles obtenido de OpenStreetMap. El código subdivide las vías para el cálculo del riesgo y conserva la relación con sus aristas originales. Sobre esa base se incorporan los aportes ciudadanos, sus validaciones y las decisiones del personal autorizado. [R08, R09]

El repositorio conserva la evolución del almacenamiento de reportes, la incorporación de usuarios y la ampliación hacia los procesos de revisión. Las migraciones posteriores agregan

publicación, archivos, anuncios y controles de acceso a los datos. Estos cambios constituyen el antecedente técnico del modelo descrito en el tercer avance. [R18]

2.3 Objetivo del proyecto

Desarrollar una plataforma web de información ciudadana para Ica que permita registrar y revisar incidentes georreferenciados, consultar su aporte al riesgo por tramo y comparar recorridos, con una aplicación de escritorio para la administración del sistema.

2.4 Alcance del proyecto

El producto comprende una web para visitantes, ciudadanos y agentes, una API central y un cliente de escritorio reservado al administrador. Incluye la base de datos, los archivos asociados a los reportes y los adaptadores de servicios que utiliza este conjunto. La cobertura funcional del mapa corresponde a la provincia de Ica y a los datos de vías y lugares incorporados por el proyecto.

El tercer avance desarrolla el análisis de requisitos, el modelado de procesos, la arquitectura y el diseño de datos. También presenta la organización del trabajo hasta la semana 17 y los mockups de las interfaces incluidas en este alcance. La aprobación académica del avance no equivale al cierre del desarrollo.

2.5 Requerimientos del proyecto

Los requerimientos se organizan en cinco áreas: acceso y perfil, participación ciudadana, consulta geográfica, revisión de información y administración. Cada área tiene un responsable funcional, reglas de autorización y registros que permiten reconstruir sus operaciones. La definición detallada se presenta en los puntos 5 y 10, con identificadores estables para su trazabilidad.

Para operar, la solución necesita un servidor de API, una conexión PostgreSQL, almacenamiento persistente para adjuntos y un cliente con acceso a Internet. La cartografía y los servicios externos requieren la configuración que corresponda a cada proveedor. El permiso de ubicación del navegador resulta necesario para las acciones que exigen cercanía al incidente.

2.6 Estrategia de desarrollo

El trabajo se organiza por incrementos funcionales. Cada incremento relaciona una interfaz, un contrato de API y una regla persistida. Por ejemplo, publicar un reporte requiere que el formulario, la validación de archivos y la creación de Report e Incident funcionen como un mismo flujo. La revisión de un cambio debe seguir esa relación y evitar que una pantalla habilite acciones que el servidor todavía no autoriza.

GitHub conserva el código y el historial de cambios. El flujo de verificaciones del repositorio contempla controles para backend, frontend y escritorio. La organización en sprints de este documento sirve como planificación académica del ciclo. Los resultados de una ejecución concreta de pruebas deben conservarse junto al cambio evaluado. [R17]

3 Descripción de la empresa

CiviGo es una iniciativa académica desarrollada en el curso integrador 1: Sistema software de la Universidad Tecnológica del Perú, sede Ica. El equipo diseña e implementa una solución de información ciudadana cuya operación distingue a la comunidad que aporta información del personal que la revisa y administra.

Para efectos del modelo de negocio del curso, la organización responsable es el equipo del proyecto CiviGo. Los procesos de agentes y administradores describen responsabilidades del software. No establecen una vinculación institucional con una municipalidad, la Policía Nacional u otra entidad.

3.1 Misión

Facilitar a las personas que se desplazan por Ica la consulta y comunicación de incidentes de su entorno, mediante información geográfica con estados de revisión comprensibles y un tratamiento reservado de sus datos personales.

3.2 Visión

Consolidar una plataforma de participación ciudadana para Ica que permita consultar antecedentes y situaciones vigentes por tramo de vía, con procesos de revisión que la comunidad pueda comprender y el personal autorizado pueda supervisar.

3.3 Rubro del negocio

El proyecto pertenece al desarrollo de software y a los servicios de información geográfica aplicados a la participación ciudadana. Su función principal consiste en organizar información y apoyar la elección de recorridos. Las funciones de negocios, anuncios y recompensas forman parte del diseño complementario del producto, con operaciones comerciales todavía demostrativas.

4 Planteamiento de objetivos

4.1 Objetivo general

Implementar una solución web con administración de escritorio que gestione el ciclo de los reportes ciudadanos de Ica y presente información de incidentes y riesgo registrado por tramo para la consulta del entorno y la planificación de recorridos.

4.2 Objetivos específicos

Los objetivos específicos se vinculan con una evidencia de cumplimiento para orientar la verificación de las semanas 13 a 17. Las evidencias indicadas son criterios de cierre; su inclusión en el cuadro no declara una prueba ejecutada.

Tabla 2 Objetivos y criterios de cumplimiento

ID	Objetivo específico	Evidencia esperada al cierre	Requisitos
OE01	Implementar el registro, la verificación de correo y el acceso por rol.	Aceptar el acceso válido y rechazar la sesión revocada o el permiso insuficiente.	RF01–RF04
OE02	Gestionar reportes con ubicación, fecha y evidencias, conservando la relación con el incidente.	Seguir un aporte desde su envío hasta su consulta personal, con archivos del titular.	RF06–RF08
OE03	Integrar evaluación automática y revisión humana con decisiones motivadas.	Conservar estado, actor y motivo; derivar a revisión cuando la evaluación no sea concluyente.	RF07, RF18, RF21
OE04	Representar el riesgo registrado por tramo y comparar recorridos en los tres modos de transporte.	Reproducir el cálculo de niveles y comprobar rutas a pie, en bicicleta y en automóvil.	RF05, RF11 –RF14
OE05	Desarrollar la administración de cuentas, catálogos, contenido y operaciones desde el escritorio.	Completar operaciones autorizadas y rechazar su ejecución con una cuenta sin rol ADMIN.	RF19–RF25
OE06	Mantener trazabilidad entre requisitos, casos de uso, historias y componentes.	Vincular los 25 requisitos con casos e historias y registrar la evidencia de aceptación prevista.	RF01–RF25

5 Definición de los requerimientos del proyecto

Los requisitos funcionales describen comportamientos que el usuario puede observar o que el servidor debe ejecutar para completar una operación. La prioridad principal identifica funciones necesarias para el flujo de reportes y revisión. La prioridad complementaria identifica funciones que amplían la experiencia y la administración. Esta clasificación ordena la planificación del ciclo y no expresa un porcentaje de implementación.

Tabla 3 Requisitos funcionales

ID	Requerimiento y resultado esperado	Prioridad
RF01	Registrar una cuenta ciudadana. Crear una cuenta con nickname, Gmail, teléfono y fecha de nacimiento válidos.	Principal
RF02	Iniciar y cerrar sesión. Autenticar credenciales y crear o invalidar la sesión propia de CiviGo.	Principal
RF03	Verificar el correo. Comprobar con Google que el correo verificado coincide con la cuenta actual.	Principal
RF04	Gestionar el perfil y recuperar acceso. Actualizar datos permitidos, cambiar contraseña y tramitar recuperación de acceso.	Principal
RF05	Consultar el mapa y sus filtros. Consultar incidentes publicados y su detalle según los filtros disponibles.	Principal
RF06	Registrar un reporte georreferenciado. Validar tipo, ubicación, fecha y hasta tres adjuntos del ciudadano.	Principal
RF07	Evaluar y decidir la publicación inicial. Procesar evaluación y origen de imágenes, agrupar aportes y conservar la revisión pendiente.	Principal
RF08	Dar seguimiento a los reportes propios. Consultar aportes, incorporar evidencia autorizada y presentar apelaciones.	Principal
RF09	Registrar participación sobre incidentes. Confirmar, proponer resolución o remitir avisos según tipo, ubicación y permisos.	Principal
RF10	Mantener conversaciones de un incidente. Consultar y enviar mensajes en el espacio habilitado y permitir moderación autorizada.	Complementaria
RF11	Representar el riesgo registrado por tramo. Calcular puntos y niveles de 0 a 5 sobre segmentos de la red vial.	Principal
RF12	Comparar recorridos por modo. Calcular rutas rápida, más segura o equilibrada para caminar, bicicleta y automóvil.	Principal
RF13	Buscar lugares y construir itinerarios. Buscar destinos y lugares del corredor e incorporar paradas al recorrido.	Complementaria
RF14	Conservar recorridos personales. Gestionar favoritos, historial y estadísticas dentro de los límites del perfil.	Complementaria
RF15	Consultar notificaciones. Consultar notificaciones de la cuenta y marcar su lectura.	Principal
RF16	Consultar participación e incentivos. Consultar puntos, ranking y recompensas, y solicitar canjes demostrativos con saldo y stock.	Complementaria

ID	Requerimiento y resultado esperado	Prioridad
RF17	Consultar el asistente. Responder consultas sobre CiviGo con contexto público y sin afirmar acciones no ejecutadas.	Complementaria
RF18	Revisar aportes e incidentes. Validar, reclasificar, ajustar gravedad, resolver o reabrir según autorización vigente.	Principal
RF19	Administrar cuentas y permisos. Gestionar usuarios, roles, bloqueo, distrito y permisos desde el escritorio.	Principal
RF20	Mantener catálogo y reglas. Gestionar categorías, tipos y parámetros aceptados por el backend.	Principal
RF21	Resolver apelaciones y recuperaciones. Registrar decisiones motivadas y sus efectos sobre cuenta, reporte y puntos.	Principal
RF22	Consultar control y exportaciones. Filtrar estadísticas, revisar auditoría y generar exportaciones administrativas autorizadas.	Complementaria
RF23	Administrar contenido del mapa. Mantener negocios, anuncios e imágenes y moderar avisos externos de tráfico.	Complementaria
RF24	Importar antecedentes históricos. Revisar una vista previa de los registros y confirmar una importación autorizada.	Complementaria
RF25	Ejecutar operaciones de mantenimiento. Procesar plazos, cierre de ranking y gestión de recompensas mediante acciones administrativas.	Complementaria

Los siguientes requisitos no funcionales establecen condiciones de diseño y criterios de revisión. Cuando existe un mecanismo en el repositorio, la tabla lo identifica. Un mecanismo implementado debe comprobarse en el ambiente de entrega antes de declarar cumplido el requisito.

Tabla 4 Requisitos no funcionales

ID	Condición	Criterio y comprobación
RNF01	Autorización	El backend debe comprobar sesión, rol, bloqueo, permisos y ámbito en cada operación protegida. auth.js y agent-permissions.js; comprobar acceso cruzado y revocación.
RNF02	Reserva de información	Los datos personales y los archivos privados deben entregarse solo al propietario o al personal autorizado. Proyecciones de respuesta y rutas de uploads; comprobar respuesta pública y descarga.
RNF03	Integridad transaccional	Las decisiones que afecten estados, puntos, saldo o stock deben conservar un resultado coherente. Transacciones serializables y restricciones únicas; comprobar concurrencia y repetición.
RNF04	Gestión de sesión	Las credenciales no deben almacenarse en texto claro y la sesión debe poder revocarse. scrypt, token aleatorio y hash de sesión; comprobar cierre y cambio de contraseña.
RNF05	Validación y límites	Las entradas, archivos y solicitudes deben respetar formatos, tamaños y cuotas. Validadores de API, multer y contadores; comprobar límites y contenido inválido.
RNF06	Tratamiento de fallos externos	Una falla de IA, tráfico o almacenamiento debe tener un estado y mensaje identificables. Adaptadores y errores controlados; comprobar indisponibilidad sin aprobación ficticia.

ID	Condición	Criterio y comprobación
RNF07	Usabilidad	Las pantallas deben diferenciar espera, error, ausencia de datos y resultado satisfactorio. Estados de componentes y controles de formulario; verificar lectura y navegación.
RNF08	Mantenibilidad	Cada cambio debe conservar contratos y permitir revisar la lógica fuera de la interfaz. Separación de rutas, librerías y componentes; revisar módulos y verificaciones de CI.
RNF09	Aislamiento del escritorio	El renderer debe usar el puente permitido y no exponer la sesión ni acceso general de red. Preload, IPC, CSP y sandbox; comprobar rutas y orígenes rechazados.
RNF10	Trazabilidad	Una decisión de revisión debe conservar actor, acción, motivo y referencias afectadas. Review y AuditLog; comprobar correspondencia con reportes y apelaciones.

6 Definición del alcance de la solución

6.1 Descripción del proyecto

La solución aplica una arquitectura de clientes separados con un backend común. La web presenta las funciones ciudadanas y el espacio de trabajo de los agentes. El programa de escritorio contiene las herramientas administrativas y transmite las solicitudes a través del proceso principal de Electron. La API determina la autorización, ejecuta las reglas de negocio y accede a la base de datos. [R01–R04, R16]

La base de datos almacena las cuentas, reportes, incidentes y decisiones. Los archivos se conservan en un almacenamiento local persistente o en un bucket privado de Supabase, según la configuración. La cartografía se muestra mediante Mapbox GL, mientras que la red utilizada para calcular recorridos procede de los datos de OpenStreetMap incorporados en el backend. [R09, R12]

6.2 Descripción de procesos de negocio

El proceso de participación comienza cuando una persona crea su cuenta y verifica el correo. Para reportar selecciona un tipo del catálogo, ubica el hecho y adjunta la evidencia exigida por ese tipo. El servidor verifica la solicitud, analiza los datos disponibles y crea o vincula el incidente. Los envíos que necesitan una evaluación adicional conservan su estado de revisión.

La consulta comunitaria reúne el mapa, el detalle del incidente y las acciones permitidas para el ciudadano. Las confirmaciones cercanas pueden incrementar la validación de hechos comunitarios. Los reportes individuales de delitos siguen el tratamiento de pruebas privadas previsto en el catálogo. Un agente o administrador puede registrar una decisión motivada de acuerdo con sus permisos.

La planificación de recorridos combina la red vial, el modo de transporte y el riesgo registrado. El usuario compara alternativas y puede consultar lugares o guardar un recorrido. La gestión administrativa mantiene las cuentas, revisa solicitudes y supervisa la información publicada. Estos procesos comparten registros y deben conservar la coherencia entre lo que se muestra y lo que se autoriza.

6.3 Aspectos dentro del alcance

- Web pública y autenticada, incluida la interfaz de trabajo de los agentes.
- Backend, base PostgreSQL, migraciones y almacenamiento de archivos de los componentes incluidos.
- Aplicación de escritorio del administrador, sus comunicaciones con la API y las exportaciones autorizadas.
- Reportes, incidentes, evidencias, revisión, apelaciones, notificaciones y participación comunitaria.
- Cartografía, riesgo por tramos, búsqueda de lugares, recorridos, favoritos e historial.
- Configuración de servicios, anuncios, negocios y mecanismos demostrativos de recompensas y Premium.

6.4 Aspectos fuera del alcance

La aplicación móvil independiente y su carpeta mobile quedan fuera de este proyecto documental. Tampoco se incluyen su compilación, sus dependencias nativas ni sus interfaces. El alcance no contempla un sistema de despacho de emergencias, el seguimiento policial de personas o una garantía sobre la seguridad de un recorrido.

Los pagos reales, la contratación de servicios comerciales y la entrega efectiva de premios no se consideran operaciones concluidas del producto. La existencia de un conector tampoco acredita que una entidad pública suministre datos mediante un convenio o una API oficial.

6.5 Restricciones

La cobertura depende de la red de vías y de los lugares incorporados para Ica. La disponibilidad de los servicios externos depende de credenciales, cuotas y conectividad. El navegador requiere permisos para utilizar la ubicación y las funciones de voz. El cliente de escritorio se configura para Windows. [R03, R09, R21]

El acceso administrativo depende del rol ADMIN y la revisión de agentes depende de permisos específicos y ámbito territorial. Las evidencias privadas deben pasar por la autorización del backend. La continuidad del almacenamiento exige un volumen persistente o la configuración de Supabase Storage. El cierre académico está previsto para la semana 17.

7 Elaboración del cronograma

El cronograma comprende las 17 semanas del ciclo y fija el Avance 3 en la semana 12. Distingue la distribución académica de las actividades hasta ese corte y la programación de cierre de las semanas 13 a 17. Las barras representan periodos planificados; el cumplimiento se registra con entregas y evidencias, no con el tiempo transcurrido.

Se emplean semanas académicas para evitar asignar fechas de calendario que no han sido fijadas en esta planificación. Los hitos de las semanas 4 y 8 representan cortes de revisión del equipo. El hito confirmado para este informe es el Avance 3 en la semana 12. Las dependencias de documentación y desarrollo se leen en conjunto.

7.1 Cronograma de documentación

Tabla 5 Cronograma de documentación

Actividad	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Problema, alcance y objetivos	■	■	■														
Requisitos y procesos		■	■	■	■												
Casos de uso y actividades				■	■	■	■										
Historias y trazabilidad					■	■	■	■	■								
Arquitectura y estructura							■	■	■	■							
Modelo y diccionario de datos								■	■	■	■	■					
Mockups y consolidación del Avance 3										■	■	■					
Actualización con verificaciones													■	■	■		
Edición de entrega y sustentación																■	■

El borde de la semana 12 marca el corte del Avance 3. Las barras indican periodos planificados; las semanas 13 a 17 corresponden al cierre.

Tabla 6 Entregas y responsabilidades de documentación

Actividad	Entrega prevista	Responsabilidad
Problema, alcance y objetivos	Documento base	Equipo CiviGo
Requisitos y procesos	Catálogo y reglas	Análisis
Casos de uso y actividades	Modelos de negocio y sistema	Análisis
Historias y trazabilidad	Matrices RF–CU–HU	Análisis
Arquitectura y estructura	Vista técnica por componentes	Diseño
Modelo y diccionario de datos	Esquema, restricciones y scripts	Backend
Mockups y consolidación del Avance 3	Informe al punto 20	Equipo CiviGo
Actualización con verificaciones	Hallazgos y correcciones documentadas	Equipo CiviGo
Edición de entrega y sustentación	Informe y exposición del cierre	Equipo CiviGo

7.2 Cronograma de desarrollo

Tabla 7 Cronograma de desarrollo

Actividad	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Base del proyecto y persistencia	■	■	■														
Registro, sesión y verificación		■	■	■	■												
Reporte, archivos y publicación				■	■	■	■	■									
Revisión, permisos y apelaciones						■	■	■	■	■							
Mapa, tramos y recorridos						■	■	■	■	■	■	■					
Cliente de administración								■	■	■	■	■					
Contenido, control y funciones demo									■	■	■	■					
Integración y corrección de flujos													■	■	■		
Revisión de entrega y cierre																■	■

El borde de la semana 12 marca el corte del Avance 3. Las barras indican periodos planificados; las semanas 13 a 17 corresponden al cierre.

Tabla 8 Entregas y responsabilidades de desarrollo

Actividad	Entrega prevista	Responsabilidad
Base del proyecto y persistencia	Estructura y conexión de desarrollo	Backend
Registro, sesión y verificación	Acceso de cuentas	Web y backend
Reporte, archivos y publicación	Flujo ciudadano	Web y backend
Revisión, permisos y apelaciones	Decisiones y trazabilidad	Backend
Mapa, tramos y recorridos	Consulta geográfica integrada	Web y backend
Cliente de administración	Paneles y puente de escritorio	Escritorio
Contenido, control y funciones demo	Anuncios y herramientas de gestión	Equipo CiviGo
Integración y corrección de flujos	Regresiones y contratos verificados	Equipo CiviGo
Revisión de entrega y cierre	Ambiente y versión de sustentación	Equipo CiviGo

Criterios de cierre semanal

Las actividades de cierre siguen las dependencias del producto: primero se estabilizan los contratos y los permisos; luego se comprueban la persistencia y las integraciones; finalmente se revisan los recorridos de uso y la versión de sustentación. La secuencia permite registrar el cierre semanal con un resultado concreto.

Tabla 9 Secuencia de verificación y entrega de las semanas 13 a 17

Sem.	Trabajo previsto	Dependencia	Resultado de cierre	Backlog
13	Contratos entre clientes y API; acceso por rol y ámbito.	Versión de referencia del Avance 3.	Registro de divergencias y decisiones de corrección.	B01, B02
14	Regresión de permisos, archivos y persistencia; contraste del esquema.	Contratos y correcciones priorizados.	Resultados de los escenarios de acceso y datos, con incidencias.	B01, B02, B04, B05
15	Rutas, fallos de proveedores y recorridos de uso.	Ambiente con configuración y datos de prueba definidos.	Comparación de recorridos y registro de fallos controlados.	B03–B06
16	Revisión integrada de web y escritorio; actualización documental.	Incidencias críticas atendidas y resultados revisados.	Versión candidata y correspondencia entre documento e interfaces.	B06, B07
17	Preparación de la entrega y sustentación.	Versión candidata revisada.	Versión identificada, informe final y guion de demostración.	B07

8 Análisis y modelado de los casos de uso del negocio

El modelo de negocio describe lo que necesita lograr cada participante sin limitarlo a una pantalla o endpoint. El ciudadano comunica y consulta situaciones. El agente revisa información dentro de su ámbito. El administrador mantiene las condiciones de operación. Los servicios externos aportan datos o evaluaciones al proceso, sin asumir la responsabilidad de las decisiones humanas.

8.1 Diagrama de casos de uso del negocio

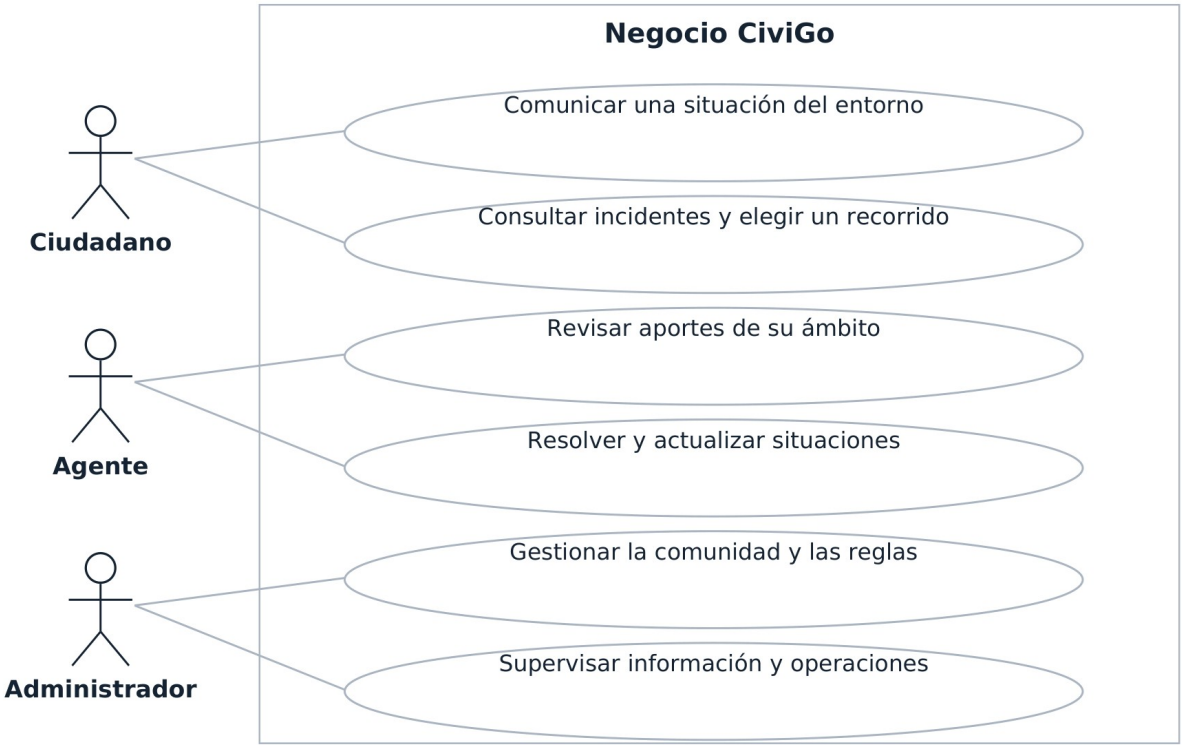


Figura 1 Casos de uso del negocio

La participación y la revisión forman un mismo proceso de información. Elaboración del equipo a partir de R06, R07 y R10.

8.2 Diagrama de actividades

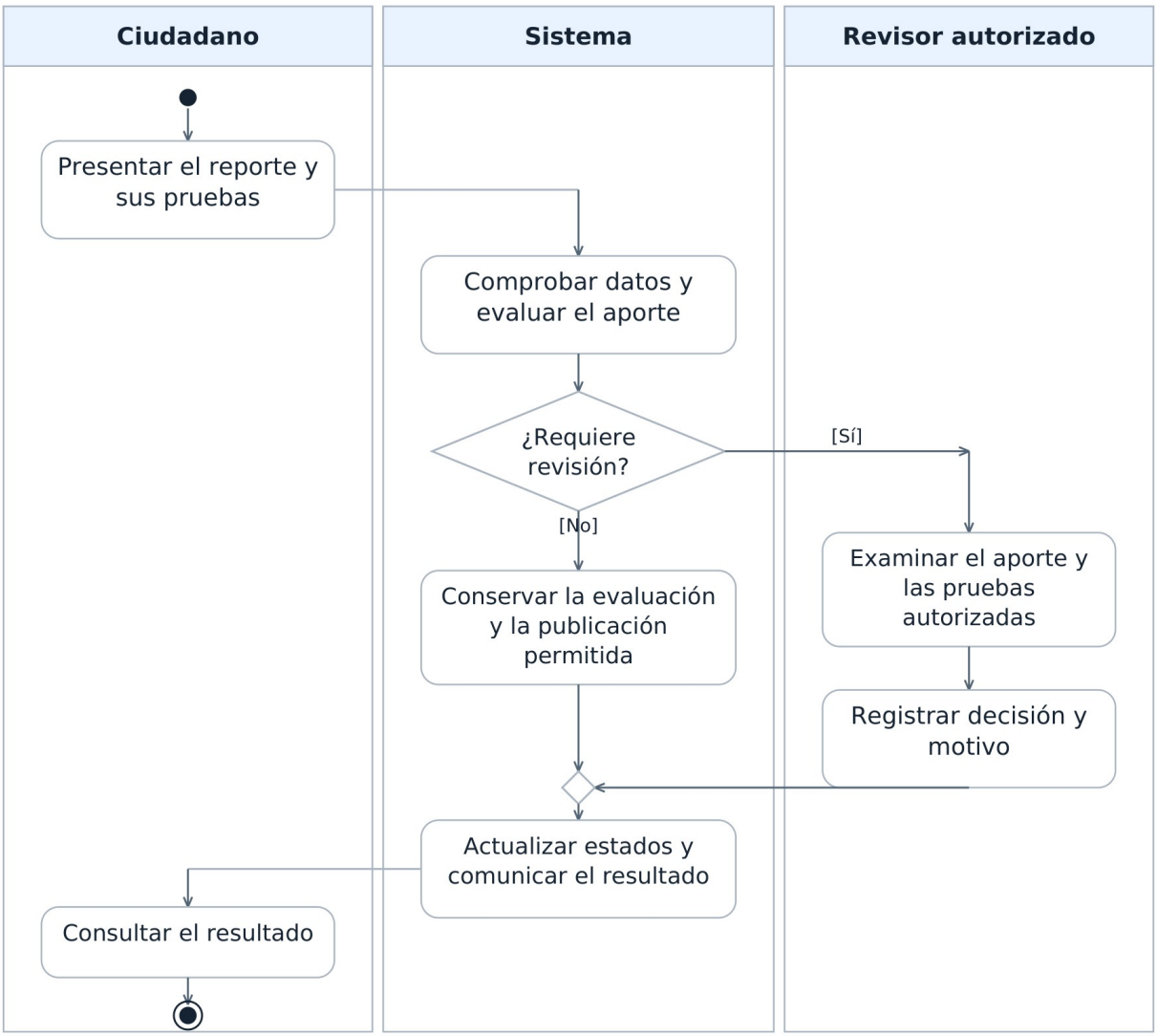


Figura 2 Actividad de atención de un reporte

Las particiones separan las responsabilidades del ciudadano, el sistema y el revisor. La publicación inicial y la excepción de emergencias se detallan en la figura 5. Fuentes R06 y R07.

9 Análisis y modelado de los casos de uso del sistema

Los casos de uso del sistema convierten los procesos anteriores en servicios verificables. Se definen veinte casos que cubren la web y el programa administrativo. Un caso puede reunir varias operaciones de un mismo objetivo, por ejemplo la consulta del mapa y de su detalle. Las matrices de los puntos 11 y 12 relacionan ese objetivo con los requisitos y las historias correspondientes.

9.1 Diagrama de caso de uso del sistema

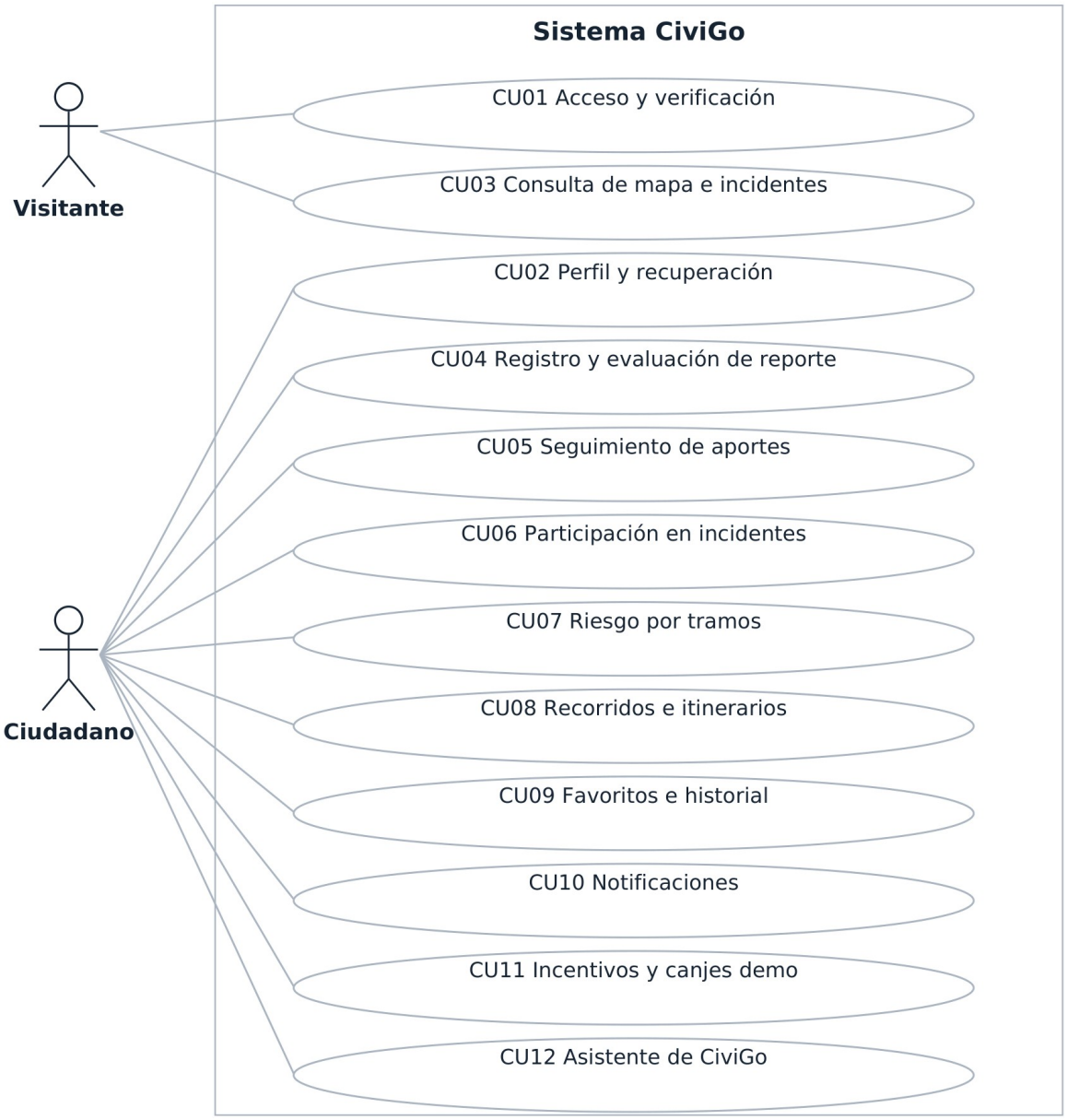


Figura 3 Casos de uso del sistema para la comunidad

CU01 a CU12. Las acciones de participación requieren las condiciones de acceso definidas por el servidor. Fuentes R05–R09 y R11.

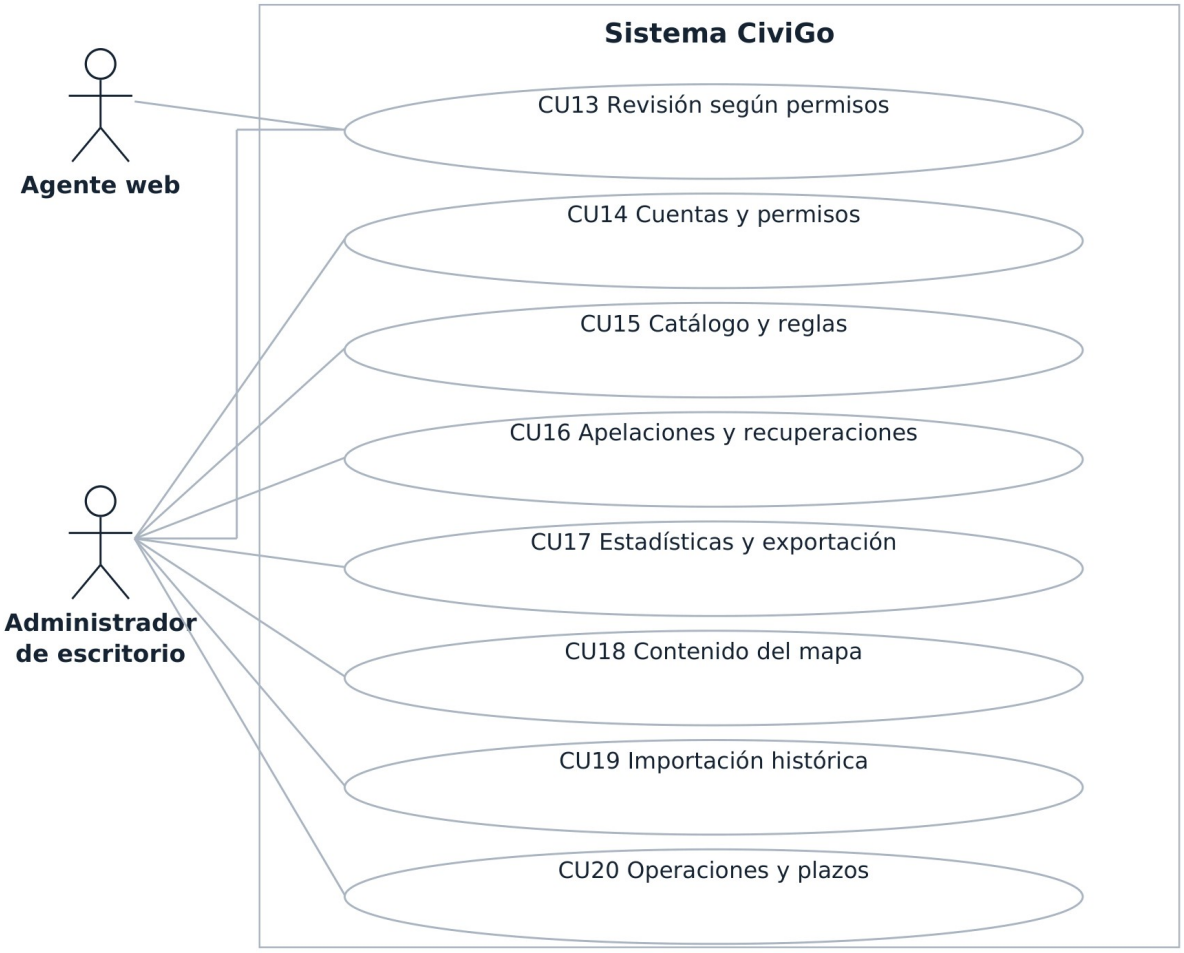


Figura 4 Casos de uso de revisión y administración

CU13 a CU20. El agente utiliza la web y el administrador utiliza el cliente de escritorio. Fuentes R03, R10, R13 y R15.

9.2 Diagrama de actividades

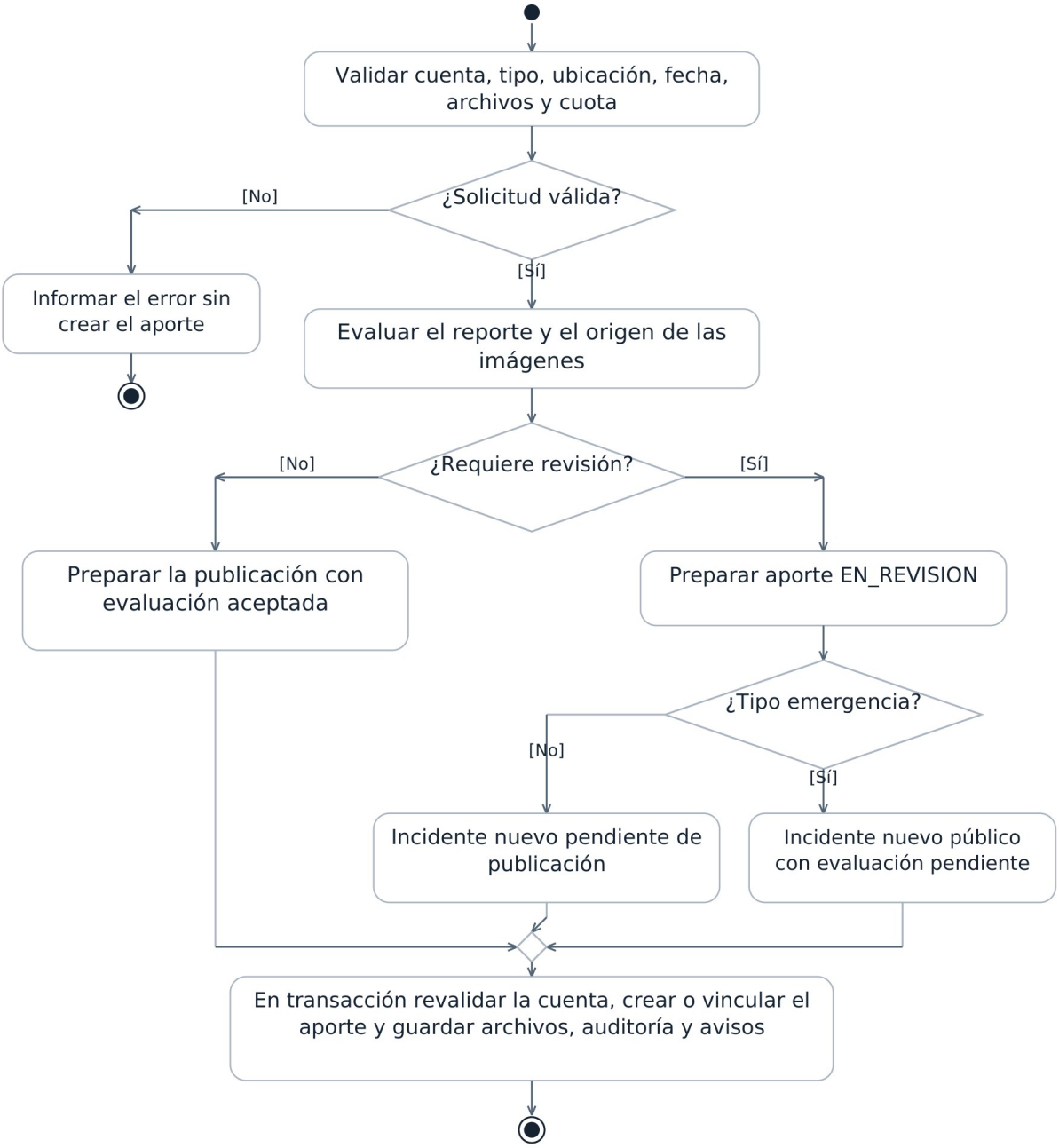


Figura 5 Registro y tratamiento de un reporte

Las condiciones entre corchetes indican cada bifurcación. La publicación descrita corresponde a un incidente nuevo; si el aporte se agrupa con uno existente, se conservan sus estados y las reglas de revisión del agente. Fuente R06.

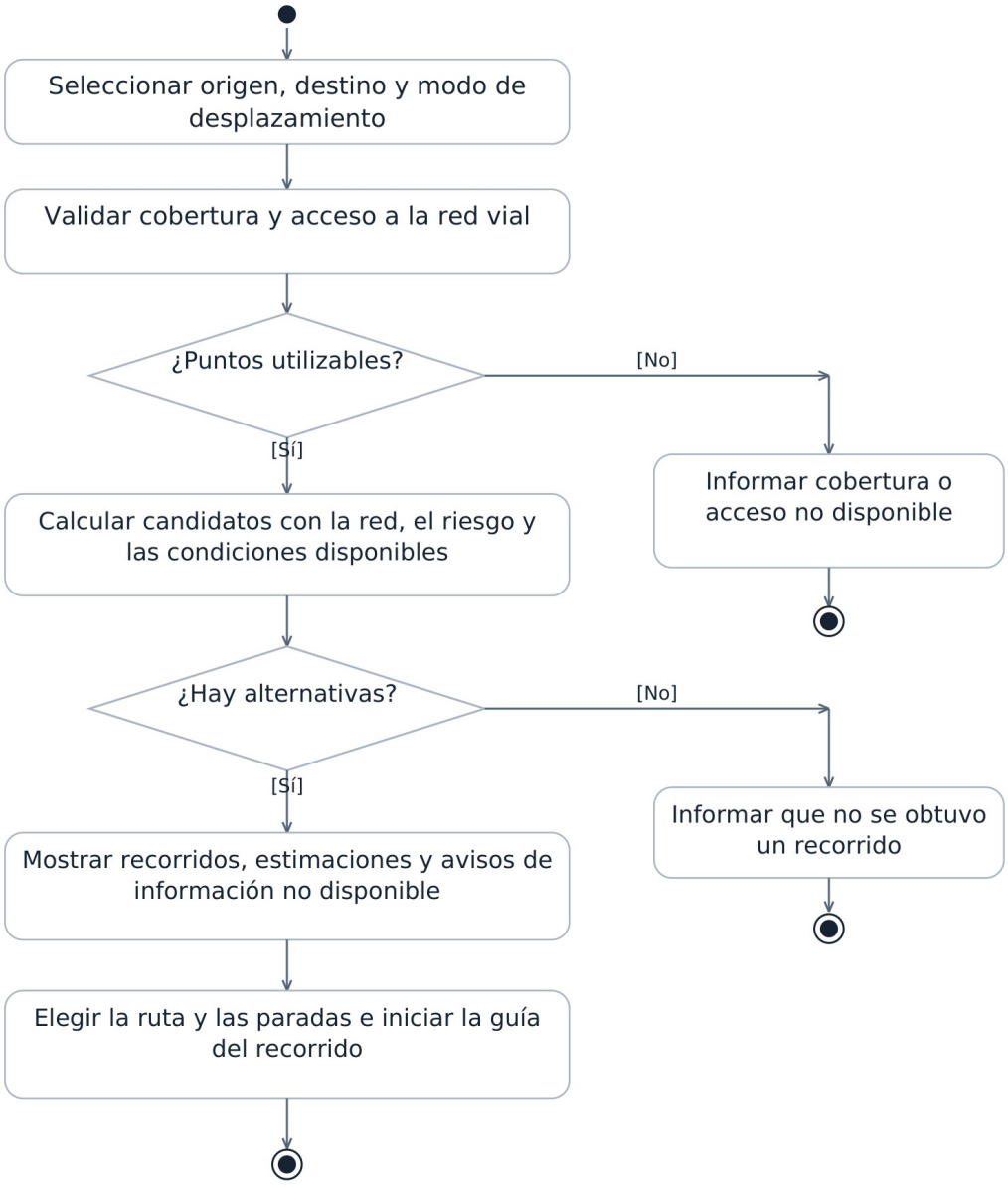


Figura 6 Selección de un recorrido

La comparación informa al usuario y mantiene diferenciados el riesgo registrado y las condiciones temporales de tráfico. Fuente R09.

10 Análisis de los requisitos del proyecto

El análisis distingue Report, que conserva el aporte, su autoría y sus archivos, de Incident, que reúne la información consultada en el mapa. Los cambios deben mantener coherencia entre ambas entidades y sus efectos sobre puntos y notificaciones. Las reglas siguientes proceden del backend y utilizan los valores iniciales de los parámetros configurables. [R04, R06, R07]

Tabla 10 Reglas de negocio

Regla	Aspecto	Tratamiento
RN01	Identidad	El registro web utiliza Gmail y exige una edad mínima de 12 años. La participación requiere correo verificado. [R05]
RN02	Gravedad y nivel	El incidente admite gravedad de 1 a 5. El tramo utiliza un nivel calculado de 0 a 5. Un valor pendiente no es gravedad cero. [R06, R08]
RN03	Ubicación del reporte	Los tipos sin ubicación remota requieren GPS y permiten corregir el punto hasta 50 m. La API comprueba la cobertura. [R06]
RN04	Fecha	La fecha debe estar dentro de la última semana. Los tipos de situación actual tienen además una ventana de tres horas. [R06]
RN05	Archivos	Un reporte admite hasta tres adjuntos. El tipo puede exigir una fotografía. Las cargas tienen límites y autorización propia. [R06, R12]
RN06	Agrupación	Los tipos no individuales pueden agruparse por tipo, distancia inicial de 50 m y ventana inicial de tres horas. [R06, R14]
RN07	Publicación	Una evaluación aceptada puede habilitar la publicación. Si requiere revisión, el aporte queda EN_REVISION. Una emergencia puede ser pública con evaluación pendiente. [R06]
RN08	Confirmación	El autor inicial no confirma su propio incidente. Los hechos individuales se revisan con pruebas privadas y no reciben votos comunitarios. [R07]
RN09	Validación comunitaria	La validación parte de 0,5 y alcanza 1 con tres confirmaciones elegibles bajo las reglas iniciales. El personal autorizado puede validar según su permiso. [R07, R14]
RN10	Resolución	La regla inicial considera cinco votos de resolución o la decisión del personal autorizado. Un antecedente histórico puede conservar su publicación. [R07, R14]
RN11	Atribución por tramos	La red se subdivide en segmentos de hasta 150 m para el riesgo. El aporte se atribuye al tramo cercano y puede influir en vecinos conectados. [R08]
RN12	Revisión y sanciones	La sospecha automática solicita revisión. Una declaración de falsedad exige revisión y motivo, con efectos sobre la contribución y posibilidad de apelación. [R07, R10]
RN13	Acceso territorial	El agente necesita el permiso de la acción y el ámbito aplicable. ADMIN y el agente COLABORADOR tienen las excepciones previstas por el código. [R05, R10]
RN14	Operaciones comerciales	Los canjes y Premium son demostrativos. El canje exige disponibilidad y saldo, y utiliza una transacción. [R11, R21]

La gravedad del incidente va de 1 a 5 o queda pendiente. El nivel del tramo va de 0 a 5 y se obtiene multiplicando gravedad, validación y peso temporal, atribuyendo los puntos al tramo y convirtiendo su total mediante levelFromPoints. [R08]

Por ejemplo, gravedad 5, validación 0,5 y peso temporal 1 producen 2,5 puntos. Un tramo con ese único aporte alcanza el nivel 1. Dos aportes de 5 puntos suman 10 puntos y producen nivel 2. Se trata de un ejemplo de cálculo, sin atribución a una calle real.

Los escenarios de aceptación cubren el flujo principal, los cambios de autorización, las solicitudes repetidas y los fallos externos. Están previstos para el cierre del ciclo. Cada ejecución deberá registrar versión, ambiente, datos de prueba, resultado obtenido e incidencias.

Tabla 11 Escenarios de aceptación previstos

ID y cobertura	Escenario previsto	Resultado esperado
VA01 RF02–RF04; RNF04	Iniciar sesión, cerrarla y volver a enviar una operación protegida con el identificador anterior.	La sesión cerrada no autoriza la operación; el cliente solicita un nuevo acceso.
VA02 RF06, RF07; RNF05	Enviar un aporte válido y repetir con ubicación no permitida, fecha fuera de plazo o archivos ajenos.	El envío válido conserva autoría; los datos inválidos se rechazan sin crear un reporte parcial.
VA03 RF07, RF18; RNF06	Interrumpir la evaluación externa durante el envío; repetir con un tipo de emergencia.	El aporte queda para revisión. La emergencia puede publicarse con evaluación pendiente, sin gravedad ficticia.
VA04 RF08, RF21; RNF02	Consultar el reporte y descargar una evidencia privada desde el titular y desde otra cuenta sin permiso.	Solo el titular o el personal autorizado accede a la prueba; la consulta pública no la expone.
VA05 RF09; RNF03	Repetir una confirmación con el mismo usuario e intentar confirmar el incidente propio.	La participación duplicada no incrementa el resultado y el autor inicial no confirma su incidente.
VA06 RF11	Calcular un tramo aislado con 0, 2,5, 5 y 10 puntos; sin aportes de vecinos.	Los niveles son, respectivamente, 0, 1, 1 y 2. La interfaz diferencia nivel cero de evaluación pendiente.
VA07 RF12, RF13; RNF06	Comparar el mismo origen y destino con los tres modos; repetir sin tráfico disponible.	Cada modo aplica sus restricciones. La indisponibilidad de tráfico no aparece como información actualizada.
VA08 RF18, RF19; RNF01, RNF10	Revisar con un agente fuera de ámbito y revocar el permiso antes de confirmar otra decisión.	Se rechaza la acción no autorizada. Una revisión permitida conserva actor, motivo y entidades afectadas.
VA09 RF21; RNF03	Aceptar una apelación válida y repetir la solicitud de decisión sobre el mismo aporte.	La decisión mantiene un estado coherente y no duplica los puntos correspondientes al aporte.
VA10 RF22; RNF09	Exportar desde el escritorio con filtros definidos y repetir la acción con la sesión revocada.	El archivo autorizado conserva los filtros; la sesión revocada impide completar una nueva exportación.

11 Matriz de trazabilidad de requisitos vs CUS

La matriz permite seguir cada requisito funcional hasta el caso de uso que lo desarrolla y su fuente de implementación. CUS se utiliza como abreviatura de caso de uso del sistema. La columna de evidencia identifica el archivo o conjunto de módulos que contiene el comportamiento, sin sustituir la comprobación funcional.

Tabla 12 Relación entre requisitos y casos de uso

RF	Requisito	CUS	Fuente
RF01	Registrar una cuenta ciudadana	CU01	R05
RF02	Iniciar y cerrar sesión	CU01	R05
RF03	Verificar el correo	CU01	R05
RF04	Gestionar el perfil y recuperar acceso	CU02	R05
RF05	Consultar el mapa y sus filtros	CU03	R07
RF06	Registrar un reporte georreferenciado	CU04	R06
RF07	Evaluar y decidir la publicación inicial	CU04	R06
RF08	Dar seguimiento a los reportes propios	CU05	R06
RF09	Registrar participación sobre incidentes	CU06	R07
RF10	Mantener conversaciones de un incidente	CU06	R07
RF11	Representar el riesgo registrado por tramo	CU07	R08
RF12	Comparar recorridos por modo	CU08	R09
RF13	Buscar lugares y construir itinerarios	CU08	R09
RF14	Conservar recorridos personales	CU09	R09
RF15	Consultar notificaciones	CU10	R11
RF16	Consultar participación e incentivos	CU11	R11
RF17	Consultar el asistente	CU12	R21
RF18	Revisar aportes e incidentes	CU13	R10
RF19	Administrar cuentas y permisos	CU14	R10
RF20	Mantener catálogo y reglas	CU15	R14
RF21	Resolver apelaciones y recuperaciones	CU16	R10
RF22	Consultar control y exportaciones	CU17	R22
RF23	Administrar contenido del mapa	CU18	R13
RF24	Importar antecedentes históricos	CU19	R15
RF25	Ejecutar operaciones de mantenimiento	CU20	R10

12 Matriz de trazabilidad de requisitos vs HU

Las historias de usuario expresan el valor esperado para un participante. Se mantienen los mismos identificadores de requisitos utilizados en la matriz anterior para evitar que la planificación incorpore funciones sin relación con el alcance.

Tabla 13 Relación entre requisitos e historias de usuario

RF	Historia	Objetivo relacionado
RF01	HU01	Registrar y habilitar el acceso
RF02	HU01	Registrar y habilitar el acceso
RF03	HU01	Registrar y habilitar el acceso
RF04	HU02	Gestionar perfil y recuperación
RF05	HU03	Consultar mapa e incidentes
RF06	HU04	Registrar y evaluar un reporte
RF07	HU04	Registrar y evaluar un reporte
RF08	HU05	Dar seguimiento a un aporte
RF09	HU06	Participar en un incidente
RF10	HU06	Participar en un incidente
RF11	HU07	Consultar riesgo por tramos
RF12	HU08	Planificar recorrido e itinerario
RF13	HU08	Planificar recorrido e itinerario
RF14	HU09	Guardar y consultar recorridos
RF15	HU10	Consultar notificaciones
RF16	HU11	Consultar incentivos y solicitar canje
RF17	HU12	Consultar el asistente de CiviGo
RF18	HU13	Revisar incidentes y aportes
RF19	HU14	Administrar cuentas y permisos
RF20	HU15	Mantener catálogo y configuración
RF21	HU16	Resolver apelaciones y recuperaciones
RF22	HU17	Consultar estadísticas y exportar
RF23	HU18	Administrar contenido del mapa
RF24	HU19	Importar antecedentes históricos
RF25	HU20	Gestionar operaciones y plazos

12.1 Historias de usuario

HU01 Registrar y habilitar el acceso

Como ciudadano, quiero crear mi cuenta, acceder y verificar mi correo para participar con una identidad de contacto comprobada.

Criterios de aceptación. 1) La API rechaza correos o datos de registro que incumplen sus validaciones. 2) Una cuenta no verificada no puede publicar un reporte. 3) Verificar otra dirección de Google no habilita la cuenta actual.

HU02 Gestionar perfil y recuperación

Como ciudadano, quiero mantener mis datos y recuperar mi acceso para conservar el control de mi cuenta.

Criterios de aceptación. 1) Otra cuenta no puede editar el perfil consultado. 2) El cambio de contraseña no deja una sesión revocada como vigente. 3) Las solicitudes que necesitan revisión no se presentan como aceptadas.

HU03 Consultar mapa e incidentes

Como persona que se desplaza por Ica, quiero consultar incidentes y filtros para conocer la información registrada en mi entorno.

Criterios de aceptación. 1) Los incidentes retirados o falsos no aparecen como hechos activos públicos. 2) Los filtros afectan los resultados consultados. 3) El detalle público no incorpora las pruebas privadas del reporte.

HU04 Registrar y evaluar un reporte

Como ciudadano, quiero reportar un hecho con ubicación y pruebas para que la comunidad pueda consultarlo después de su tratamiento.

Criterios de aceptación. 1) Los archivos de otra cuenta no pueden incorporarse al reporte. 2) Una evaluación fallida no se registra como aprobada por IA. 3) La revisión pendiente se comunica al autor y al personal correspondiente.

HU05 Dar seguimiento a un aporte

Como autor, quiero consultar mis reportes y aportar aclaraciones para conocer y solicitar revisión de sus resultados.

Criterios de aceptación. 1) La consulta personal no devuelve reportes de otra cuenta. 2) Las pruebas privadas no se convierten en material público por añadirlas. 3) Una apelación conserva su motivo y su estado de atención.

HU06 Participar en un incidente

Como ciudadano, quiero confirmar una situación o conversar sobre ella para aportar información adicional sin duplicar el evento.

Criterios de aceptación. 1) Un voto repetido del mismo tipo no incrementa el resultado. 2) Un usuario fuera de la distancia exigida no confirma como vecino cercano. 3) La conversación aplica las condiciones de apertura y moderación.

HU07 Consultar riesgo por tramos

Como usuario del mapa, quiero distinguir el riesgo registrado por tramo para interpretar dónde se concentran los aportes vigentes.

Criterios de aceptación. 1) Una vía extensa se representa mediante segmentos del cálculo. 2) Un dato pendiente no se presenta como una evaluación de gravedad cero. 3) El nivel mostrado coincide con el intervalo de puntos de la función de cálculo.

HU08 Planificar recorrido e itinerario

Como ciudadano, quiero comparar recorridos y buscar lugares del camino para elegir un desplazamiento acorde con la información disponible.

Criterios de aceptación. 1) El modo de transporte afecta las restricciones y el cálculo. 2) Las condiciones de tráfico se distinguen del riesgo histórico. 3) La búsqueda de lugares no incorpora paradas fuera del itinerario sin selección del usuario.

HU09 Guardar y consultar recorridos

Como ciudadano, quiero guardar y revisar mis recorridos para volver a utilizar destinos y consultar mi actividad.

Criterios de aceptación. 1) Las listas se filtran por la cuenta autenticada. 2) El servidor impide exceder el límite de favoritos correspondiente. 3) Las opciones reservadas al perfil se autorizan también en la API.

HU10 Consultar notificaciones

Como usuario, quiero recibir avisos de mis operaciones para conocer revisiones y cambios relevantes.

Criterios de aceptación. 1) La consulta devuelve avisos de la cuenta actual. 2) Marcar como leída no permite modificar un aviso ajeno. 3) La notificación conserva su relación con el contexto de la operación.

HU11 Consultar incentivos y solicitar canje

Como participante, quiero consultar mi actividad y las recompensas de demostración para entender cómo se reconocen los aportes validados.

Criterios de aceptación. 1) La operación no anuncia un pago real ni una entrega efectiva. 2) La concurrencia no puede consumir dos veces el mismo saldo disponible. 3) Un aporte validado no recibe el mismo premio dos veces por la misma clave.

HU12 Consultar el asistente de CiviGo

Como usuario, quiero aclarar cómo funciona CiviGo para utilizar las funciones disponibles y entender sus límites.

Criterios de aceptación. 1) El contexto no incluye datos privados de otras cuentas. 2) La respuesta no certifica que una ruta sea segura. 3) La interfaz diferencia una respuesta de una falla del servicio.

HU13 Revisar incidentes y aportes

Como revisor autorizado, quiero examinar el aporte y registrar una decisión motivada para mantener información coherente.

Criterios de aceptación. 1) Un agente de otro ámbito no recibe autorización por ocultarse un filtro. 2) Una decisión sobre un aporte no sanciona automáticamente a autores distintos. 3) El ajuste de gravedad respeta el intervalo de 1 a 5.

HU14 Administrar cuentas y permisos

Como administrador, quiero mantener cuentas y permisos para asignar responsabilidades de revisión de forma controlada.

Criterios de aceptación. 1) El cliente de escritorio rechaza el acceso operativo de una cuenta ciudadana. 2) Los permisos se comprueban por acción y ámbito en el backend. 3) Un cambio de estado de la cuenta afecta las operaciones posteriores.

HU15 Mantener catálogo y configuración

Como administrador, quiero mantener tipos y parámetros para que el sistema aplique reglas conocidas y revisables.

Criterios de aceptación. 1) Los umbrales inválidos no sustituyen la configuración válida. 2) Un tipo desactivado no se ofrece como nuevo reporte disponible. 3) Los registros previos conservan su relación con el catálogo.

HU16 Resolver apelaciones y recuperaciones

Como administrador, quiero revisar apelaciones y recuperaciones para corregir decisiones justificadas y atender solicitudes de acceso.

Criterios de aceptación. 1) La decisión conserva su respuesta y estado. 2) La restauración de puntos evita duplicados del mismo aporte. 3) Las pruebas de recuperación no quedan expuestas en respuestas públicas.

HU17 Consultar estadísticas y exportar

Como administrador, quiero consultar datos y exportaciones con filtros para revisar la actividad del sistema con sus referencias.

Criterios de aceptación. 1) Un rol ciudadano no accede a exportaciones administrativas. 2) Los filtros del archivo corresponden a la consulta elegida. 3) La pérdida de sesión interrumpe el acceso protegido.

HU18 Administrar contenido del mapa

Como administrador, quiero mantener avisos, negocios e imágenes para publicar contenido vigente y revisar alertas externas.

Criterios de aceptación. 1) La ubicación del anuncio pertenece a las opciones admitidas. 2) El adjunto del anuncio conserva el acceso permitido para esa publicación. 3) Un aviso externo oculto por moderación se trata según su identificador de proveedor.

HU19 Importar antecedentes históricos

Como administrador, quiero revisar una importación antes de confirmarla para conservar antecedentes con fuente y estructura consistentes.

Criterios de aceptación. 1) La vista previa no publica por sí misma los registros. 2) El compromiso de importación exige autorización actual. 3) Los datos importados conservan fecha y fuente según el contrato.

HU20 Gestionar operaciones y plazos

Como administrador, quiero revisar plazos y cierres para mantener los estados y los incentivos de acuerdo con las reglas del sistema.

Criterios de aceptación. 1) Una operación protegida no se ejecuta con rol no autorizado. 2) El cierre conserva un identificador y datos de periodo. 3) Los plazos respetan el tipo y el estado del aporte.

13 Análisis de la solución planteada

13.1 Problemática de la empresa

El problema que atiende CiviGo es la falta de contexto suficiente para interpretar un aporte ciudadano dentro de un recorrido. La ubicación imprecisa, la repetición de publicaciones y la mezcla de hechos actuales con antecedentes dificultan esa interpretación. Por ello, el sistema debe conservar la fecha del hecho, relacionar aportes compatibles y distinguir el estado de revisión de la gravedad asignada.

En el componente de recorridos, el sistema necesita asociar los aportes con sectores concretos de la red vial. La subdivisión de aristas y la consulta por tramos permiten evitar que un incidente localizado determine por sí solo la representación de una vía extensa. El resultado sigue dependiendo de la cobertura y de la información registrada. [R08, R09]

13.2 Tecnologías a emplear

Las versiones indicadas son las declaradas en los manifiestos de la revisión del repositorio. Una dependencia expresada con rango puede resolverse a otra versión compatible en su archivo de bloqueo. [R01–R03]

Tabla 14 Tecnologías de los componentes

Componente	Tecnología declarada	Uso	Fuente
Frontend web	Next.js 16.3.8, React 19.2.8, TypeScript	Pantallas, navegación del cliente y acceso a la API.	R01
Estilos y mapa	Tailwind CSS con rango 4, CSS, Mapbox GL con rango ^3.31.0	Presentación de la web y capas cartográficas.	R01
Backend	Node.js declarado >=24 <25, Express ^5.2.1	API HTTP, validación y orquestación de reglas.	R02
Persistencia	Prisma y @prisma/client ^6.19.3, PostgreSQL	Modelo relacional, consultas y migraciones.	R02, R04
Escritorio	Electron 44.5.1, Vite ^8.3.2, React 19.2.8	Cliente Windows con renderer y proceso principal separados.	R03
Exportación	ExcelJS 4.4.0	Generación de archivos administrativos en escritorio.	R03, R22
Verificación de tokens	jose ^6.2.12 y Firebase ^12.19.0 en web	Comprobación del token de identidad de correo.	R01, R02, R05
Control de versiones	GitHub y GitHub Actions	Historial de cambios y verificaciones configuradas.	R17

Tabla 15 Servicios e integraciones

Servicio	Ubicación	Función	Condición
Mapbox	Frontend	Muestra el mapa y sus capas mediante un token público.	Su disponibilidad no equivale a la disponibilidad del motor de rutas.

Servicio	Ubicación	Función	Condición
OpenStreetMap	Backend y datos locales	Aporta la red vial y datos geográficos utilizados por las rutas.	La cobertura y actualidad dependen del conjunto importado.
TomTom	Backend	Aporta tráfico y avisos temporales, con uso aplicable al automóvil.	Requiere habilitación, clave privada y control de cuota.
OpenAI	Backend	Evalúa reportes y responde consultas del asistente.	Una evaluación dudosa o fallida conserva la revisión humana.
Google Cloud Vision	Backend	Busca coincidencias web de imágenes del reporte.	Una coincidencia no demuestra falsedad. Su ausencia no acredita autoría.
Firebase Authentication	Web y backend	Verifica el correo coincidente de la cuenta con Google.	No sustituye la sesión propia de CiviGo.
Resend	Backend	Envía correos y recordatorios según configuración.	No es el mecanismo de verificación con Google.
Supabase	Persistencia y archivos	Puede alojar PostgreSQL y el bucket privado de adjuntos.	La API debe conservar autorización por usuario y ámbito.
Vercel y Railway	Configuración de despliegue	Destinos previstos para frontend y backend en los módulos de integración.	La configuración versionada no es una medición de disponibilidad en producción.

Google verifica el correo; la sesión sigue siendo propia de CiviGo. Resend envía correos y recordatorios. El token público de Mapbox se utiliza en la web, mientras que las credenciales privadas se configuran en el backend. [R05, R21]

13.3 Descripción de la empresa

El equipo CiviGo asume el desarrollo y la documentación académica de la solución. El ciudadano es la fuente de los reportes que presenta. El agente revisa según sus permisos y el administrador mantiene las cuentas y los parámetros del servicio. Esta distribución permite separar la contribución ciudadana del poder de validar, sancionar o modificar el catálogo.

13.4 Alcance de la solución del proyecto

La solución utiliza un backend modular y dos clientes. Las rutas de Express delegan las reglas en módulos de dominio y Prisma accede al modelo común de PostgreSQL. Esta estructura permite aplicar la misma autorización y mantener relaciones coherentes entre cuentas, aportes y decisiones. Los conectores encapsulan la comunicación con servicios externos. [R02, R04, R21]

El administrador trabaja en el escritorio y el agente en la web. En Electron, las operaciones privilegiadas pasan por el proceso principal mediante un contrato limitado. El cierre debe comprobar esa frontera, los contratos con la API y la consistencia del mapa. El alcance excluye los pagos reales y la aplicación móvil independiente. [R16]

14 Especificaciones de los casos de uso

Las especificaciones precisan el actor, las condiciones previas, el flujo principal y las alternativas que afectan el resultado. Los permisos se verifican en el servidor, incluso cuando la interfaz oculta una acción. Cada caso conserva su vínculo con los requisitos y con la historia del mismo número. La postcondición describe el estado que debe mantenerse al finalizar correctamente la operación.

CU01 Registrar y habilitar el acceso

Actor: Visitante y ciudadano. Requisitos: RF01, RF02, RF03. Historia: HU01. [R05]

Condición previa. El visitante dispone de un correo Gmail y de los datos solicitados por el registro. El servicio de verificación debe estar configurado para completar ese paso.

1. El visitante registra sus datos y una contraseña válida.
2. La API valida formatos y duplicados, calcula el hash y crea la cuenta.
3. El ciudadano inicia sesión y recibe una sesión propia de CiviGo.
4. La web solicita la verificación con Google y remite el token al backend.
5. El servidor comprueba la firma, el proyecto y la coincidencia del correo antes de habilitar la participación.

Alternativas. Los datos duplicados o inválidos se rechazan. Un token de otro correo no verifica la cuenta. Si Google no está disponible, la cuenta conserva su condición pendiente.

Postcondición. La cuenta conserva su estado real de verificación. El cierre de sesión invalida el acceso que corresponde.

CU02 Gestionar perfil y recuperación

Actor: Ciudadano. Requisitos: RF04. Historia: HU02. [R05]

Condición previa. La edición del perfil requiere sesión vigente. La recuperación utiliza el procedimiento disponible y su validación correspondiente.

1. El usuario consulta sus datos privados en el perfil.
2. Presenta los cambios permitidos o solicita la recuperación de acceso.
3. El servidor valida identidad, formatos y conflictos de datos.
4. La operación actualiza la cuenta o registra la solicitud para su revisión.
5. Un cambio de contraseña aplica el hash y los efectos previstos sobre las sesiones.

Alternativas. Un correo o teléfono ya utilizado genera conflicto. Una solicitud de recuperación que requiere revisión conserva estado pendiente. Una sesión inválida no modifica el perfil.

Postcondición. Solo se guardan cambios autorizados y la cuenta conserva un estado de acceso coherente.

CU03 Consultar mapa e incidentes

Actor: Visitante y ciudadano. Requisitos: RF05. Historia: HU03. [R07]

Condición previa. La API dispone de datos de cobertura. La cartografía necesita su configuración para mostrarse.

1. El usuario abre el mapa y establece los filtros disponibles.
2. La web consulta incidentes publicados y sus datos de presentación.
3. El backend aplica las condiciones de publicación y privacidad.
4. El mapa representa los resultados y permite abrir el detalle.
5. El detalle muestra estado, descripción y acciones correspondientes a la cuenta.

Alternativas. La ausencia de registros se presenta como un resultado de consulta. Un incidente no público no se entrega como detalle público. Los problemas de red muestran un error identificable.

Postcondición. El usuario consulta información publicada sin recibir archivos ni datos personales reservados.

CU04 Registrar y evaluar un reporte

Actor: Ciudadano verificado. Requisitos: RF06, RF07. Historia: HU04. [R06]

Condición previa. La cuenta está habilitada y verificada. El tipo existe y está activo. Los adjuntos deben pertenecer al usuario y estar disponibles.

1. El ciudadano selecciona el tipo, ubica el hecho y proporciona los datos exigidos.
2. La API comprueba cobertura, fechas, GPS, archivos y límites de envío.
3. Los adaptadores procesan la evaluación y el origen de imágenes cuando están disponibles.
4. En una transacción, el backend vuelve a comprobar la cuenta y crea o vincula Incident y Report.
5. El sistema vincula los archivos y registra el resultado y las notificaciones de revisión.

Alternativas. Un reporte que necesita revisión queda EN_REVISION. Una emergencia puede publicarse con evaluación pendiente. Un archivo utilizado por otro aporte o una cuota excedida impide completar el envío.

Postcondición. El aporte tiene autoría y estado persistidos. La publicación del incidente refleja el resultado permitido por las reglas.

CU05 Dar seguimiento a un aporte

Actor: Autor del reporte. Requisitos: RF08. Historia: HU05. [R06]

Condición previa. Existe un reporte de la cuenta. El envío de evidencia o apelación requiere cumplir las condiciones del estado y del tipo.

1. El ciudadano abre Mis reportes.
2. La API recupera los aportes de la cuenta y su contexto autorizado.
3. El usuario consulta el estado y selecciona la acción permitida.
4. El servidor vincula la nueva evidencia o registra la apelación con su motivo.
5. La interfaz muestra el estado de la solicitud y la información disponible.

Alternativas. La API no acepta actuar sobre un aporte ajeno. Los archivos deben conservar la autorización y los límites de carga. Una solicitud que requiere decisión humana permanece pendiente.

Postcondición. La evidencia o apelación queda vinculada al reporte correcto sin sustituir su historial.

CU06 Participar en un incidente

Actor: Ciudadano verificado. Requisitos: RF09, RF10. Historia: HU06. [R07]

Condición previa. El incidente permite participación. La cuenta está habilitada y el tipo no restringe la acción solicitada.

1. El usuario abre el detalle del incidente.
2. Selecciona confirmar, proponer resolución, enviar aviso o escribir un mensaje.
3. La API comprueba estado, verificación, proximidad y participación previa.
4. El servidor conserva el voto, aviso o mensaje y actualiza los efectos que correspondan.
5. La interfaz refresca el detalle y su estado.

Alternativas. El autor inicial no confirma su propio incidente. El voto duplicado se rechaza. Los hechos individuales no reciben votos comunitarios y el chat respeta las reglas de vigencia.

Postcondición. La contribución queda asociada al usuario y al incidente, con restricciones de duplicación y acceso.

CU07 Consultar riesgo por tramos

Actor: Visitante y ciudadano. Requisitos: RF11. Historia: HU07. [R08]

Condición previa. Existe una red vial cubierta y un conjunto de incidentes elegibles para el cálculo.

1. La web solicita la geometría y el riesgo de las vías.
2. El backend obtiene los segmentos y los incidentes publicados elegibles.
3. La lógica calcula puntos según gravedad, validación y antigüedad.
4. El sistema atribuye los aportes a segmentos y calcula niveles de 0 a 5.
5. El mapa representa esos niveles junto a su leyenda y estados pendientes.

Alternativas. Los incidentes falsos, retirados o no publicados no aportan puntos. Un incidente sin gravedad evaluada conserva condición pendiente. La falta de datos no acredita seguridad.

Postcondición. La capa representa el resultado de las reglas y su contexto temporal, sin sustituir la gravedad de cada hecho.

CU08 Planificar recorrido e itinerario

Actor: Ciudadano. Requisitos: RF12, RF13. Historia: HU08. [R09]

Condición previa. La cuenta tiene sesión y el origen y destino se encuentran en la cobertura utilizable.

1. El usuario busca el destino y selecciona caminar, bicicleta o automóvil.
2. La API valida las coordenadas y consulta la red, las restricciones y los incidentes.
3. El motor calcula candidatos y combina condiciones de tráfico cuando son aplicables y están disponibles.
4. La web presenta las alternativas, duración estimada y advertencias.
5. El usuario elige la ruta y puede incorporar lugares al itinerario.

Alternativas. Los puntos fuera de cobertura o sin conexión vial producen un resultado controlado. Si el tráfico no está disponible, no se presenta como dato actualizado. La duración continúa siendo estimada.

Postcondición. El usuario dispone de alternativas y decide el recorrido. No se registra una garantía de seguridad ni de tiempo exacto.

CU09 Guardar y consultar recorridos

Actor: Ciudadano. Requisitos: RF14. Historia: HU09. [R09]

Condición previa. La cuenta tiene sesión y existe una ruta o una solicitud válida de favorito.

1. El usuario solicita guardar un favorito o registrar un recorrido.
2. La API comprueba titularidad, estructura y límites de la cuenta.
3. La operación conserva origen, destino, modo y datos del recorrido.
4. El usuario consulta su lista, historial o estadísticas habilitadas.
5. El sistema aplica las opciones permitidas para ese perfil.

Alternativas. Los límites de favoritos o historial se aplican en el servidor. Las opciones Premium se mantienen dentro de la condición de demostración. Una cuenta no consulta los recorridos de otra.

Postcondición. Los datos personales de rutas quedan vinculados a su propietario.

CU10 Consultar notificaciones

Actor: Ciudadano o personal autorizado. Requisitos: RF15. Historia: HU10. [R11]

Condición previa. La cuenta tiene sesión y existen notificaciones dirigidas a ella.

1. El usuario abre la sección de notificaciones.
2. La API recupera los avisos de la cuenta.
3. La interfaz muestra título, mensaje y estado de lectura.
4. El usuario marca una notificación como leída.
5. El servidor actualiza únicamente el aviso de su titular.

Alternativas. Una notificación ajena no se modifica. La ausencia de avisos se muestra sin convertirla en un error de operación.

Postcondición. La lectura queda registrada sin alterar la decisión que dio origen al aviso.

CU11 Consultar incentivos y solicitar canje

Actor: Ciudadano verificado. Requisitos: RF16. Historia: HU11. [R11]

Condición previa. La cuenta está habilitada. El canje requiere correo verificado, saldo suficiente y recompensa activa con stock.

1. El usuario consulta ranking, participación o recompensas.
2. La API presenta los datos y su carácter demostrativo.
3. El usuario solicita un canje disponible.
4. Una transacción comprueba y descuenta saldo y stock.
5. El sistema registra la solicitud de canje en estado de demostración.

Alternativas. Un saldo insuficiente o la falta de stock impide el canje. Los puntos de un aporte no se duplican por repetir una operación de validación.

Postcondición. El canje conserva su costo y su estado demostrativo. Los registros permiten revisar el origen de los puntos.

CU12 Consultar el asistente de CiviGo

Actor: Usuario de la web. Requisitos: RF17. Historia: HU12. [R21]

Condición previa. La solicitud cumple las condiciones de acceso y cuota de la API. El proveedor del asistente puede estar configurado o no.

1. El usuario escribe una consulta relacionada con CiviGo.
2. La API valida el mensaje y limita el contexto enviado.
3. El adaptador consulta el modelo cuando está disponible.
4. La respuesta se limita a la información y reglas públicas permitidas.
5. La web presenta la respuesta o el mensaje de indisponibilidad correspondiente.

Alternativas. Una solicitud fuera del ámbito recibe el tratamiento definido. Una falla de proveedor no permite afirmar que se publicó, confirmó o modificó información.

Postcondición. Se devuelve una orientación textual sin ejecutar por sí misma operaciones de reporte o administración.

CU13 Revisar incidentes y aportes

Actor: Agente con permiso o administrador. Requisitos: RF18. Historia: HU13. [R07, R10]

Condición previa. El actor tiene sesión, permiso vigente y ámbito aplicable. La revisión de pruebas privadas exige el permiso correspondiente.

1. El revisor consulta su centro de trabajo y abre el aporte.
2. La API aplica ámbito, permisos y autorización de evidencias.
3. El revisor selecciona la acción y expresa su motivo y gravedad cuando corresponde.
4. La transacción vuelve a consultar el actor y actualiza estados y efectos autorizados.
5. El sistema conserva revisión, auditoría y avisos asociados.

Alternativas. La revocación del permiso impide completar la acción. Un incidente agrupado no permite sancionar indiscriminadamente a todos sus autores. Un estado final puede exigir reapertura o apelación previa.

Postcondición. La decisión conserva autor, motivo y efecto sobre las entidades correspondientes.

CU14 Administrar cuentas y permisos

Actor: Administrador de escritorio. Requisitos: RF19. Historia: HU14. [R10, R16]

Condición previa. El programa de escritorio tiene una sesión ADMIN aceptada por la API.

1. El administrador consulta la lista de usuarios con sus filtros.
2. Abre la edición o el alta autorizada de una cuenta.
3. Define los datos, rol, ámbito y permisos que corresponden.
4. El proceso principal transmite una solicitud permitida a la API.
5. El backend valida la operación y registra el cambio y su auditoría.

Alternativas. Una cuenta sin rol ADMIN no mantiene la sesión administrativa. Los datos duplicados y las combinaciones inválidas se rechazan. La interfaz no sustituye la validación del servidor.

Postcondición. La cuenta conserva permisos válidos y el cambio puede rastrearse.

CU15 Mantener catálogo y configuración

Actor: Administrador de escritorio. Requisitos: RF20. Historia: HU15. [R10, R14]

Condición previa. Existe sesión administrativa y los nuevos valores cumplen los validadores.

1. El administrador consulta categorías, tipos o reglas.
2. Presenta un cambio y sus valores.
3. La API comprueba campos, referencias y rangos aceptados.
4. El backend guarda la configuración o el catálogo y registra la operación.
5. Las consultas posteriores utilizan los datos persistidos.

Alternativas. Una configuración fuera de rango se rechaza. Desactivar un tipo no debe borrar los antecedentes que lo referencian. Las referencias existentes se conservan según las restricciones.

Postcondición. El catálogo y las reglas mantienen una versión persistida que puede consultarse.

CU16 Resolver apelaciones y recuperaciones

Actor: Administrador de escritorio. Requisitos: RF21. Historia: HU16. [R10]

Condición previa. Existe una solicitud pendiente y una cuenta administrativa vigente. Los archivos requieren acceso autorizado.

1. El administrador consulta la solicitud y su contexto.
2. Revisa motivo, aporte y pruebas accesibles.
3. Selecciona una decisión y registra la respuesta exigida.
4. La API aplica los cambios y restaura o conserva estados y puntos según la decisión.
5. El sistema mantiene auditoría y comunica el resultado.

Alternativas. Una decisión no compatible con el estado actual se rechaza. La aceptación de una apelación no crea una segunda recompensa por el mismo aporte. Una prueba no autoriza por sí sola cualquier cambio de identidad.

Postcondición. La solicitud queda atendida de manera consistente con sus entidades y con el historial del caso.

CU17 Consultar estadísticas y exportar

Actor: Administrador de escritorio. Requisitos: RF22. Historia: HU17. [R22, R16]

Condición previa. La cuenta administrativa está vigente. El backend incluye el contrato de estadísticas o exportación solicitado.

1. El administrador selecciona un conjunto de datos y sus filtros.
2. El escritorio transmite la consulta permitida a la API.
3. El backend valida el alcance y produce el conjunto autorizado.
4. La interfaz muestra estadísticas o prepara una exportación.
5. El programa solicita el destino de guardado cuando el usuario exporta el archivo.

Alternativas. Una sesión revocada impide guardar una exportación autorizada con una sesión anterior. Un contrato no disponible debe informarse. La auditoría mantiene la reserva de sus datos.

Postcondición. La consulta o el archivo corresponde al filtro y a la autorización vigentes.

CU18 Administrar contenido del mapa

Actor: Administrador de escritorio. Requisitos: RF23. Historia: HU18. [R13]

Condición previa. Existe sesión ADMIN. Los anuncios cumplen las reglas de tipo, ubicación, fechas y archivos.

1. El administrador consulta el contenido y elige crear, modificar, retirar o moderar.
2. Registra texto, vigencia y la imagen o referencia permitida.
3. La API valida reglas y propiedad del adjunto.
4. El sistema guarda el contenido o la decisión de moderación.
5. La web consulta únicamente contenido visible para su contexto.

Alternativas. Una imagen privada o vinculada a un reporte no se reutiliza como anuncio. Un intervalo de fechas inválido se rechaza. Los avisos de tráfico no se convierten en riesgo histórico por publicarse.

Postcondición. El contenido visible cumple las condiciones persistidas y la moderación conserva su motivo.

CU19 Importar antecedentes históricos

Actor: Administrador de escritorio. Requisitos: RF24. Historia: HU19. [R15]

Condición previa. El archivo o conjunto de entrada cumple el contrato de importación y la cuenta tiene permiso administrativo.

1. El administrador presenta los registros para una vista previa.
2. La API valida campos y construye el resultado de revisión.
3. La interfaz permite examinar registros aceptables y observaciones.
4. El administrador confirma el conjunto que corresponde.
5. El servidor persiste el resultado y sus referencias de origen.

Alternativas. Los registros inválidos deben mostrar sus observaciones. Una vista previa no equivale a una importación confirmada. Las fechas deben representar el hecho histórico y no la carga como si fuera un incidente actual.

Postcondición. Los antecedentes confirmados conservan su tratamiento histórico y su fuente.

CU20 Gestionar operaciones y plazos

Actor: Administrador y tarea programada del backend. Requisitos: RF25. Historia: HU20. [R10, R11]

Condición previa. La acción manual tiene sesión y permiso ADMIN. Las tareas automáticas usan la configuración del servidor.

1. El proceso programado o el administrador solicita una operación de ciclo de vida o cierre.
2. El backend identifica los registros y periodos correspondientes.
3. La lógica aplica las reglas y evita repetir premios o cierres ya registrados.
4. El sistema actualiza los estados y emite los avisos previstos.
5. El administrador consulta los registros de la operación.

Alternativas. Un fallo debe conservar un estado recuperable y no convertir la operación en un cierre confirmado. La repetición de una solicitud no justifica duplicar puntos.

Postcondición. Los cambios y cierres permanecen vinculados con sus registros de control.

15 Diagramas de análisis del sistema

15.1 Arquitectura del sistema

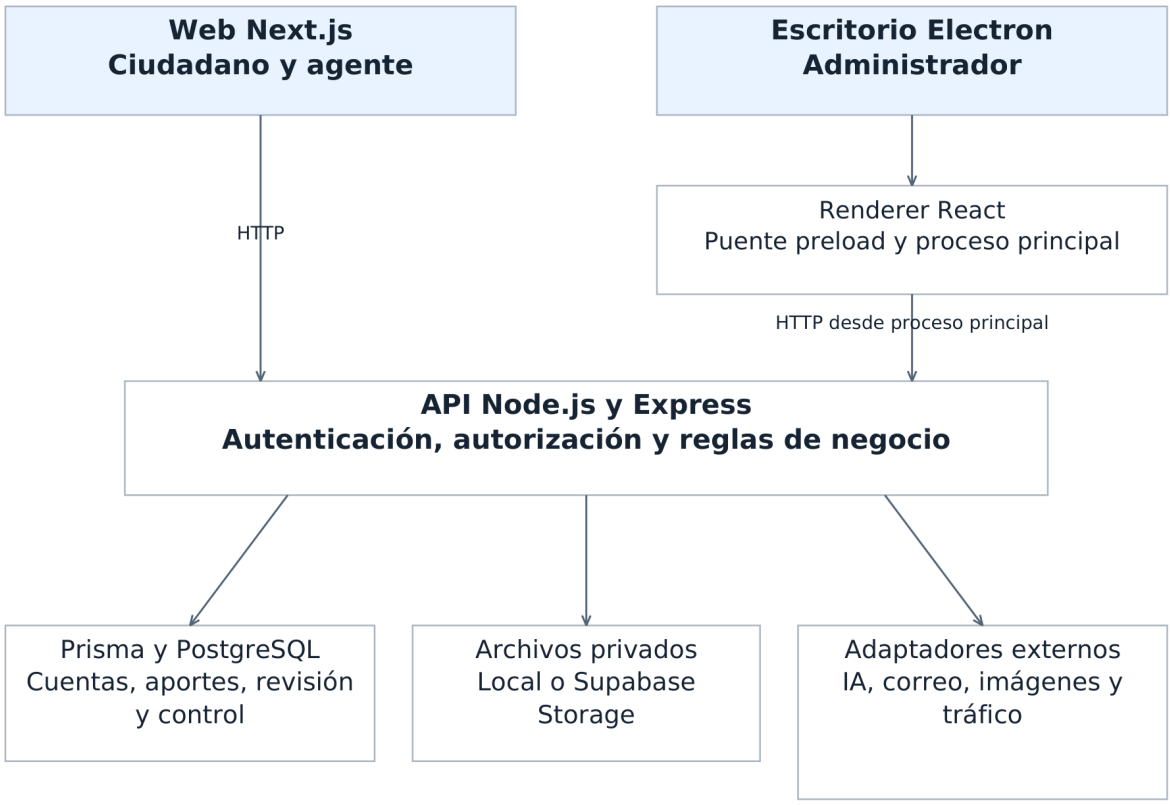


Figura 7 Arquitectura de CiviGo

Los dos clientes utilizan la API. Las claves privadas y el acceso a los datos se mantienen en el servidor. Fuentes R01–R04, R16 y R21.

El cliente administrativo contiene un renderer de React aislado del proceso principal de Electron. La comunicación utiliza una interfaz limitada en preload. El proceso principal valida el origen de las solicitudes, conserva la sesión en memoria y ejecuta las peticiones permitidas. El servidor vuelve a comprobar el rol administrador. [R16]

En la web, los componentes organizan las pantallas y los módulos de acceso centralizan las solicitudes a la API. En el backend se concentran autenticación, autorización, validación de entradas y transacciones. Este reparto permite que una regla de publicación o revisión tenga la misma aplicación para todos los clientes.

15.2 Realización de los casos de uso y diagramas de secuencia y comunicación

Los diagramas siguientes representan colaboraciones de los casos principales. Los pasos internos agrupan operaciones del mismo módulo cuando esa simplificación permite leer la secuencia sin alterar sus dependencias. Los demás casos utilizan las mismas fronteras entre interfaz, API, dominio y persistencia.

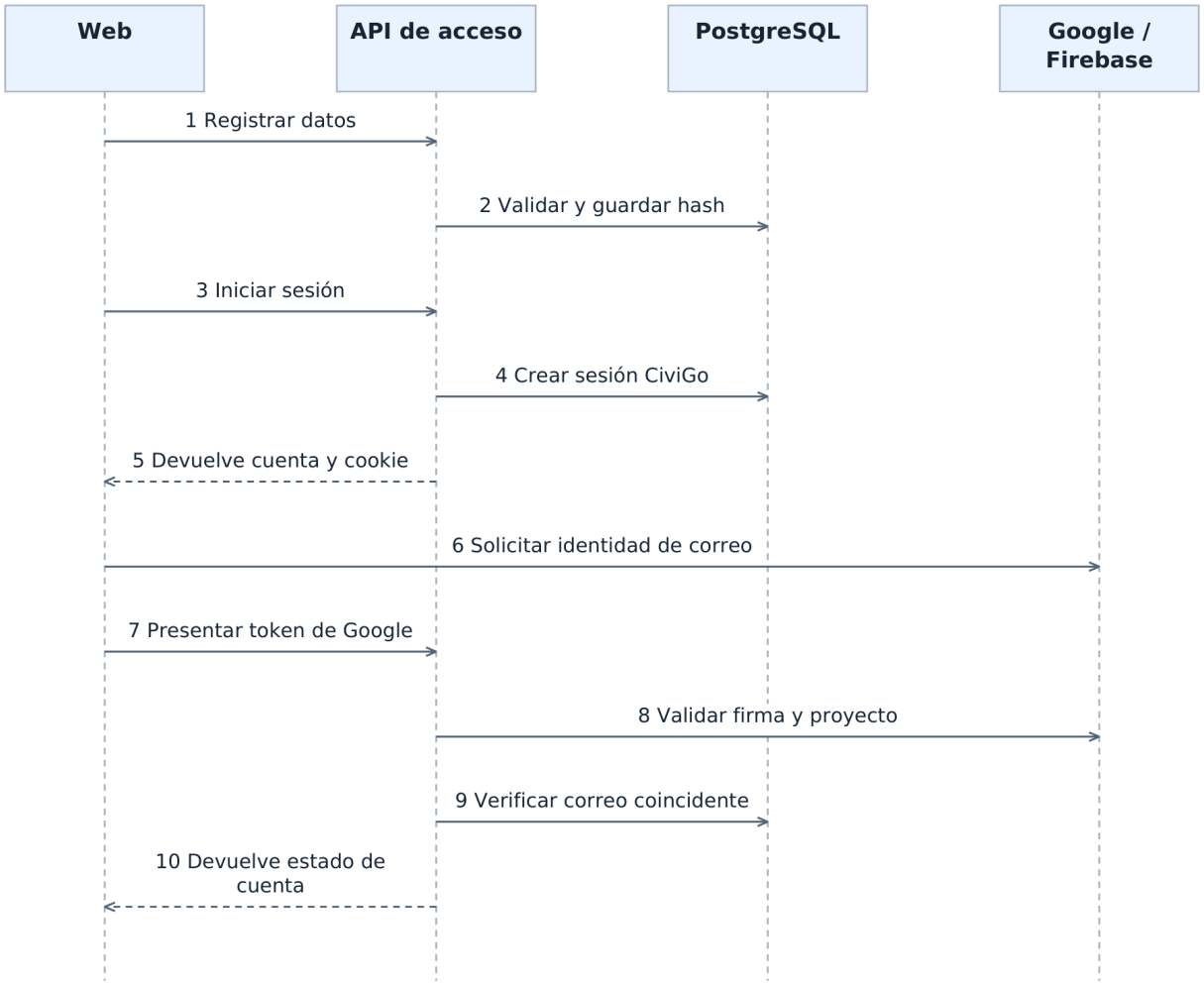


Figura 8 Secuencia de acceso y verificación

CU01. El inicio de sesión crea una sesión propia. La verificación posterior comprueba el token de Google y su correspondencia con el correo. Fuente R05.

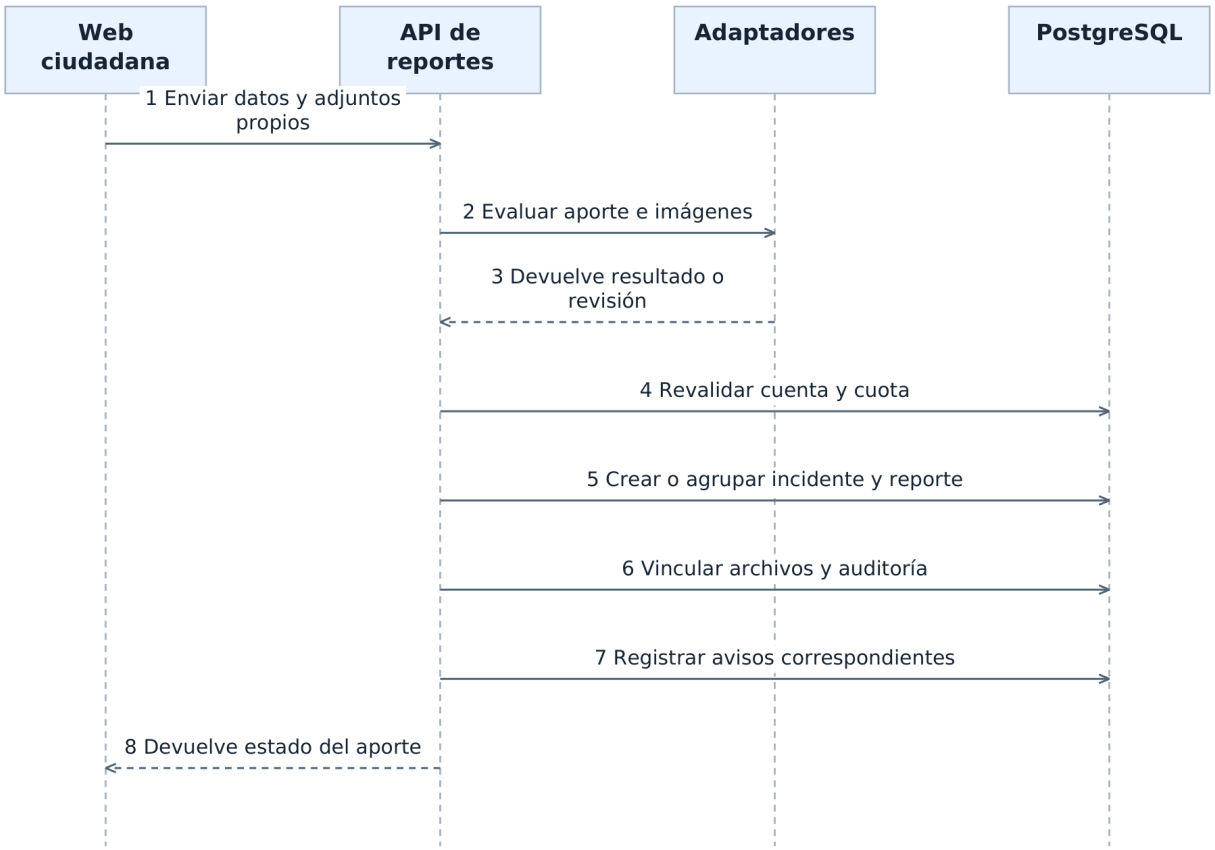


Figura 9 Secuencia de registro de reporte

CU04. El análisis externo precede a la transacción que vuelve a comprobar la cuenta y persiste el resultado. Fuente R06.

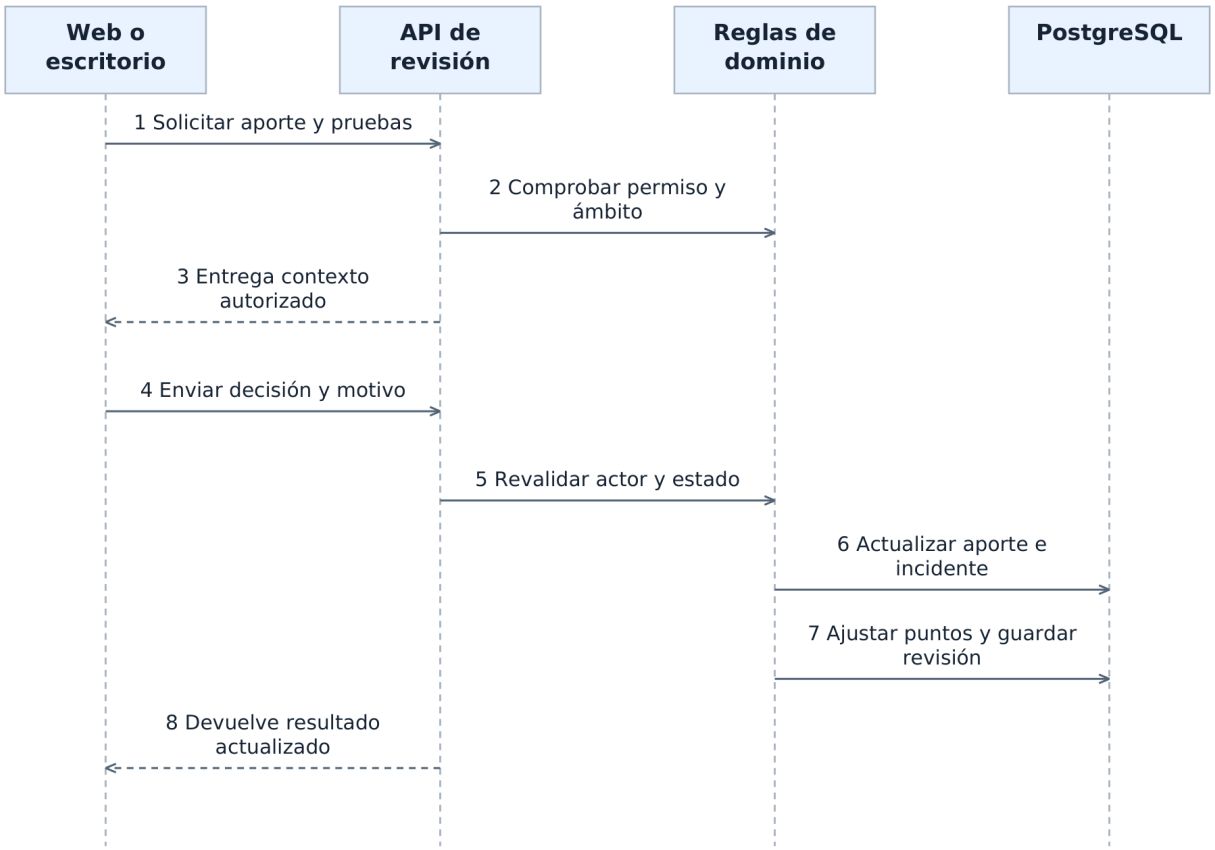


Figura 10 Secuencia de revisión de un aporte

CU13 y CU16. La decisión incluye el motivo y las comprobaciones de permiso y pruebas privadas. Fuentes R07 y R10.

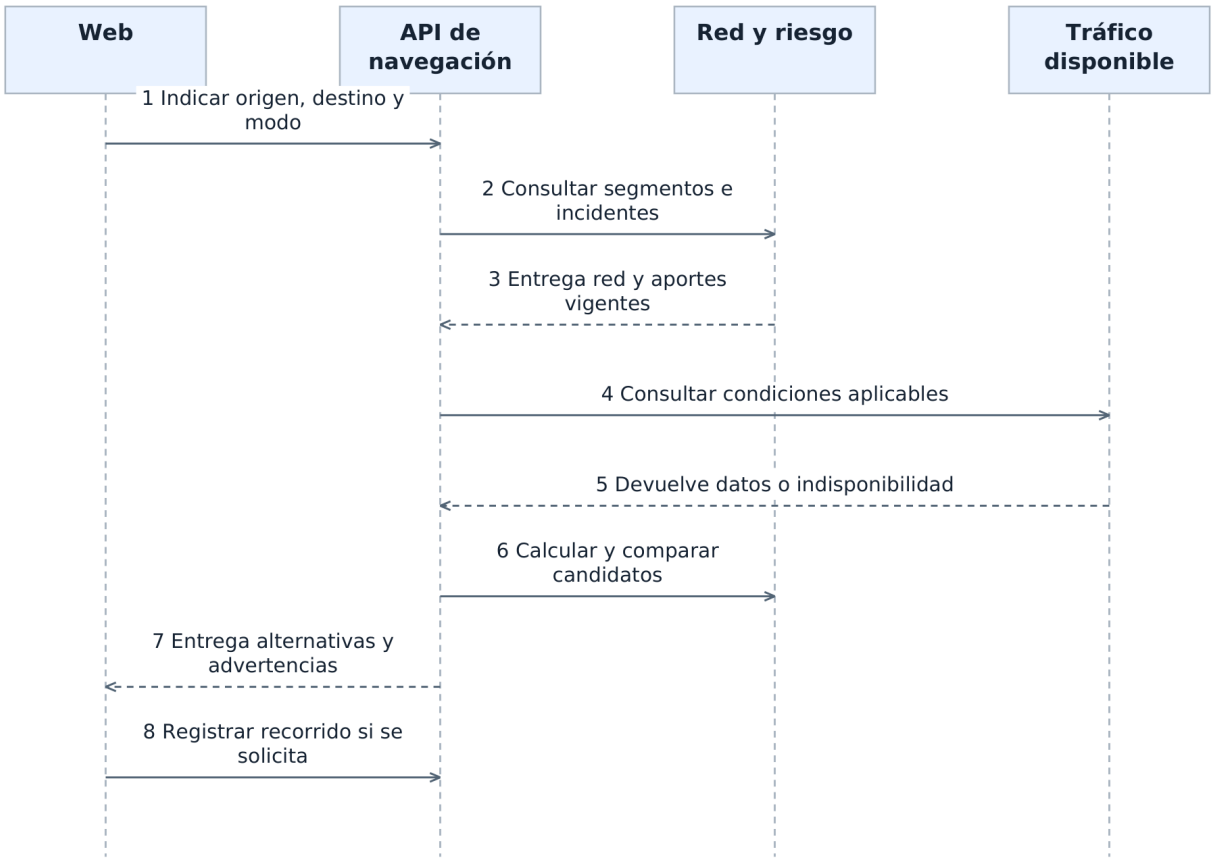


Figura 11 Secuencia de cálculo de rutas

CU08. La API combina red vial, incidentes y condiciones de tráfico disponibles antes de devolver alternativas. Fuente R09.

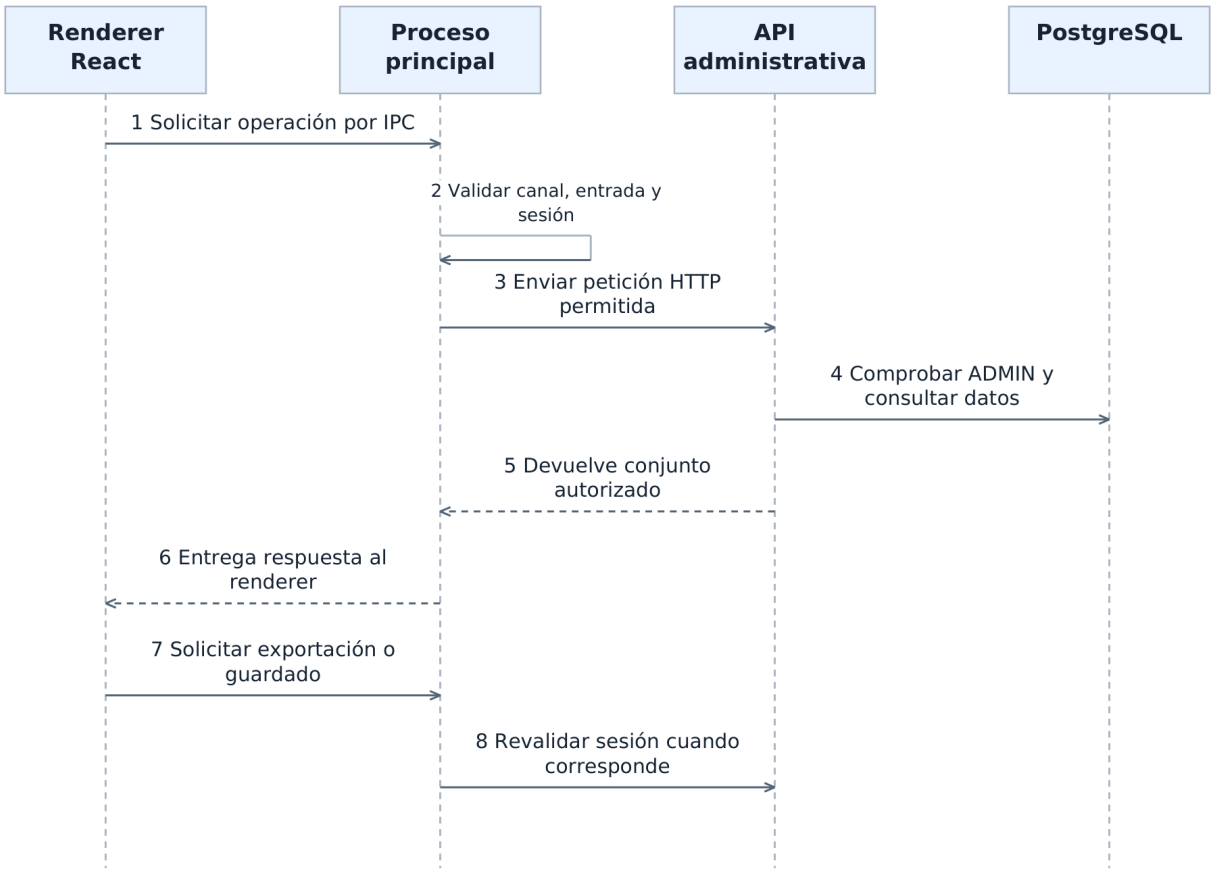


Figura 12 Secuencia de operación administrativa

CU14 y CU17. El renderer solicita la operación a través del puente autorizado. Fuentes R16 y R10.

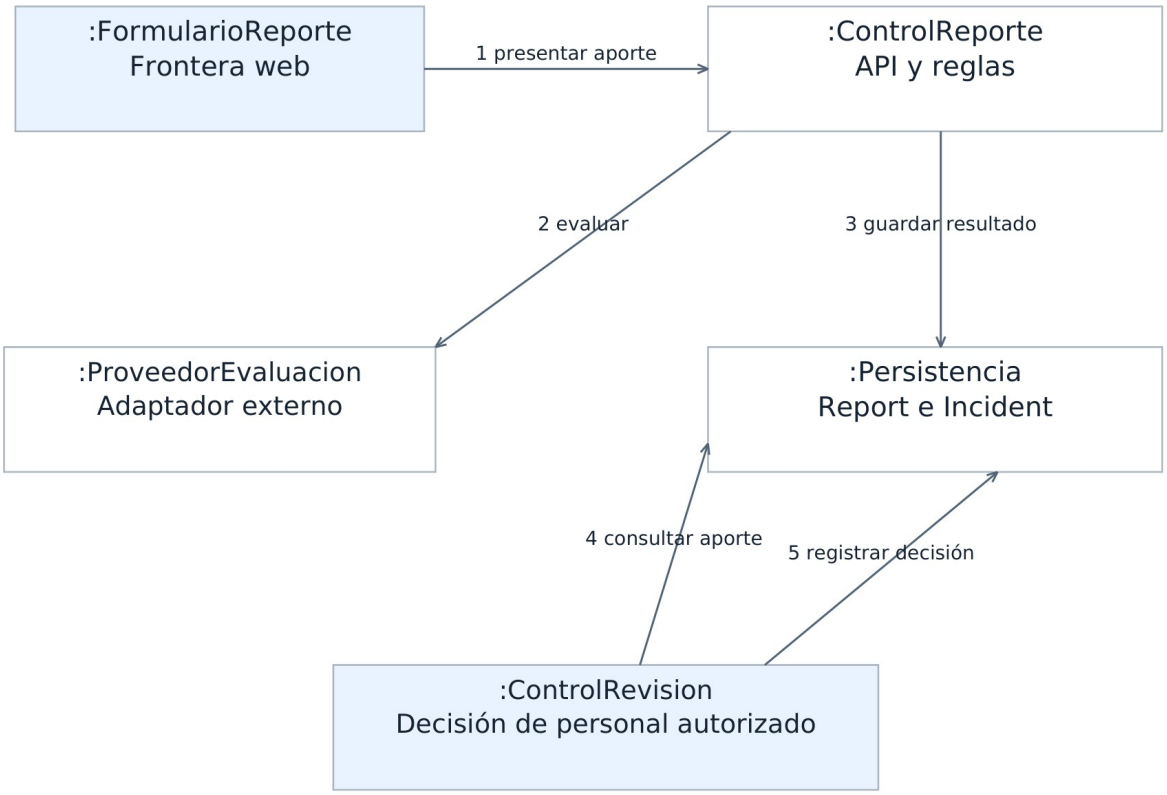


Figura 13 Comunicación para la publicación y revisión

La numeración muestra el orden de colaboración entre los objetos de frontera, control y entidad. Fuentes R06 y R07.

Tabla 16 Realización de casos complementarios

Casos relacionados	Colaboración de implementación
CU02, CU05, CU10	Perfil y aportes propios consultan rutas de usuarios, reportes y notificaciones con la sesión actual.
CU03, CU06, CU07	Mapa y participación comparten criterios de elegibilidad pública y el estado persistido del incidente.
CU09, CU11, CU12	Servicios personales, incentivos y asistente utilizan contratos separados y las cuotas de su operación.
CU15, CU18, CU19, CU20	Las herramientas del escritorio llaman a contratos administrativos para catálogo, contenido, importaciones y mantenimiento.

16 Estructura del software

El repositorio separa el código por cliente y servidor. La siguiente estructura presenta los directorios que intervienen en el alcance del informe y su responsabilidad. Los módulos de dominio no constituyen servicios desplegados de forma independiente.

Tabla 17 Organización del código

Directorio	Responsabilidad
frontend/app	Rutas y páginas de la web, incluidos mapa, reportes, perfil y agente.
frontend/components	Controles reutilizables, mapa, rutas y centro de revisión del agente.
frontend/lib	Contratos, sesiones, búsqueda, navegación y reglas de presentación.
backend/src/routes	Endpoints agrupados por área funcional y controles de entrada.
backend/src/lib	Reglas de dominio, geografía, autorización y adaptadores de servicios.
backend/prisma	Modelo de datos y migraciones PostgreSQL.
backend/scripts	Carga inicial, importaciones y herramientas operativas autorizadas.
desktop/src	Interfaz React, estilos y paneles del administrador.
desktop/electron	Proceso principal, puente, cliente de API y políticas de la aplicación.
.github/workflows	Verificaciones configuradas para los componentes del repositorio.
compose.yaml y deploy	Configuración de contenedores y referencia de proxy del sistema web.

La ruta administrativa de la web mantiene el espacio del agente. El conjunto más amplio de herramientas del administrador se encuentra en desktop/src/components/management. Las solicitudes de ese cliente pasan por desktop/electron/client.cjs y por las validaciones de desktop/electron/policy.cjs. [R03, R16]

16.1 Patrones de diseño

La organización del código utiliza adaptadores de proveedor, funciones de creación de dependencias y controles compartidos de autorización. Estos mecanismos separan responsabilidades que cambian por razones distintas, como la comunicación con un servicio externo y la decisión de publicar un aporte. El acceso mediante Prisma se describe como persistencia; el repositorio revisado no exige una capa Repository independiente.

Tabla 18 Mecanismos de diseño identificados

Mecanismo	Implementación	Aplicación	Fuente
Adaptadores de proveedor	createStorage y createTomTomClient	Presentan operaciones propias a la aplicación y encapsulan HTTP, configuración y errores de cada proveedor.	R09, R12
Funciones de creación e inyección	createFirebaseEmailVerifier y createImageWebCheck	Permiten recibir dependencias como transporte, reloj o resolutores para separar integración y lógica.	R05, R21
Middleware de autorización	auth, requireEmail y requirePermission	Encadenan comprobaciones antes del controlador de Express y comparten criterios de acceso.	R05, R10
Frontera entre procesos	preload y canales IPC del escritorio	Exponen un contrato limitado al renderer y validan en el proceso principal las acciones autorizadas.	R16
Unidad transaccional	transaction en workflows.js	Agrupa cambios de estado y efectos asociados, con reintento acotado para conflictos serializables.	R07
Fachada de acceso del cliente	lib/api.ts y ApiClient de escritorio	Centralizan solicitudes y errores para que los componentes consuman un contrato común.	R16, R20

17 Diagramas de modelo de datos

El esquema Prisma de la revisión contiene 29 modelos persistentes. El modelo conceptual presenta los conceptos del dominio. El lógico incorpora entidades y relaciones. El físico resume su representación en PostgreSQL. Los diagramas se dividen por grupos para conservar la lectura y comparten los mismos nombres de entidades. [R04]

17.1 Diagrama conceptual

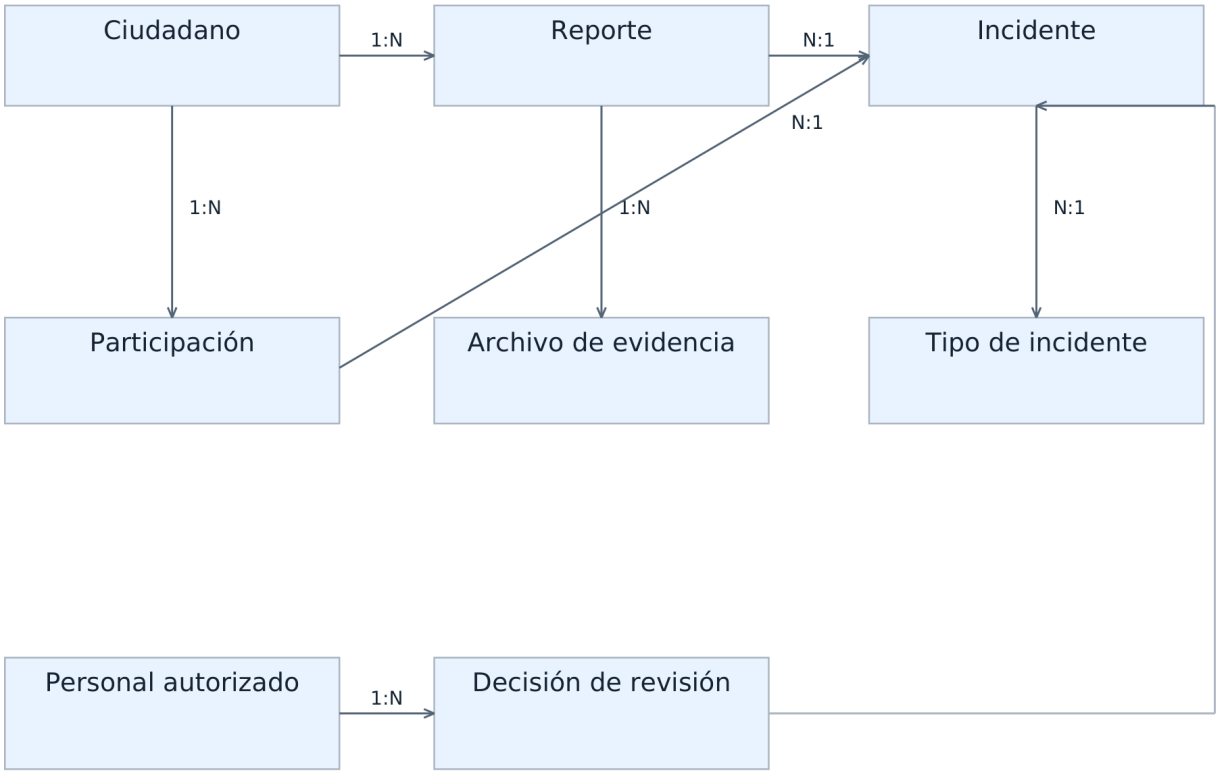


Figura 14 Modelo conceptual de participación y revisión

Report representa el aporte e Incident representa el evento consultado. Elaboración a partir de R04.

17.2 Diagrama lógico

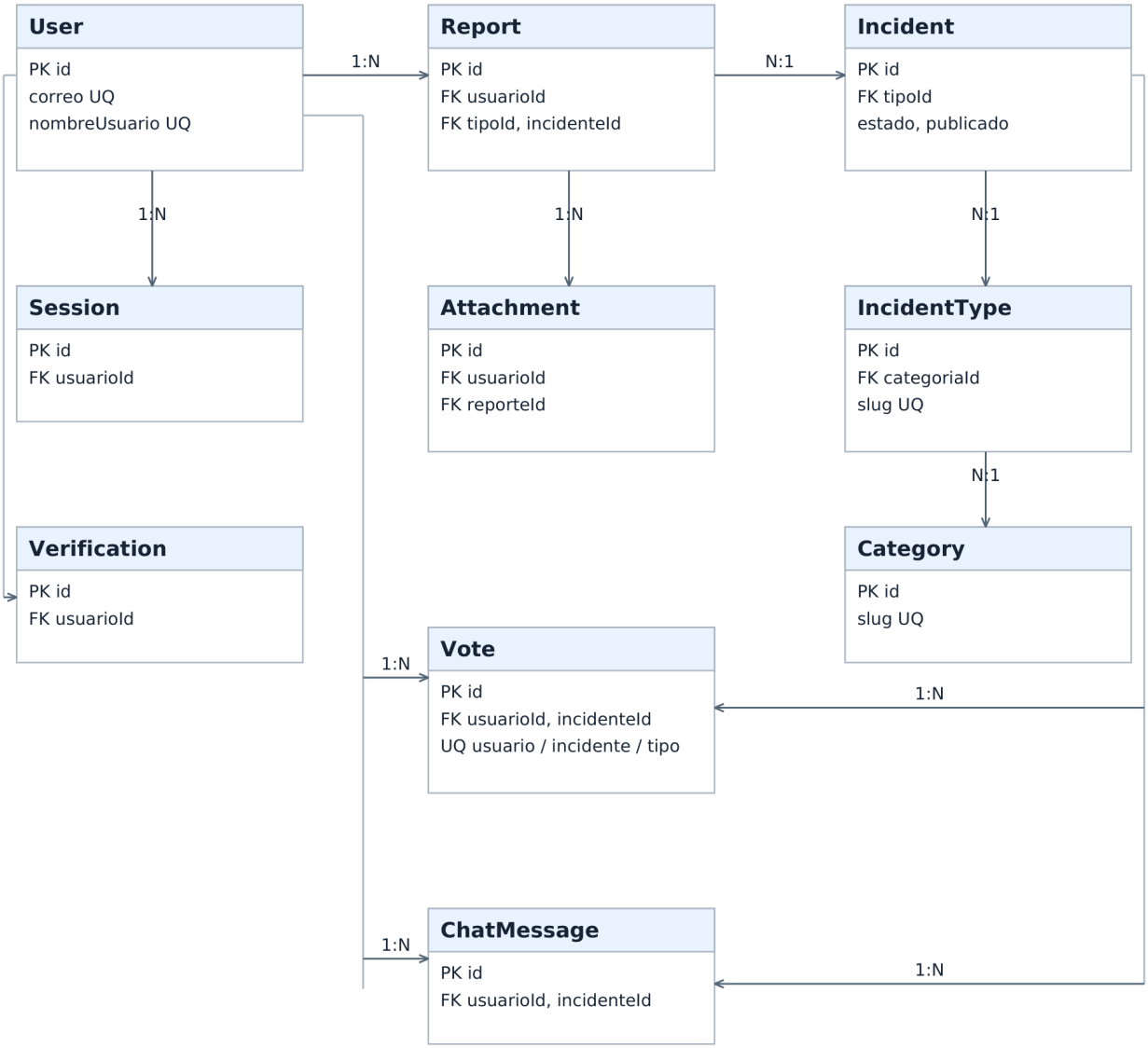


Figura 15 Modelo lógico de cuentas e incidentes

Se muestran las relaciones declaradas en Prisma. La obligatoriedad de una relación depende de si su clave admite nulos. Fuente R04.

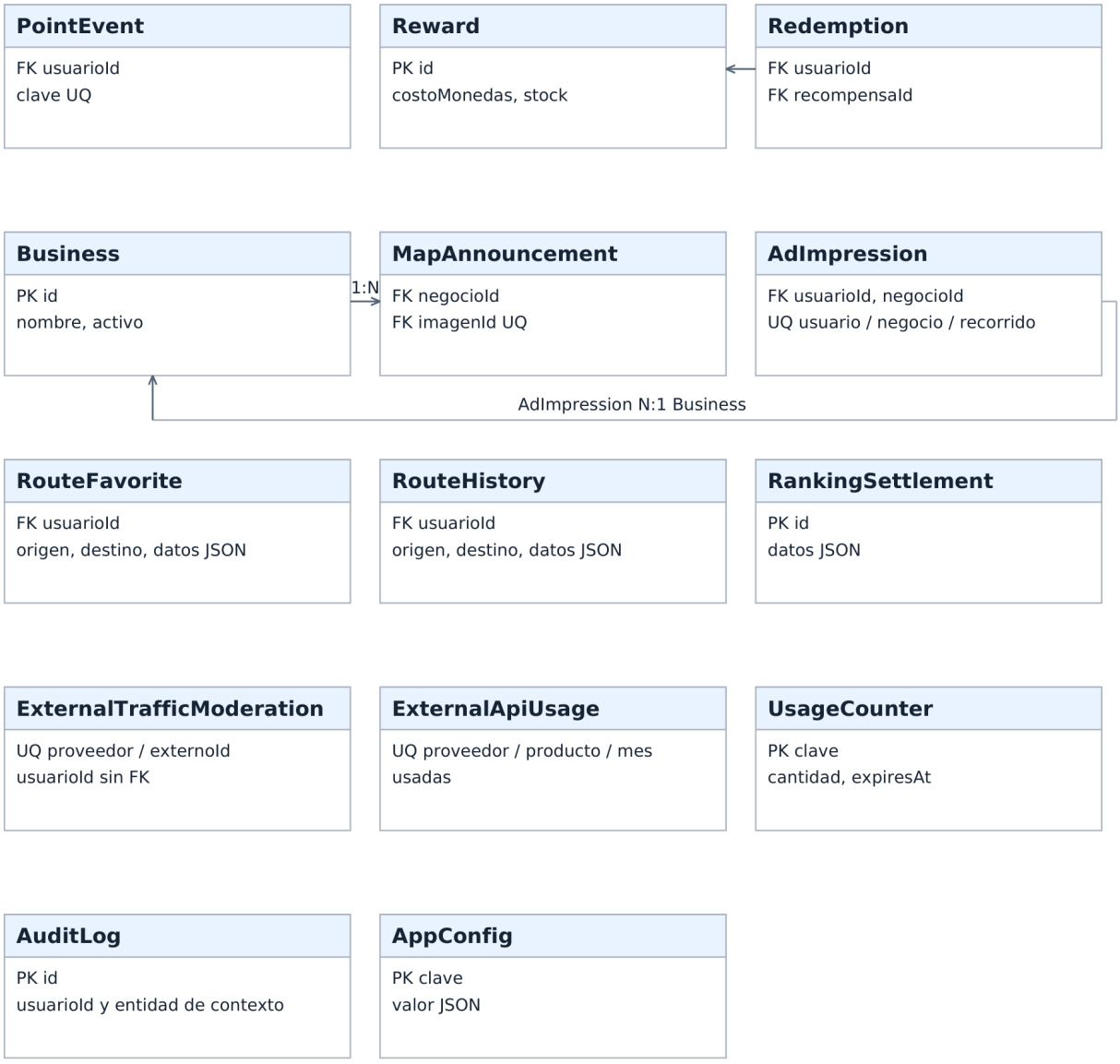


Figura 16 Modelo lógico de incentivos y contenido

RouteFavorite y RouteHistory se relacionan con User. Los registros de control sin relación declarada conservan identificadores o datos de contexto. Fuente R04.

17.3 Diagrama físico

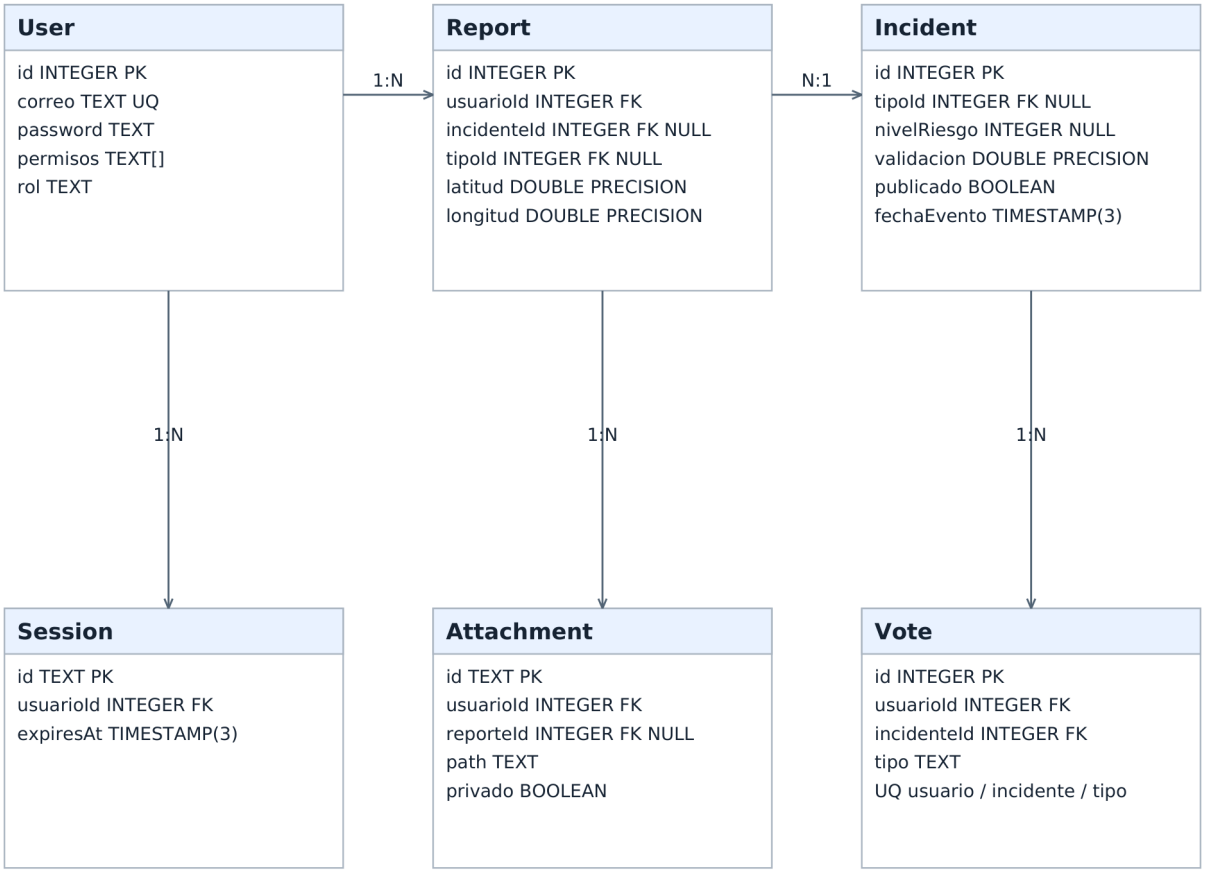


Figura 17 Esquema físico del núcleo de datos

PK identifica claves primarias y FK relaciones declaradas. Los nombres de las tablas respetan los modelos Prisma y requieren comillas cuando se escriben en SQL. Fuente R04.

Las coordenadas se almacenan en columnas Float que PostgreSQL representa como DOUBLE PRECISION. El esquema no declara columnas geométricas de PostGIS. Las búsquedas y la atribución vial utilizan la lógica geográfica de los módulos del backend. Los campos Json se representan como JSONB y los arreglos de texto como TEXT[].

18 Análisis de la calidad de la base de datos

18.1 Normalización y entidades de la DB

El diseño separa cuentas, sesiones, reportes, incidentes y participaciones. Una cuenta puede presentar varios reportes y un incidente puede recibir varios aportes. Vote resuelve la participación de una cuenta en un incidente y restringe duplicados por usuario, incidente y tipo de acción. Attachment conserva los metadatos del archivo y lo vincula con su propietario y, cuando corresponde, con un reporte o anuncio. [R04]

La organización satisface criterios de separación de entidades y dependencias identificables por sus claves. Sin embargo, el modelo utiliza arreglos en permisos y tags, y documentos JSON para configuraciones, rutas y datos de auditoría. Esos campos no permiten afirmar una tercera forma normal estricta sobre toda la semántica almacenada. Su validación depende también de las reglas de aplicación.

El tipo de incidente aparece como referencia tipoId y como texto tipo en Report e Incident. También se conservan contadores y saldos, como totalReportes, validacion y monedas. Estas redundancias facilitan consultas y preservan contexto, pero necesitan actualizaciones coherentes. Las transacciones de revisión, recompensas y canjes deben mantenerlas alineadas con los registros de origen.

Tabla 19 Evaluación del modelo de datos

Aspecto	Evidencia del modelo	Revisión necesaria
Identificación	PK en los 29 modelos y claves únicas en datos de cuenta y catálogo.	Mantener unicidad y mensajes de conflicto al modificar cuentas.
Relaciones	FK para aportes, incidentes, votos, archivos y otros registros de dominio.	Auditar referencias de usuario o entidad guardadas como valores sin FK.
Unicidad de participación	Restricción compuesta de Vote y claves de PointEvent.	Comprobar repetición y concurrencia en votos y premios.
Reserva de datos	Privacidad de Attachment, autorización en API y migración de privilegios.	Verificar permisos reales de conexión y ausencia de acceso web directo.
Consulta	Índices por usuario y fecha, publicación y estado, y uso de proveedor.	Medir consultas con volumen y filtros representativos antes de optimizar.
Consistencia de saldos	Canjes y premios agrupan operaciones mediante transacciones.	Revisar exactitud de Float para monedas si se habilitan importes reales.
Trazabilidad	Review, AuditLog, Appeal y RankingSettlement conservan decisiones o cierres.	Definir retención y revisión de contenido de los campos JSON.
Evolución	Historial versionado de migraciones y preparación local con comprobaciones.	Contrastar esquema y migraciones para evitar diferencias entre ambientes.

La migración de datos privados habilita seguridad por filas y retira privilegios de acceso directo a los roles web de Supabase sobre las tablas del proyecto. El backend mantiene las comprobaciones por usuario, rol y ámbito. Este control de diseño debe revisarse junto con las credenciales y permisos del ambiente de entrega. [R18]

18.2 Diagrama entidad relación

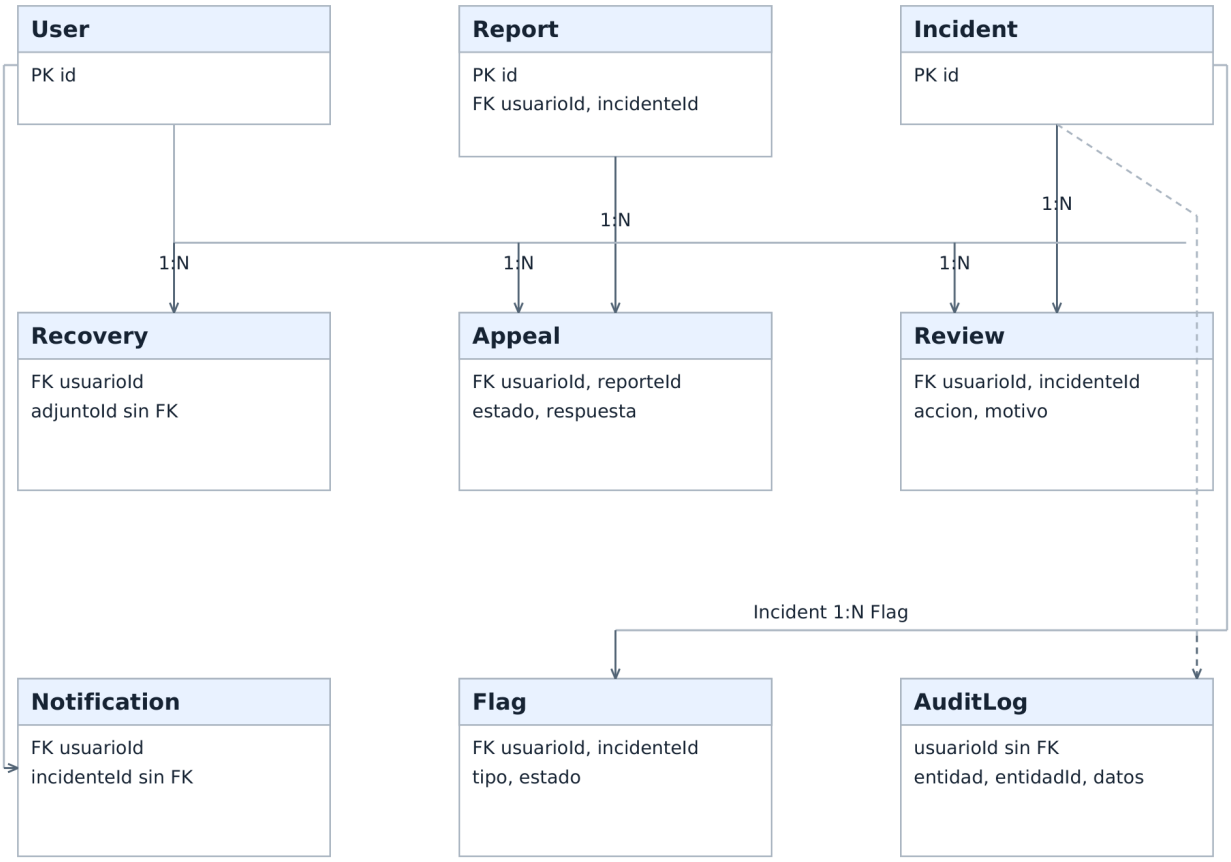


Figura 18 Entidades de revisión y trazabilidad

Review, Appeal y Flag conservan acciones diferentes y no deben sustituirse entre sí. Las referencias sin FK declarada se presentan con línea discontinua. Fuente R04.

18.3 Diccionario de datos

El diccionario comprende todos los campos escalares de los 29 modelos. Los campos de relación de Prisma se representan mediante sus claves foráneas y no como columnas adicionales. Nulo indica si el esquema acepta ausencia de valor. PK, FK y UQ corresponden a clave primaria, clave foránea y unicidad. Las restricciones compuestas se describen después de cada tabla.

Tabla 20 Diccionario de User

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
nombreUsuario	TEXT	No	UQ	Nickname público único de la cuenta.

Campo	Tipo SQL	Nulo	Clave	Descripción
nombres	TEXT	No	—	Nombres privados del titular.
apellidos	TEXT	No	—	Apellidos privados del titular.
fechaNacimiento	TIMESTAMP(3)	Sí	—	Fecha de nacimiento para el control de registro.
correo	TEXT	No	UQ	Correo de contacto de la cuenta.
telefono	TEXT	No	UQ	Teléfono privado de contacto.
password	TEXT	No	—	Hash de la contraseña procesado por el backend.
reputacion	INTEGER	Sí	—	Credibilidad calculada; puede permanecer sin clasificación.
faltas	INTEGER	No	—	Contador de faltas aplicadas tras revisión.
rol	TEXT	No	—	Rol de autorización de la cuenta.
tipoAgente	TEXT	Sí	—	Clasificación que interviene en el ámbito de revisión.
distrito	TEXT	Sí	—	Distrito asociado al usuario o al incidente.
permisos	TEXT[]	No	—	Arreglo de autorizaciones explícitas del agente.
telefonoVerificado	BOOLEAN	No	—	Indicador persistido de verificación de teléfono.
correoVerificado	BOOLEAN	No	—	Indicador de correo comprobado para participar.
bloqueado	BOOLEAN	No	—	Restricción de participación de la cuenta.
premium	BOOLEAN	No	—	Indicador del perfil Premium de demostración.
ocultarAnuncios	BOOLEAN	No	—	Preferencia sujeta a las condiciones del perfil.
monedas	DOUBLE PRECISION	No	—	Saldo de monedas del sistema de incentivos.
fechaCreacion	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 21 Diccionario de Session

Campo	Tipo SQL	Nulo	Clave	Descripción
id	TEXT	No	PK	Hash del token de sesión, no la cookie original.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
expiresAt	TIMESTAMP(3)	No	—	Vencimiento de la sesión, código o contador.
createdAt	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([usuarioId]).

Tabla 22 Diccionario de Verification

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
destino	TEXT	No	—	Dirección o medio al que corresponde la verificación.
codigoHash	TEXT	No	—	Resumen del código de verificación, sin texto original.

Campo	Tipo SQL	Nulo	Clave	Descripción
intentos	INTEGER	No	—	Intentos de comprobación contabilizados.
expiresAt	TIMESTAMP(3)	No	—	Vencimiento de la sesión, código o contador.
usado	BOOLEAN	No	—	Indica que la verificación ya fue utilizada.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([usuarioId, tipo]).

Tabla 23 Diccionario de Category

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
nombre	TEXT	No	—	Denominación del elemento.
slug	TEXT	No	UQ	Identificador textual estable para consultas del catálogo.
orden	INTEGER	No	—	Orden de presentación.

Tabla 24 Diccionario de IncidentType

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
nombre	TEXT	No	—	Denominación del elemento.
slug	TEXT	No	UQ	Identificador textual estable para consultas del catálogo.
categoriaId	INTEGER	No	FK	Referencia a la categoría del catálogo. Relación Category.id.
emergencia	BOOLEAN	No	—	Indicador de tratamiento como emergencia.
historico	BOOLEAN	No	—	Indica que participa como antecedente histórico.
fotoObligatoria	BOOLEAN	No	—	Exige una imagen en el flujo de reporte.
individual	BOOLEAN	No	—	Activa el tratamiento individual y de pruebas privadas.
ubicacionRemota	BOOLEAN	No	—	Permite el registro conforme a reglas de ubicación remota.
persistente	BOOLEAN	No	—	Indica que la situación conserva vigencia hasta resolución.
activo	BOOLEAN	No	—	Habilita el elemento para su uso o publicación.

Tabla 25 Diccionario de Report

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
tipoId	INTEGER	Sí	FK	Referencia opcional al tipo del catálogo. Relación IncidentType.id.
descripcion	TEXT	No	—	Descripción del hecho o elemento.

Campo	Tipo SQL	Nulo	Clave	Descripción
latitud	DOUBLE PRECISION	No	—	Coordenada de latitud del punto.
longitud	DOUBLE PRECISION	No	—	Coordenada de longitud del punto.
gpsLatitud	DOUBLE PRECISION	Sí	—	Latitud GPS aportada para validar proximidad.
gpsLongitud	DOUBLE PRECISION	Sí	—	Longitud GPS aportada para validar proximidad.
nivelRiesgo	INTEGER	Sí	—	Gravedad del incidente o aporte entre 1 y 5, si está evaluada.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
confirmaciones	INTEGER	No	—	Contador legado disponible en el modelo de reporte.
imagen	TEXT	Sí	—	Referencia de imagen conservada en el reporte.
incidenteId	INTEGER	Sí	FK	Identificador del incidente asociado. Relación Incident.id.
fechaEvento	TIMESTAMP(3)	No	—	Fecha del hecho, distinta de la fecha de carga.
fechaCreacion	TIMESTAMP(3)	No	—	Fecha de creación del registro.
recordatorioEnviado	BOOLEAN	No	—	Control de envío de recordatorio.
sancionAplicada	BOOLEAN	No	—	Evita aplicar de nuevo la sanción del mismo aporte.

Restricciones compuestas e índices: Índice([usuarioId, fechaCreacion]).

Tabla 26 Diccionario de Incident

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
tipoId	INTEGER	Sí	FK	Referencia opcional al tipo del catálogo. Relación IncidentType.id.
descripcion	TEXT	No	—	Descripción del hecho o elemento.
latitud	DOUBLE PRECISION	No	—	Coordenada de latitud del punto.
longitud	DOUBLE PRECISION	No	—	Coordenada de longitud del punto.
distrito	TEXT	Sí	—	Distrito asociado al usuario o al incidente.
nivelRiesgo	INTEGER	Sí	—	Gravedad del incidente o aporte entre 1 y 5, si está evaluada.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
totalReportes	INTEGER	No	—	Contador de aportes asociados al incidente.
validacion	DOUBLE PRECISION	No	—	Factor de validación utilizado por el dominio.
publicado	BOOLEAN	No	—	Control de disponibilidad pública del incidente.

Campo	Tipo SQL	Nulo	Clave	Descripción
evaluacion	TEXT	No	—	Origen o estado de la evaluación de gravedad.
individual	BOOLEAN	No	—	Activa el tratamiento individual y de pruebas privadas.
historico	BOOLEAN	No	—	Indica que participa como antecedente histórico.
emergencia	BOOLEAN	No	—	Indicador de tratamiento como emergencia.
persistente	BOOLEAN	No	—	Indica que la situación conserva vigencia hasta resolución.
fuente	TEXT	No	—	Procedencia declarada del incidente.
motivoRetiro	TEXT	Sí	—	Motivo registrado al retirar o resolver.
fechaEvento	TIMESTAMP(3)	No	—	Fecha del hecho, distinta de la fecha de carga.
fechaPublicacion	TIMESTAMP(3)	Sí	—	Inicio efectivo de publicación del incidente.
fechaCreacion	TIMESTAMP(3)	No	—	Fecha de creación del registro.
fechaActualizacion	TIMESTAMP(3)	No	—	Fecha de actualización mantenida por Prisma.

Restricciones compuestas e índices: Índice([publicado, estado, fechaCreacion]). Índice([tipoId, fechaCreacion]).

Tabla 27 Diccionario de UsageCounter

Campo	Tipo SQL	Nulo	Clave	Descripción
clave	TEXT	No	PK	Identificador único del contador, evento o configuración.
cantidad	INTEGER	No	—	Conteo acumulado dentro de la vigencia.
expiresAt	TIMESTAMP(3)	No	—	Vencimiento de la sesión, código o contador.

Restricciones compuestas e índices: Índice([expiresAt]).

Tabla 28 Diccionario de Attachment

Campo	Tipo SQL	Nulo	Clave	Descripción
id	TEXT	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
reporteId	INTEGER	Sí	FK	Referencia al aporte ciudadano. Relación Report.id.
nombre	TEXT	No	—	Denominación del elemento.
mimeType	TEXT	No	—	Tipo MIME validado del archivo.
size	INTEGER	No	—	Tamaño del archivo en bytes.
path	TEXT	No	—	Referencia del archivo en el almacenamiento.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
privado	BOOLEAN	No	—	Exige acceso autorizado al archivo.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([usuarioId, creadoEn]). Índice([estado, reporteId, creadoEn]).

Tabla 29 Diccionario de Vote

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
incidenteId	INTEGER	No	FK	Identificador del incidente asociado. Relación Incident.id.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
latitud	DOUBLE PRECISION	No	—	Coordenada de latitud del punto.
longitud	DOUBLE PRECISION	No	—	Coordenada de longitud del punto.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Unicidad([usuarioId, incidenteId, tipo]).

Tabla 30 Diccionario de ChatMessage

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
incidenteId	INTEGER	No	FK	Identificador del incidente asociado. Relación Incident.id.
mensaje	TEXT	No	—	Texto del mensaje o notificación.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([incidenteId, creadoEn]).

Tabla 31 Diccionario de Notification

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
titulo	TEXT	No	—	Título de presentación.
mensaje	TEXT	No	—	Texto del mensaje o notificación.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
incidenteId	INTEGER	Sí	—	Referencia de contexto sin FK declarada.
leida	BOOLEAN	No	—	Estado de lectura de la notificación.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([usuarioId, creadoEn]).

Tabla 32 Diccionario de Flag

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.

Campo	Tipo SQL	Nulo	Clave	Descripción
incidenteId	INTEGER	No	FK	Identificador del incidente asociado. Relación Incident.id.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
motivo	TEXT	No	—	Justificación de la solicitud o decisión.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Unicidad([usuarioId, incidenteId, tipo]).

Tabla 33 Diccionario de Review

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
incidenteId	INTEGER	No	FK	Identificador del incidente asociado. Relación Incident.id.
accion	TEXT	No	—	Operación registrada por el revisor o la auditoría.
motivo	TEXT	No	—	Justificación de la solicitud o decisión.
datos	JSONB	Sí	—	Datos estructurados de contexto en JSON.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 34 Diccionario de Appeal

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
reporteId	INTEGER	No	FK	Referencia al aporte ciudadano. Relación Report.id.
motivo	TEXT	No	—	Justificación de la solicitud o decisión.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
respuesta	TEXT	Sí	—	Respuesta de atención de la apelación.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 35 Diccionario de Recovery

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
telefonoNuevo	TEXT	No	—	Teléfono propuesto en una recuperación.
adjuntoId	TEXT	Sí	—	Identificador de prueba, sin FK declarada en este modelo.
motivo	TEXT	No	—	Justificación de la solicitud o decisión.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([adjuntoId]).

Tabla 36 Diccionario de PointEvent

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
clave	TEXT	No	UQ	Identificador único del contador, evento o configuración.
tipo	TEXT	No	—	Tipo de acción o clasificación del registro.
puntos	DOUBLE PRECISION	No	—	Valor del evento de participación.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Índice([usuarioId, creadoEn]).

Tabla 37 Diccionario de RankingSettlement

Campo	Tipo SQL	Nulo	Clave	Descripción
id	TEXT	No	PK	Identificador del registro.
datos	JSONB	No	—	Datos estructurados de contexto en JSON.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 38 Diccionario de Reward

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
nombre	TEXT	No	—	Denominación del elemento.
descripcion	TEXT	No	—	Descripción del hecho o elemento.
costoMonedas	DOUBLE PRECISION	No	—	Costo de la recompensa o importe conservado en el canje.
stock	INTEGER	No	—	Unidades disponibles para el canje.
activo	BOOLEAN	No	—	Habilita el elemento para su uso o publicación.
demo	BOOLEAN	No	—	Identifica el contenido de demostración.

Tabla 39 Diccionario de Redemption

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
recompensaId	INTEGER	No	FK	Referencia a la recompensa solicitada. Relación Reward.id.
costoMonedas	DOUBLE PRECISION	No	—	Costo de la recompensa o importe conservado en el canje.
estado	TEXT	No	—	Estado del flujo correspondiente a la entidad.

Campo	Tipo SQL	Nulo	Clave	Descripción
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 40 Diccionario de Business

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
nombre	TEXT	No	—	Denominación del elemento.
descripcion	TEXT	No	—	Descripción del hecho o elemento.
direccion	TEXT	No	—	Dirección de presentación del negocio.
horario	TEXT	No	—	Horario informado del negocio.
sitioWeb	TEXT	Sí	—	Sitio web asociado al negocio.
telefono	TEXT	Sí	—	Teléfono de contacto del negocio.
latitud	DOUBLE PRECISION	No	—	Coordenada de latitud del punto.
longitud	DOUBLE PRECISION	No	—	Coordenada de longitud del punto.
activo	BOOLEAN	No	—	Habilita el elemento para su uso o publicación.
demo	BOOLEAN	No	—	Identifica el contenido de demostración.

Tabla 41 Diccionario de MapAnnouncement

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
tipo	TEXT	No	—	Clase NOVEDAD o NEGOCIO del anuncio.
ubicacion	TEXT	No	—	Zona de publicación permitida para el anuncio.
titulo	TEXT	No	—	Título de presentación.
mensaje	TEXT	No	—	Texto del mensaje o notificación.
enlace	TEXT	Sí	—	Destino del anuncio cuando corresponde.
imagenId	TEXT	Sí	UQ/ FK	Referencia única al archivo de imagen del anuncio. Relación Attachment.id.
imagenAlt	TEXT	No	—	Texto alternativo de la imagen.
negocioId	INTEGER	Sí	FK	Referencia al negocio asociado. Relación Business.id.
activo	BOOLEAN	No	—	Habilita el elemento para su uso o publicación.
orden	INTEGER	No	—	Orden de presentación.
inicio	TIMESTAMP(3)	Sí	—	Inicio opcional de vigencia del anuncio.
fin	TIMESTAMP(3)	Sí	—	Fin opcional de vigencia del anuncio.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.
actualizadoEn	TIMESTAMP(3)	No	—	Fecha de actualización mantenida por Prisma.

Restricciones compuestas e índices: Índice([activo, orden]). Índice([ubicacion, activo, orden]).

Tabla 42 Diccionario de ExternalTrafficModeration

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
proveedor	TEXT	No	—	Identificador del servicio externo.
externoId	TEXT	No	—	Identificador del aviso en el proveedor externo.
oculto	BOOLEAN	No	—	Indica que el aviso externo está oculto por moderación.
motivo	TEXT	No	—	Justificación de la solicitud o decisión.
usuarioId	INTEGER	No	—	Identificador de actor sin FK declarada.
datos	JSONB	Sí	—	Datos estructurados de contexto en JSON.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.
actualizadoEn	TIMESTAMP(3)	No	—	Fecha de actualización mantenida por Prisma.

Restricciones compuestas e índices: Unicidad([proveedor, externoId]). Índice([proveedor, oculto]).

Tabla 43 Diccionario de ExternalApiUsage

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
proveedor	TEXT	No	—	Identificador del servicio externo.
producto	TEXT	No	—	Producto o API cuyo uso se contabiliza.
mes	TEXT	No	—	Periodo mensual del control de uso.
usadas	INTEGER	No	—	Número de solicitudes contabilizadas.
actualizadoEn	TIMESTAMP(3)	No	—	Fecha de actualización mantenida por Prisma.

Restricciones compuestas e índices: Unicidad([proveedor, producto, mes]).

Tabla 44 Diccionario de AdImpression

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
negocioId	INTEGER	No	FK	Referencia al negocio asociado. Relación Business.id.
recorridoId	TEXT	No	—	Referencia del recorrido para evitar impresiones repetidas.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Restricciones compuestas e índices: Unicidad([usuarioId, negocioId, recorridoId]).

Tabla 45 Diccionario de RouteFavorite

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.

Campo	Tipo SQL	Nulo	Clave	Descripción
nombre	TEXT	No	—	Denominación del elemento.
modo	TEXT	No	—	Modo de transporte del recorrido.
origen	JSONB	No	—	Punto y contexto del origen del recorrido en JSON.
destino	JSONB	No	—	Punto y contexto del destino del recorrido en JSON.
datos	JSONB	No	—	Datos estructurados de contexto en JSON.
tags	TEXT[]	No	—	Etiquetas del favorito como arreglo de texto.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 46 Diccionario de RouteHistory

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	No	FK	Identificador de la cuenta asociada. Relación User.id.
modo	TEXT	No	—	Modo de transporte del recorrido.
origen	JSONB	No	—	Punto y contexto del origen del recorrido en JSON.
destino	JSONB	No	—	Punto y contexto del destino del recorrido en JSON.
datos	JSONB	No	—	Datos estructurados de contexto en JSON.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 47 Diccionario de AuditLog

Campo	Tipo SQL	Nulo	Clave	Descripción
id	INTEGER	No	PK	Identificador del registro.
usuarioId	INTEGER	Sí	—	Identificador de actor sin FK declarada.
accion	TEXT	No	—	Operación registrada por el revisor o la auditoría.
entidad	TEXT	No	—	Tipo de entidad afectada por una acción de auditoría.
entidadId	TEXT	Sí	—	Identificador textual de la entidad auditada.
datos	JSONB	Sí	—	Datos estructurados de contexto en JSON.
creadoEn	TIMESTAMP(3)	No	—	Fecha de creación del registro.

Tabla 48 Diccionario de AppConfig

Campo	Tipo SQL	Nulo	Clave	Descripción
clave	TEXT	No	PK	Identificador único del contador, evento o configuración.
valor	JSONB	No	—	Configuración estructurada en JSON.
actualizadoEn	TIMESTAMP(3)	No	—	Fecha de actualización mantenida por Prisma.

18.4 Scripts de creación de DB y tablas

La instalación del proyecto utiliza PostgreSQL y las migraciones versionadas de Prisma. La secuencia prevista en los scripts del backend genera el cliente y aplica el historial de cambios. DATABASE_URL identifica la conexión del ambiente y no debe incorporarse con sus credenciales al documento. [R02, R18]

En un servidor PostgreSQL local nuevo, la base puede prepararse con CREATE DATABASE civigo. En un servicio administrado, la base y el usuario de conexión se configuran en la plataforma. El esquema se aplica desde backend con los comandos del proyecto.

```
npm run db:generate
npm run db:migrate
npm run db:seed
```

El DDL siguiente consolida la estructura del modelo vigente para su lectura académica. Permite identificar tipos, claves e índices sin recorrer los cambios históricos. El despliegue del proyecto debe conservar las migraciones originales para mantener también sus transformaciones de datos, restricciones y controles de permisos.

Tabla User

```
CREATE TABLE "User" (
  "id" SERIAL NOT NULL,
  "nombreUsuario" TEXT NOT NULL,
  "nombres" TEXT NOT NULL DEFAULT '',
  "apellidos" TEXT NOT NULL DEFAULT '',
  "fechaNacimiento" TIMESTAMP(3),
  "correo" TEXT NOT NULL,
  "telefono" TEXT NOT NULL,
  "password" TEXT NOT NULL,
  "reputacion" INTEGER,
  "faltas" INTEGER NOT NULL DEFAULT 0,
  "rol" TEXT NOT NULL DEFAULT 'USUARIO',
  "tipoAgente" TEXT,
  "distrito" TEXT,
  "permisos" TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
  "telefonoVerificado" BOOLEAN NOT NULL DEFAULT false,
  "correoVerificado" BOOLEAN NOT NULL DEFAULT false,
  "bloqueado" BOOLEAN NOT NULL DEFAULT false,
  "premium" BOOLEAN NOT NULL DEFAULT false,
  "ocultarAnuncios" BOOLEAN NOT NULL DEFAULT false,
  "monedas" DOUBLE PRECISION NOT NULL DEFAULT 0,
  "fechaCreacion" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Session

```
CREATE TABLE "Session" (
  "id" TEXT NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "expiresAt" TIMESTAMP(3) NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Verification

```
CREATE TABLE "Verification" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "tipo" TEXT NOT NULL,
  "destino" TEXT NOT NULL,
  "codigoHash" TEXT NOT NULL,
  "intentos" INTEGER NOT NULL DEFAULT 0,
  "expiresAt" TIMESTAMP(3) NOT NULL,
  "usado" BOOLEAN NOT NULL DEFAULT false,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Category

```
CREATE TABLE "Category" (
  "id" SERIAL NOT NULL,
  "nombre" TEXT NOT NULL,
  "slug" TEXT NOT NULL,
  "orden" INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY ("id")
);
```

Tabla IncidentType

```
CREATE TABLE "IncidentType" (
  "id" SERIAL NOT NULL,
  "nombre" TEXT NOT NULL,
  "slug" TEXT NOT NULL,
  "categoriaId" INTEGER NOT NULL,
  "emergencia" BOOLEAN NOT NULL DEFAULT false,
  "historico" BOOLEAN NOT NULL DEFAULT false,
  "fotoObligatoria" BOOLEAN NOT NULL DEFAULT true,
  "individual" BOOLEAN NOT NULL DEFAULT false,
  "ubicacionRemota" BOOLEAN NOT NULL DEFAULT false,
  "persistente" BOOLEAN NOT NULL DEFAULT false,
  "activo" BOOLEAN NOT NULL DEFAULT true,
  PRIMARY KEY ("id")
);
```

Tabla Report

```
CREATE TABLE "Report" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "tipo" TEXT NOT NULL,
  "tipoId" INTEGER,
  "descripcion" TEXT NOT NULL,
  "latitud" DOUBLE PRECISION NOT NULL,
  "longitud" DOUBLE PRECISION NOT NULL,
  "gpsLatitud" DOUBLE PRECISION,
  "gpsLongitud" DOUBLE PRECISION,
  "nivelRiesgo" INTEGER,
  "estado" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "confirmaciones" INTEGER NOT NULL DEFAULT 0,
  "imagen" TEXT,
  "incidenteId" INTEGER,
  "fechaEvento" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "fechaCreacion" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  "recordatorioEnviado" BOOLEAN NOT NULL DEFAULT false,
  "sancionAplicada" BOOLEAN NOT NULL DEFAULT false,
  PRIMARY KEY ("id")
);
```

Tabla Incident

```
CREATE TABLE "Incident" (
  "id" SERIAL NOT NULL,
  "tipo" TEXT NOT NULL,
  "tipoId" INTEGER,
  "descripcion" TEXT NOT NULL DEFAULT '',
  "latitud" DOUBLE PRECISION NOT NULL,
  "longitud" DOUBLE PRECISION NOT NULL,
  "distrito" TEXT,
  "nivelRiesgo" INTEGER,
  "estado" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "totalReportes" INTEGER NOT NULL DEFAULT 1,
  "validacion" DOUBLE PRECISION NOT NULL DEFAULT 0.5,
  "publicado" BOOLEAN NOT NULL DEFAULT false,
  "evaluacion" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "individual" BOOLEAN NOT NULL DEFAULT false,
  "historico" BOOLEAN NOT NULL DEFAULT false,
  "emergencia" BOOLEAN NOT NULL DEFAULT false,
  "persistente" BOOLEAN NOT NULL DEFAULT false,
  "fuente" TEXT NOT NULL DEFAULT 'CIUDADANO',
  "motivoRetiro" TEXT,
  "fechaEvento" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "fechaPublicacion" TIMESTAMP(3),
  "fechaCreacion" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "fechaActualizacion" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("id")
);
```

Tabla UsageCounter

```
CREATE TABLE "UsageCounter" (
  "clave" TEXT NOT NULL,
  "cantidad" INTEGER NOT NULL DEFAULT 0,
  "expiresAt" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("clave")
);
```

Tabla Attachment

```
CREATE TABLE "Attachment" (
  "id" TEXT NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "reporteId" INTEGER,
  "nombre" TEXT NOT NULL,
  "mimeType" TEXT NOT NULL,
  "size" INTEGER NOT NULL,
  "path" TEXT NOT NULL,
  "estado" TEXT NOT NULL DEFAULT 'LISTO',
  "privado" BOOLEAN NOT NULL DEFAULT false,
  "tipo" TEXT NOT NULL DEFAULT 'PUBLICO',
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Vote

```
CREATE TABLE "Vote" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "incidenteId" INTEGER NOT NULL,
  "tipo" TEXT NOT NULL,
  "latitud" DOUBLE PRECISION NOT NULL,
  "longitud" DOUBLE PRECISION NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla ChatMessage

```
CREATE TABLE "ChatMessage" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "incidenteId" INTEGER NOT NULL,
  "mensaje" TEXT NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Notification

```
CREATE TABLE "Notification" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "titulo" TEXT NOT NULL,
  "mensaje" TEXT NOT NULL,
  "tipo" TEXT NOT NULL DEFAULT 'INFO',
  "incidenteId" INTEGER,
  "leida" BOOLEAN NOT NULL DEFAULT false,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Flag

```
CREATE TABLE "Flag" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "incidenteId" INTEGER NOT NULL,
  "tipo" TEXT NOT NULL,
  "motivo" TEXT NOT NULL,
  "estado" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Review

```
CREATE TABLE "Review" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "incidenteId" INTEGER NOT NULL,
  "accion" TEXT NOT NULL,
  "motivo" TEXT NOT NULL,
```

```
  "datos" JSONB,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Appeal

```
CREATE TABLE "Appeal" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "reporteId" INTEGER NOT NULL,
  "motivo" TEXT NOT NULL,
  "estado" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "respuesta" TEXT,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Recovery

```
CREATE TABLE "Recovery" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "telefonoNuevo" TEXT NOT NULL,
  "adjuntoId" TEXT,
  "motivo" TEXT NOT NULL,
  "estado" TEXT NOT NULL DEFAULT 'PENDIENTE',
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla PointEvent

```
CREATE TABLE "PointEvent" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "clave" TEXT NOT NULL,
  "tipo" TEXT NOT NULL,
  "puntos" DOUBLE PRECISION NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla RankingSettlement

```
CREATE TABLE "RankingSettlement" (
  "id" TEXT NOT NULL,
  "datos" JSONB NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Reward

```
CREATE TABLE "Reward" (
  "id" SERIAL NOT NULL,
  "nombre" TEXT NOT NULL,
  "descripcion" TEXT NOT NULL,
```

```
  "costoMonedas" DOUBLE PRECISION NOT NULL,
  "stock" INTEGER NOT NULL DEFAULT 0,
  "activo" BOOLEAN NOT NULL DEFAULT true,
  "demo" BOOLEAN NOT NULL DEFAULT true,
  PRIMARY KEY ("id")
);
```

Tabla Redemption

```
CREATE TABLE "Redemption" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "recompensaId" INTEGER NOT NULL,
  "costoMonedas" DOUBLE PRECISION NOT NULL,
  "estado" TEXT NOT NULL DEFAULT 'SOLICITADO_DEMO',
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla Business

```
CREATE TABLE "Business" (
  "id" SERIAL NOT NULL,
  "nombre" TEXT NOT NULL,
  "descripcion" TEXT NOT NULL,
  "direccion" TEXT NOT NULL,
  "horario" TEXT NOT NULL DEFAULT '',
  "sitioWeb" TEXT,
  "telefono" TEXT,
  "latitud" DOUBLE PRECISION NOT NULL,
  "longitud" DOUBLE PRECISION NOT NULL,
  "activo" BOOLEAN NOT NULL DEFAULT true,
  "demo" BOOLEAN NOT NULL DEFAULT true,
  PRIMARY KEY ("id")
);
```

Tabla MapAnnouncement

```
CREATE TABLE "MapAnnouncement" (
  "id" SERIAL NOT NULL,
  "tipo" TEXT NOT NULL DEFAULT 'NOVEDAD',
  "ubicacion" TEXT NOT NULL DEFAULT 'CABECERA',
  "titulo" TEXT NOT NULL,
  "mensaje" TEXT NOT NULL DEFAULT '',
  "enlace" TEXT,
  "imagenId" TEXT,
  "imagenAlt" TEXT NOT NULL DEFAULT '',
  "negocioId" INTEGER,
  "activo" BOOLEAN NOT NULL DEFAULT true,
  "orden" INTEGER NOT NULL DEFAULT 0,
  "inicio" TIMESTAMP(3),
  "fin" TIMESTAMP(3),
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "actualizadoEn" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("id")
);
```

Tabla ExternalTrafficModeration

```
CREATE TABLE "ExternalTrafficModeration" (
  "id" SERIAL NOT NULL,
  "proveedor" TEXT NOT NULL DEFAULT 'TOMTOM',
  "externoId" TEXT NOT NULL,
  "oculto" BOOLEAN NOT NULL DEFAULT true,
  "motivo" TEXT NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "datos" JSONB,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "actualizadoEn" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("id")
);
```

Tabla ExternalApiUsage

```
CREATE TABLE "ExternalApiUsage" (
  "id" SERIAL NOT NULL,
  "proveedor" TEXT NOT NULL,
  "producto" TEXT NOT NULL,
  "mes" TEXT NOT NULL,
  "usadas" INTEGER NOT NULL DEFAULT 0,
  "actualizadoEn" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("id")
);
```

Tabla AdImpression

```
CREATE TABLE "AdImpression" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "negocioId" INTEGER NOT NULL,
  "recorridoId" TEXT NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla RouteFavorite

```
CREATE TABLE "RouteFavorite" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "nombre" TEXT NOT NULL,
  "modo" TEXT NOT NULL,
  "origen" JSONB NOT NULL,
  "destino" JSONB NOT NULL,
  "datos" JSONB NOT NULL,
  "tags" TEXT[] NOT NULL DEFAULT ARRAY[]::TEXT[],
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla RouteHistory

```
CREATE TABLE "RouteHistory" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER NOT NULL,
  "modo" TEXT NOT NULL,
```

```
  "origen" JSONB NOT NULL,
  "destino" JSONB NOT NULL,
  "datos" JSONB NOT NULL,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla AuditLog

```
CREATE TABLE "AuditLog" (
  "id" SERIAL NOT NULL,
  "usuarioId" INTEGER,
  "accion" TEXT NOT NULL,
  "entidad" TEXT NOT NULL,
  "entidadId" TEXT,
  "datos" JSONB,
  "creadoEn" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY ("id")
);
```

Tabla AppConfig

```
CREATE TABLE "AppConfig" (
  "clave" TEXT NOT NULL,
  "valor" JSONB NOT NULL,
  "actualizadoEn" TIMESTAMP(3) NOT NULL,
  PRIMARY KEY ("clave")
);
```

Relaciones e índices

```
CREATE UNIQUE INDEX "User_nombreUsuario_key"
  ON "User"("nombreUsuario");
CREATE UNIQUE INDEX "User_correo_key"
  ON "User"("correo");
CREATE UNIQUE INDEX "User_telefono_key"
  ON "User"("telefono");
ALTER TABLE "Session" ADD CONSTRAINT "Session_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
CREATE INDEX "Session_usuarioId_idx"
  ON "Session"("usuarioId");
ALTER TABLE "Verification" ADD CONSTRAINT "Verification_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
CREATE INDEX "Verification_usuarioId_tipo_idx"
  ON "Verification"("usuarioId", "tipo");
CREATE UNIQUE INDEX "Category_slug_key"
  ON "Category"("slug");
ALTER TABLE "IncidentType" ADD CONSTRAINT "IncidentType_categoriaId_fkey"
  FOREIGN KEY ("categoriaId") REFERENCES "Category"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE UNIQUE INDEX "IncidentType_slug_key"
  ON "IncidentType"("slug");
ALTER TABLE "Report" ADD CONSTRAINT "Report_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Report" ADD CONSTRAINT "Report_tipoId_fkey"
  FOREIGN KEY ("tipoId") REFERENCES "IncidentType"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;
ALTER TABLE "Report" ADD CONSTRAINT "Report_incidenteId_fkey"
```

```
  FOREIGN KEY ("incidenteId") REFERENCES "Incident"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;
CREATE INDEX "Report_usuarioId_fechaCreacion_idx"
  ON "Report"("usuarioId", "fechaCreacion");
ALTER TABLE "Incident" ADD CONSTRAINT "Incident_tipoId_fkey"
  FOREIGN KEY ("tipoId") REFERENCES "IncidentType"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;
CREATE INDEX "Incident_publicado_estado_fechaCreacion_idx"
  ON "Incident"("publicado", "estado", "fechaCreacion");
CREATE INDEX "Incident_tipoId_fechaCreacion_idx"
  ON "Incident"("tipoId", "fechaCreacion");
CREATE INDEX "UsageCounter_expiresAt_idx"
  ON "UsageCounter"("expiresAt");
ALTER TABLE "Attachment" ADD CONSTRAINT "Attachment_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Attachment" ADD CONSTRAINT "Attachment_reporteId_fkey"
  FOREIGN KEY ("reporteId") REFERENCES "Report"("id")
  ON DELETE SET NULL ON UPDATE CASCADE;
CREATE INDEX "Attachment_usuarioId_creadoEn_idx"
  ON "Attachment"("usuarioId", "creadoEn");
CREATE INDEX "Attachment_estado_reporteId_creadoEn_idx"
  ON "Attachment"("estado", "reporteId", "creadoEn");
ALTER TABLE "Vote" ADD CONSTRAINT "Vote_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Vote" ADD CONSTRAINT "Vote_incidenteId_fkey"
  FOREIGN KEY ("incidenteId") REFERENCES "Incident"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE UNIQUE INDEX "Vote_usuarioId_incidenteId_tipo_key"
  ON "Vote"("usuarioId", "incidenteId", "tipo");
ALTER TABLE "ChatMessage" ADD CONSTRAINT "ChatMessage_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "ChatMessage" ADD CONSTRAINT "ChatMessage_incidenteId_fkey"
  FOREIGN KEY ("incidenteId") REFERENCES "Incident"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE INDEX "ChatMessage_incidenteId_creadoEn_idx"
  ON "ChatMessage"("incidenteId", "creadoEn");
ALTER TABLE "Notification" ADD CONSTRAINT "Notification_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE INDEX "Notification_usuarioId_creadoEn_idx"
  ON "Notification"("usuarioId", "creadoEn");
ALTER TABLE "Flag" ADD CONSTRAINT "Flag_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Flag" ADD CONSTRAINT "Flag_incidenteId_fkey"
  FOREIGN KEY ("incidenteId") REFERENCES "Incident"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE UNIQUE INDEX "Flag_usuarioId_incidenteId_tipo_key"
  ON "Flag"("usuarioId", "incidenteId", "tipo");
ALTER TABLE "Review" ADD CONSTRAINT "Review_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Review" ADD CONSTRAINT "Review_incidenteId_fkey"
  FOREIGN KEY ("incidenteId") REFERENCES "Incident"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Appeal" ADD CONSTRAINT "Appeal_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Appeal" ADD CONSTRAINT "Appeal_reporteId_fkey"
  FOREIGN KEY ("reporteId") REFERENCES "Report"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Recovery" ADD CONSTRAINT "Recovery_usuarioId_fkey"
```

```
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE INDEX "Recovery_adjuntoId_idx"
  ON "Recovery"("adjuntoId");
ALTER TABLE "PointEvent" ADD CONSTRAINT "PointEvent_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE UNIQUE INDEX "PointEvent_clave_key"
  ON "PointEvent"("clave");
CREATE INDEX "PointEvent_usuarioId_creadoEn_idx"
  ON "PointEvent"("usuarioId", "creadoEn");
ALTER TABLE "Redemption" ADD CONSTRAINT "Redemption_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "Redemption" ADD CONSTRAINT "Redemption_recompensaId_fkey"
  FOREIGN KEY ("recompensaId") REFERENCES "Reward"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_imagenId_fkey"
  FOREIGN KEY ("imagenId") REFERENCES "Attachment"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_negocioId_fkey"
  FOREIGN KEY ("negocioId") REFERENCES "Business"("id")
  ON DELETE CASCADE ON UPDATE CASCADE;
CREATE UNIQUE INDEX "MapAnnouncement_imagenId_key"
  ON "MapAnnouncement"("imagenId");
CREATE INDEX "MapAnnouncement_activo_orden_idx"
  ON "MapAnnouncement"("activo", "orden");
CREATE INDEX "MapAnnouncement_ubicacion_activo_orden_idx"
  ON "MapAnnouncement"("ubicacion", "activo", "orden");
CREATE UNIQUE INDEX "ExternalTrafficModeration_proveedor_externoId_key"
  ON "ExternalTrafficModeration"("proveedor", "externoId");
CREATE INDEX "ExternalTrafficModeration_proveedor_oculto_idx"
  ON "ExternalTrafficModeration"("proveedor", "oculto");
CREATE UNIQUE INDEX "ExternalApiUsage_proveedor_producto_mes_key"
  ON "ExternalApiUsage"("proveedor", "producto", "mes");
ALTER TABLE "AdImpression" ADD CONSTRAINT "AdImpression_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "AdImpression" ADD CONSTRAINT "AdImpression_negocioId_fkey"
  FOREIGN KEY ("negocioId") REFERENCES "Business"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
CREATE UNIQUE INDEX "AdImpression_usuarioId_negocioId_recorridoId_key"
  ON "AdImpression"("usuarioId", "negocioId", "recorridoId");
ALTER TABLE "RouteFavorite" ADD CONSTRAINT "RouteFavorite_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE "RouteHistory" ADD CONSTRAINT "RouteHistory_usuarioId_fkey"
  FOREIGN KEY ("usuarioId") REFERENCES "User"("id")
  ON DELETE RESTRICT ON UPDATE CASCADE;
```

Las migraciones incorporan además restricciones específicas para los anuncios y el uso de servicios externos. El siguiente extracto corresponde a esos controles y a la protección de acceso directo de los roles web. [R18]

```
-- Restricciones definidas en las migraciones
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_tipo_check" CHECK ("tipo" IN
('NOVEDAD', 'NEGOCIO' ));
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_negocio_check" CHECK (("tipo" =
'NEGOCIO' AND "negocioId" IS NOT NULL AND "enlace" IS NULL) OR ("tipo" = 'NOVEDAD' AND
"negocioId" IS NULL));
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_fechas_check" CHECK ("inicio"
IS NULL OR "fin" IS NULL OR "fin" > "inicio" );
ALTER TABLE "ExternalApiUsage" ADD CONSTRAINT "ExternalApiUsage_usadas_check" CHECK ("usadas"
```

```
>= 0);
ALTER TABLE "MapAnnouncement" ADD CONSTRAINT "MapAnnouncement_ubicacion_check" CHECK
("ubicacion" IN ('CABECERA', 'REPORTAR' ));


-- CiviGo accesses these tables through its backend connection. The Supabase
-- browser roles must never bypass the API's per-user/district authorization.
DO $$
DECLARE table_name TEXT; api_role TEXT;
BEGIN
FOREACH table_name IN ARRAY ARRAY['User', 'Session' , 'Verification' , 'Category' ,
'IncidentType' , 'Report' , 'Incident' , 'Attachment' , 'Vote' , 'ChatMessage' ,
'Notification' , 'Flag' , 'Review' , 'Appeal' , 'Recovery' , 'PointEvent' ,
'RankingSettlement' , 'Reward' , 'Redemption' , 'Business' , 'MapAnnouncement' ,
'ExternalTrafficModeration' , 'ExternalApiUsage' , 'AdImpression' , 'RouteFavorite' ,
'RouteHistory' , 'AuditLog' , 'AppConfig' , 'UsageCounter' ]
  LOOP
    IF to_regclass(format('public.%I', table_name)) IS NOT NULL THEN
      EXECUTE format('ALTER TABLE public.%I ENABLE ROW LEVEL SECURITY', table_name);
      EXECUTE format('REVOKE ALL PRIVILEGES ON TABLE public.%I FROM PUBLIC', table_name);
      FOREACH api_role IN ARRAY ARRAY['anon', 'authenticated'] LOOP
        IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = api_role) THEN
EXECUTE format('REVOKE ALL PRIVILEGES ON TABLE public.%I FROM %I', table_name, api_role);
        END IF;
      END LOOP;
    END IF;
  END LOOP;
END $$;
```

18.5 Scripts de inserción de data

La carga inicial oficial del backend se ejecuta con npm run db:seed. El script prepara el catálogo y las reglas mediante upsert, y enlaza referencias de tipos en registros previos. El catálogo inicial contiene siete categorías y veintidós tipos. No crea por sí mismo cuentas administradoras ni presenta información demostrativa como si fueran reportes ciudadanos reales. [R14, R19]

El siguiente SQL expresa una carga inicial del catálogo y de las reglas para explicar la estructura de inserción. Utiliza los valores del código y conserva los identificadores existentes mediante las claves slug. Las cláusulas DO NOTHING omiten los registros existentes; para actualizar un catálogo previamente cargado debe utilizarse el script oficial con su lógica de upsert. La creación de cuentas requiere el flujo autorizado y el procesamiento de contraseña del backend.

```
BEGIN;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('seguridad', 'Seguridad y delitos', 0)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'robo', 'Robo', c."id", false, true, false, true, true, false, true
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'hurto', 'Hurto', c."id", false, true, false, true, true, false, true
```

```
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'intento-de-robo' , 'Intento de robo' , c."id", false, true, false, true, true, false,
    true
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'amenazas', 'Amenazas', c."id", false, true, false, true, true, false, true
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'extorsion', 'Extorsión', c."id", false, true, false, true, true, false, true
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'persona-sospechosa' , 'Persona sospechosa' , c."id", true, false, false, false,
false, false, true
FROM "Category" c WHERE c."slug" = 'seguridad'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('emergencias', 'Emergencias y desastres', 1)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'incendio', 'Incendio', c."id", true, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'emergencias'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'humo', 'Humo', c."id", false, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'emergencias'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'fuga-de-gas', 'Fuga de gas', c."id", true, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'emergencias'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'inundacion', 'Inundación', c."id", true, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'emergencias'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'derrumbe', 'Derrumbe', c."id", true, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'emergencias'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('transito', 'Tránsito y movilidad', 2)
```

```
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'accidente-vehicular' , 'Accidente vehicular' , c."id", true, false, true, false,
false, false, true
FROM "Category" c WHERE c."slug" = 'transito'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'semaforo-danado' , 'Semáforo dañado' , c."id", false, false, true, false, false,
false, true
FROM "Category" c WHERE c."slug" = 'transito'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'calle-bloqueada' , 'Calle bloqueada' , c."id", false, false, true, false, false,
false, true
FROM "Category" c WHERE c."slug" = 'transito'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('infraestructura', 'Infraestructura y servicios', 3)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'bache', 'Bache', c."id", false, false, true, false, false, true, true
FROM "Category" c WHERE c."slug" = 'infraestructura'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'mala-iluminacion' , 'Mala iluminación' , c."id", false, false, true, false, false,
true, true
FROM "Category" c WHERE c."slug" = 'infraestructura'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('convivencia', 'Convivencia y entorno', 4)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'pelea', 'Pelea', c."id", false, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'convivencia'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'disturbio', 'Disturbio', c."id", true, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'convivencia'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'basura-acumulada' , 'Basura acumulada' , c."id", false, false, true, false, false,
true, true
FROM "Category" c WHERE c."slug" = 'convivencia'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
```

```
SELECT 'animal-peligroso' , 'Animal peligroso' , c."id", false, false, false, false, false,
false, true
FROM "Category" c WHERE c."slug" = 'convivencia'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('busqueda', 'Búsqueda de personas', 5)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'persona-desaparecida' , 'Persona desaparecida' , c."id", false, true, false, false,
true, false, true
FROM "Category" c WHERE c."slug" = 'busqueda'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "Category" ("slug", "nombre", "orden")
VALUES ('otros', 'Otros', 6)
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "IncidentType" ("slug", "nombre" , "categoriaId" , "emergencia" , "historico" ,
"fotoObligatoria" , "individual" , "ubicacionRemota" , "persistente" , "activo" )
SELECT 'otro', 'Otro incidente', c."id", false, false, true, false, false, false, true
FROM "Category" c WHERE c."slug" = 'otros'
ON CONFLICT ("slug") DO NOTHING;

INSERT INTO "AppConfig" ("clave", "valor", "actualizadoEn")
VALUES ('reglas', '{
  "agrupacionMetros": 50,
  "agrupacionHoras": 3,
  "confirmacionMetros": 50,
  "confirmaciones": 3,
  "resoluciones": 5,
  "influenciaVecina": 0.15,
  "puntosReporte": 10,
  "puntosConfirmacion": 2,
  "puntosPrueba": 3,
  "premiosRanking": [
    100,
    80,
    60,
    50,
    40,
    30,
    20,
    15,
    10,
    5
  ],
  "anuncioMetros": 50,
  "intervaloAnuncioMetros": 150,
  "duracionAnuncioSegundos": 6,
  "maxVideoBytes": 15728640
}'::jsonb, CURRENT_TIMESTAMP)
ON CONFLICT ("clave") DO NOTHING;

COMMIT;
```

El repositorio también incluye un script separado para usuarios de demostración. Este restringe la ejecución en producción y utiliza hash de contraseña. Su propósito es la preparación de un ambiente de prueba. La carga de antecedentes históricos tiene un proceso propio de vista previa y confirmación y debe conservar su fuente y fecha del hecho. [R15, R19]

19 Gestión del desarrollo con Jira y GitHub

GitHub es la fuente de código del proyecto. La revisión documental corresponde a la rama main, commit 78c500f43ee14011802901c9ba6726515e7c2397. Para Jira se plantea la siguiente organización de épicas, historias y backlog como instrumento de seguimiento del ciclo. La planificación propuesta permite vincular los trabajos pendientes con el alcance del Avance 3 y con el cierre de la semana 17.

19.1 Jira Épicas

Tabla 49 Épicas propuestas

ID	Épica	Historias	Contenido
E01	Identidad y acceso	HU01, HU02	Registro, verificación, perfil y recuperación.
E02	Información ciudadana	HU03, HU04, HU05, HU06, HU10	Mapa, aportes, evidencia, participación y avisos.
E03	Consulta geográfica	HU07, HU08, HU09	Riesgo por tramo, rutas, lugares e historial.
E04	Revisión y administración	HU13, HU14, HU15, HU16, HU17, HU19, HU20	Permisos, revisión, catálogos, control e importación.
E05	Funciones complementarias	HU11, HU12, HU18	Incentivos demostrativos, asistente y contenido del mapa.

19.2 Jira HU del sprint 3 Prototipo

Se propone un tercer sprint académico de las semanas 9 a 12 para integrar el prototipo y consolidar este avance. Las historias seleccionadas reúnen la publicación y revisión de información con las interfaces de los dos clientes. El criterio de salida exige comprobar el flujo completo y conservar las incidencias que deban resolverse en el siguiente sprint.

Tabla 50 Historias propuestas para el sprint 3

Historias	Resultado del sprint 3 propuesto	Criterio de salida
HU03, HU04, HU05	Integrar mapa, envío y seguimiento del aporte.	El reporte conserva estado, ubicación y evidencias autorizadas.
HU07, HU08	Relacionar tramos y alternativas de recorrido.	La presentación distingue riesgo registrado, pendientes y tiempo estimado.
HU13, HU14, HU16	Integrar revisión web y administración de escritorio.	Las decisiones dependen de rol, ámbito y permiso y mantienen su motivo.
HU15, HU17, HU18	Consolidar reglas y herramientas de supervisión.	La configuración, filtros y publicación cumplen los contratos de API.
HU01, HU02, HU10	Verificar acceso y comunicación del resultado.	Los estados de verificación, sesión y notificaciones son coherentes.

Las historias no seleccionadas para esta revisión mantienen su identificación en el backlog. El cierre del sprint requiere revisar también los requisitos no funcionales que afectan el flujo, especialmente autorizaciones, archivos privados y consistencia entre el reporte y el incidente.

19.3 Jira Backlog pendientes para otro sprint

Tabla 51 Trabajos pendientes para el cierre

ID	Trabajo de cierre	Sem.	Relación
B01	Coherencia entre contrato y despliegue. Verificar que los dos clientes consuman la misma versión de API, incluidas estadísticas y exportaciones.	13–14	HU14, HU17
B02	Regresión de permisos y datos privados. Comprobar cambios de rol, bloqueo, ámbito, descargas y evidencia en operaciones concurrentes.	13–14	HU05, HU13, HU14, HU16
B03	Calibración y presentación de rutas. Comparar recorridos representativos, revisar advertencias y distinguir estimaciones de tráfico.	13–15	HU07, HU08, HU09
B04	Verificación de almacenamiento y proveedores. Revisar persistencia de archivos, mensajes de fallos, cuotas y configuración del ambiente de entrega.	14–15	HU01, HU04, HU12, HU18
B05	Consistencia del esquema y cargas. Contrastar migraciones, catálogo, referencias históricas y respaldo de datos de prueba.	14–15	HU15, HU19, HU20
B06	Revisión de usabilidad de las interfaces. Recorrer el flujo ciudadano, del agente y del administrador con estados vacíos, errores y restricciones.	15–16	HU03, HU04, HU05, HU13, HU14
B07	Integración de la entrega académica. Actualizar diagramas, matrices y evidencias y preparar la sustentación del proyecto.	16–17	Todas

19.4 Jira Roadmap resumen del progreso

Tabla 52 Roadmap del ciclo

Periodo	Propósito	Producto previsto
Semanas 1–4	Base del proyecto y primer corte interno	Problema, alcance, estructura y persistencia inicial.
Semanas 5–8	Flujos principales y segundo corte interno	Reporte, publicación, acceso y modelo de revisión.
Semanas 9–12	Integración del prototipo y Avance 3	Web y escritorio integrados en el análisis y diseño hasta el punto 20.
Semanas 13–15	Verificación y corrección	Contratos, regresiones, datos y ambiente revisados.
Semanas 16–17	Preparación y cierre académico	Versión de entrega, documento actualizado y sustentación.

El corte en semana 12 describe el estado académico del ciclo. El progreso efectivo de cada historia debe registrarse con su evidencia y criterio de aceptación. El roadmap no asigna porcentajes de terminación a partir del tiempo transcurrido.

19.5 Jira Sincronización con GitHub

La integración propuesta consiste en relacionar cada historia con una rama y una solicitud de cambios que incluyan su identificador. Por ejemplo, una tarea de HU04 puede utilizar un identificador de Jira en el nombre de la rama y en la descripción del cambio. El vínculo debe conducir al cambio real y a sus verificaciones. No reemplaza la revisión del criterio de aceptación.

El repositorio ya define verificaciones para backend y frontend, así como un trabajo de comprobación del escritorio en Windows. Esos trabajos generan evidencias cuando se ejecutan en GitHub Actions. El informe describe su configuración y reserva la validación de una ejecución determinada para el registro del sprint correspondiente. [R17]

Tabla 53 Vinculación propuesta entre Jira y GitHub

Elemento	Vínculo previsto	Evidencia a conservar
Historia Jira	Identificador presente en rama o solicitud de cambios.	Criterios de aceptación y enlace al cambio real.
Solicitud de cambios	Relación con requisito y revisión del código.	Archivos afectados y observaciones atendidas.
GitHub Actions	Verificación del commit de la solicitud.	Ejecución, resultado y ambiente de la comprobación.
Cierre de historia	Revisión del comportamiento esperado.	Evidencia de aceptación y pendientes identificados.

20 Diseño de mockups

Los mockups representan las pantallas principales de los componentes incluidos en el avance. Su distribución y sus controles proceden del código de la web y del escritorio. Los datos mostrados son ilustrativos para explicar el diseño y no constituyen estadísticas de operación. La cartografía del mockup es esquemática. [R20]

La lectura de los diseños sigue dos recorridos. El ciudadano consulta el mapa, registra un hecho y revisa el estado de su aporte. El personal autorizado examina la información y administra las cuentas o la configuración que corresponden a su función. Las observaciones de cada pantalla relacionan ese recorrido con los requisitos y las condiciones de aceptación.

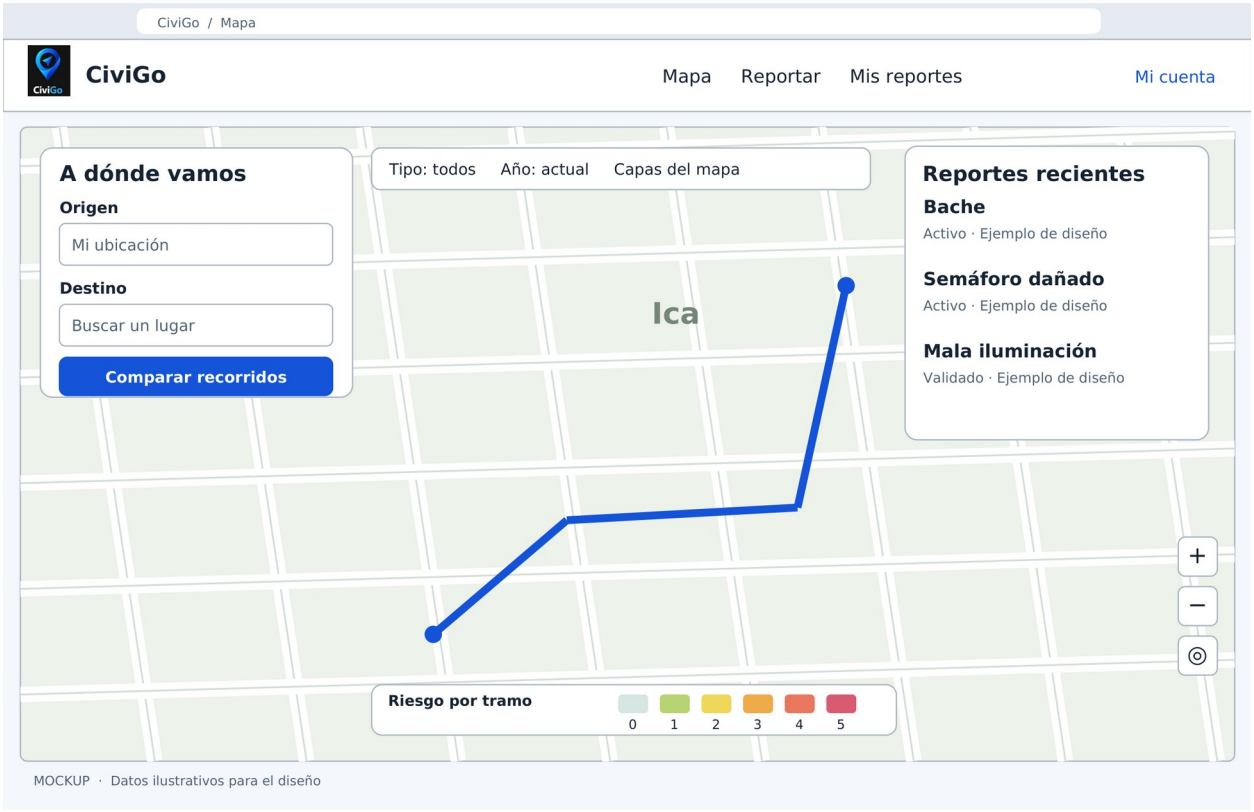


Figura 19 Mapa ciudadano y planificación de recorridos

El mapa reúne filtros, leyenda de riesgo por tramo, búsqueda de destino y controles de ubicación. Fuentes frontend/app/mapa/page.tsx y los componentes de mapa y rutas.

RF05 y RF11–RF13; CU03, CU07 y CU08. La leyenda debe permanecer visible al interpretar los tramos. Las alternativas deben identificar el modo elegido y separar la duración estimada de la valoración del riesgo. La ausencia de resultados y los puntos fuera de cobertura requieren mensajes propios.

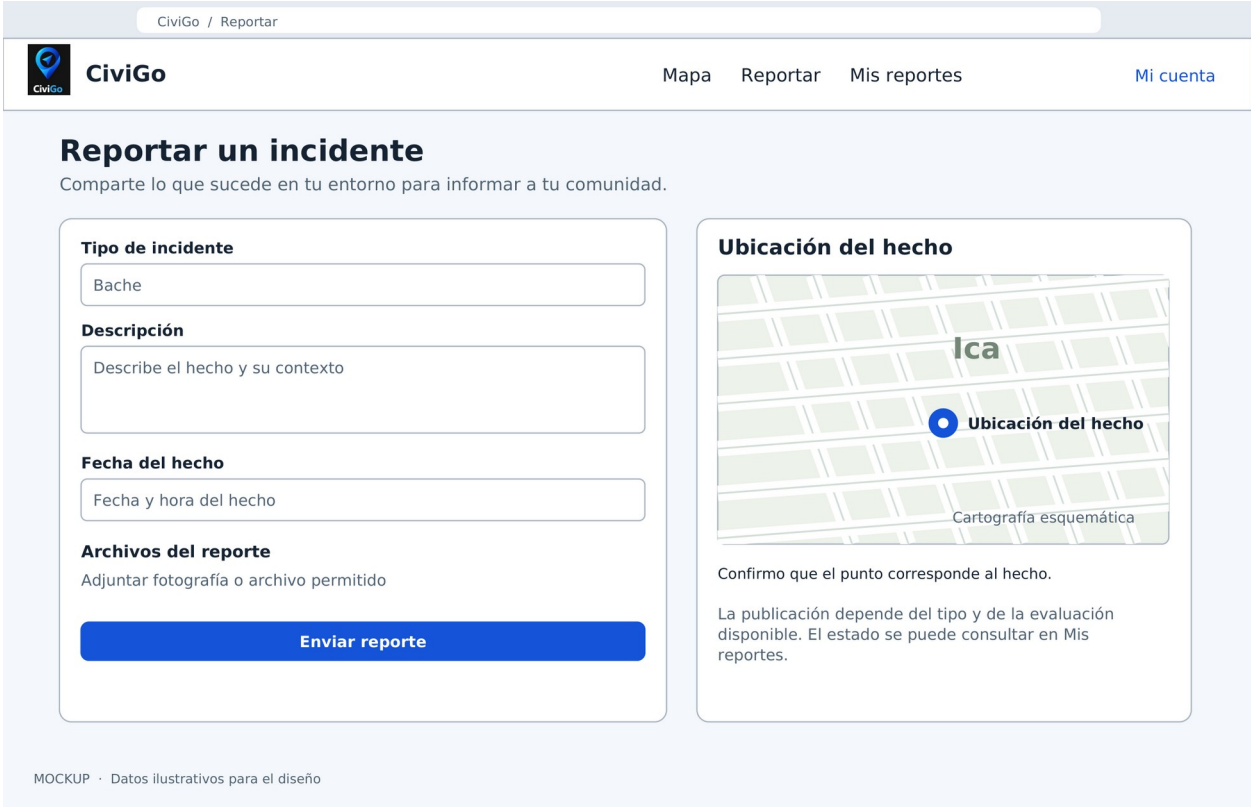


Figura 20 Formulario de reporte ciudadano

La selección del tipo condiciona la ubicación y los archivos exigidos. La evaluación posterior se comunica mediante el estado del aporte. Fuente frontend/app/reportar/page.tsx.

RF06 y RF07; CU04. El tipo seleccionado determina las condiciones de ubicación y evidencia. El formulario debe conservar los datos válidos cuando señala un error y comunicar el estado devuelto por el servidor después del envío. El máximo de adjuntos se comprueba también en la API.

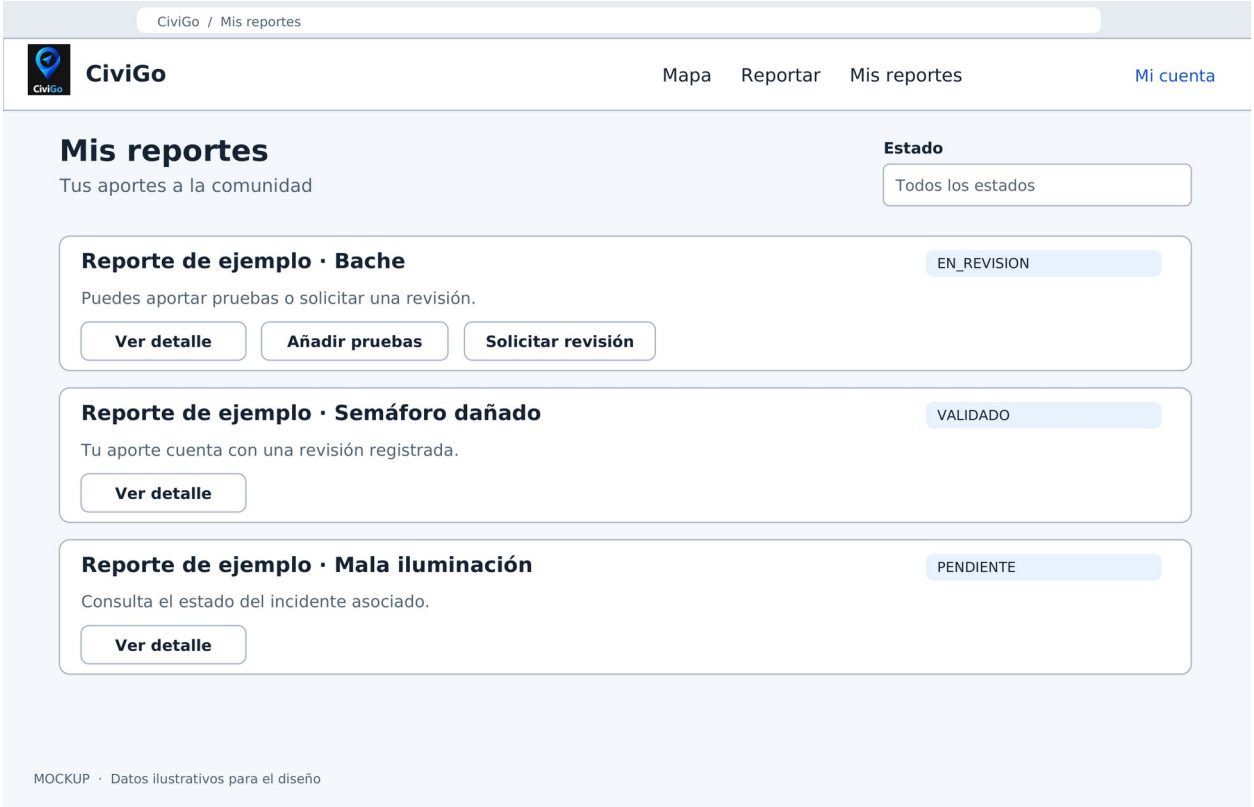


Figura 21 Seguimiento de aportes del ciudadano

El historial permite consultar el estado, añadir pruebas y solicitar revisión cuando corresponde. Fuente frontend/app/mis-reportes/page.tsx.

RF08; CU05. La pantalla organiza los aportes por su estado y muestra las acciones disponibles para su autor. Añadir pruebas o solicitar revisión depende del tipo y de la situación del reporte; las opciones no deben habilitarse solo por estar visibles en el diseño.



Figura 22 Centro web de revisión del agente

Las decisiones visibles dependen del permiso y del ámbito de la cuenta. El formulario solicita motivo para la revisión. Fuente frontend/components/management/IncidentReviewPanel.tsx.

RF18; CU13. El detalle debe ofrecer contexto antes de registrar una decisión. Los permisos de revisión y de acceso a pruebas privadas se comprueban por separado. Una pérdida de autorización debe impedir la confirmación y comunicar el motivo al revisor.

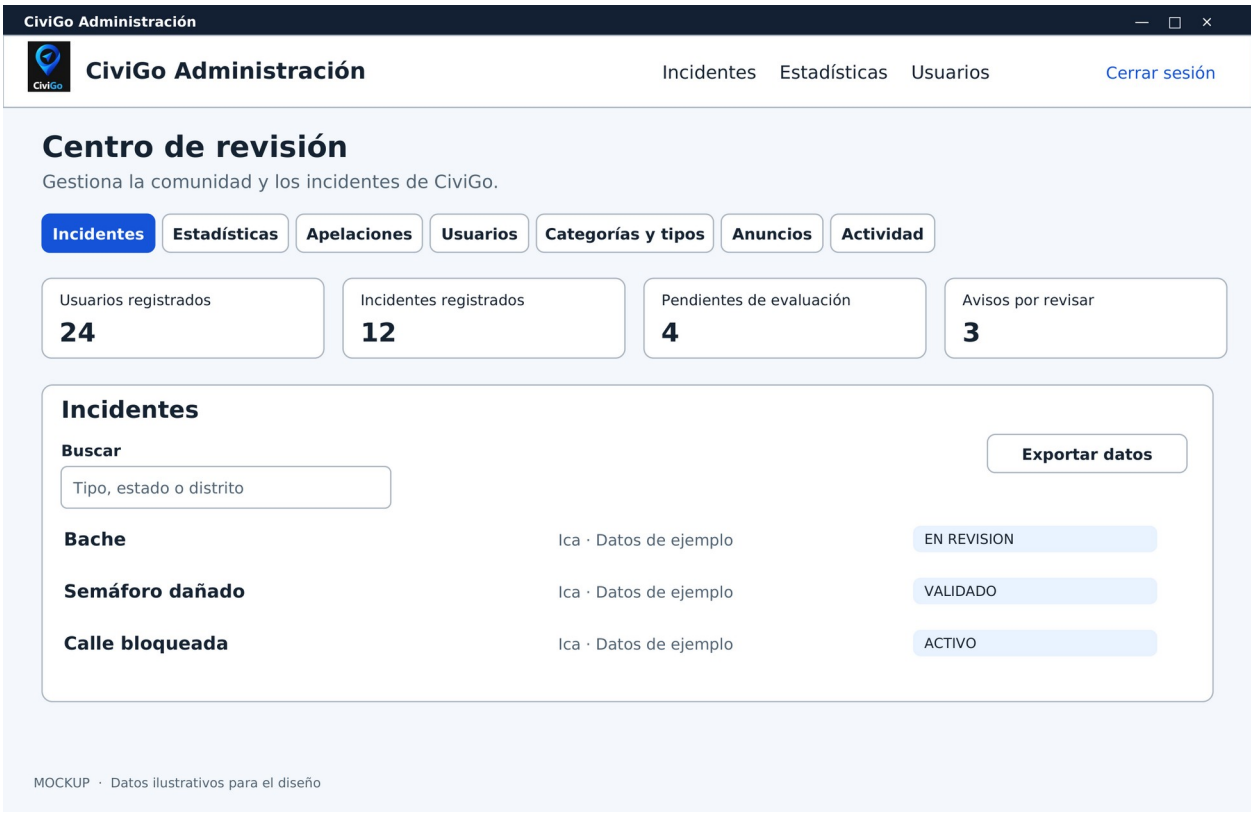


Figura 23 Administración desde el programa de escritorio

El administrador accede a las herramientas de comunidad, revisión y configuración. Fuentes desktop/src/App.tsx y ManagementDashboard.tsx.

RF18–RF25; CU13–CU20. El panel organiza las operaciones de supervisión sin trasladar reglas de autorización a la interfaz. Los indicadores ilustran la distribución visual; la versión operativa debe obtenerlos del contrato administrativo y mostrar el estado de carga o error.

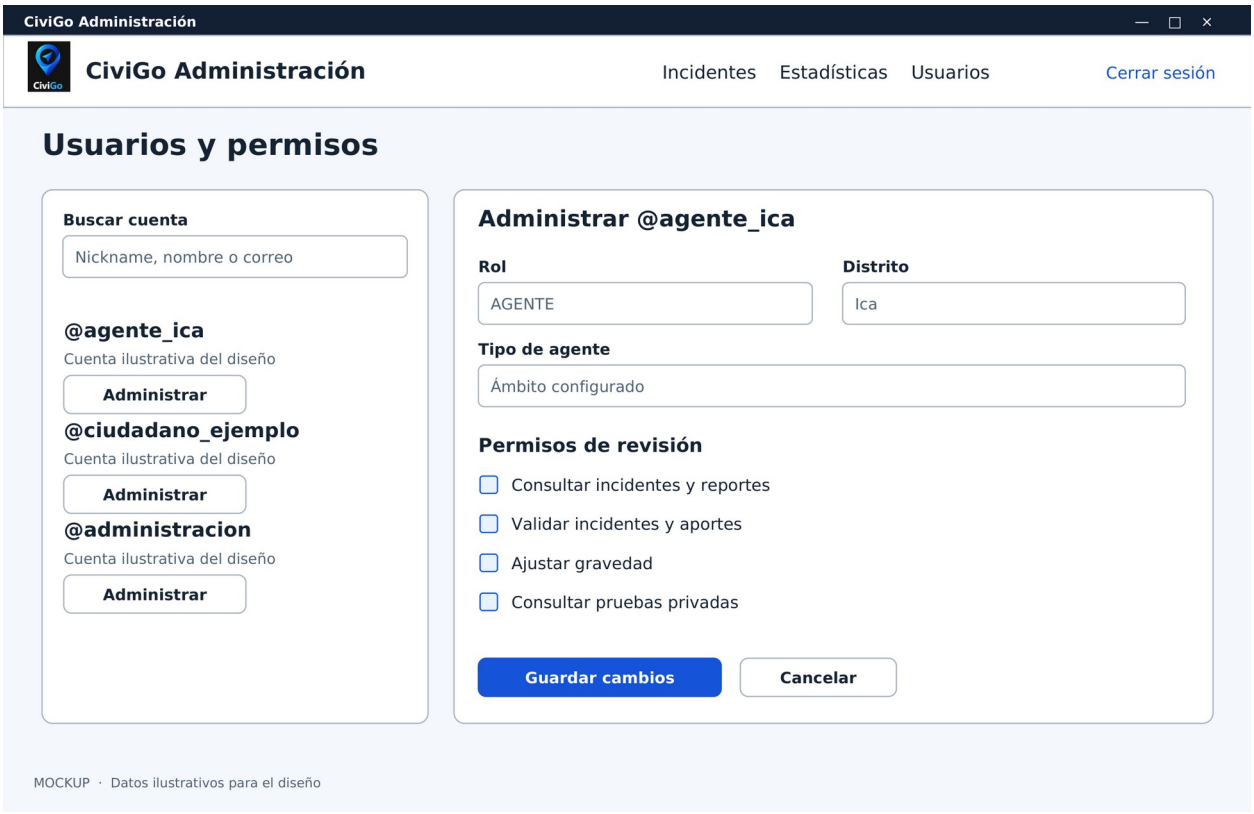


Figura 24 Gestión de usuarios y permisos en escritorio

El diseño diferencia el rol de la cuenta, su ámbito territorial y las autorizaciones de revisión. Fuente desktop/src/components/management/UsersPanel.tsx.

RF19; CU14. El rol, el tipo de agente, el distrito y los permisos son campos distintos. La revisión del formulario debe comprobar combinaciones admitidas, datos duplicados y cambios de autorización durante la operación antes de comunicar que se guardó el resultado.

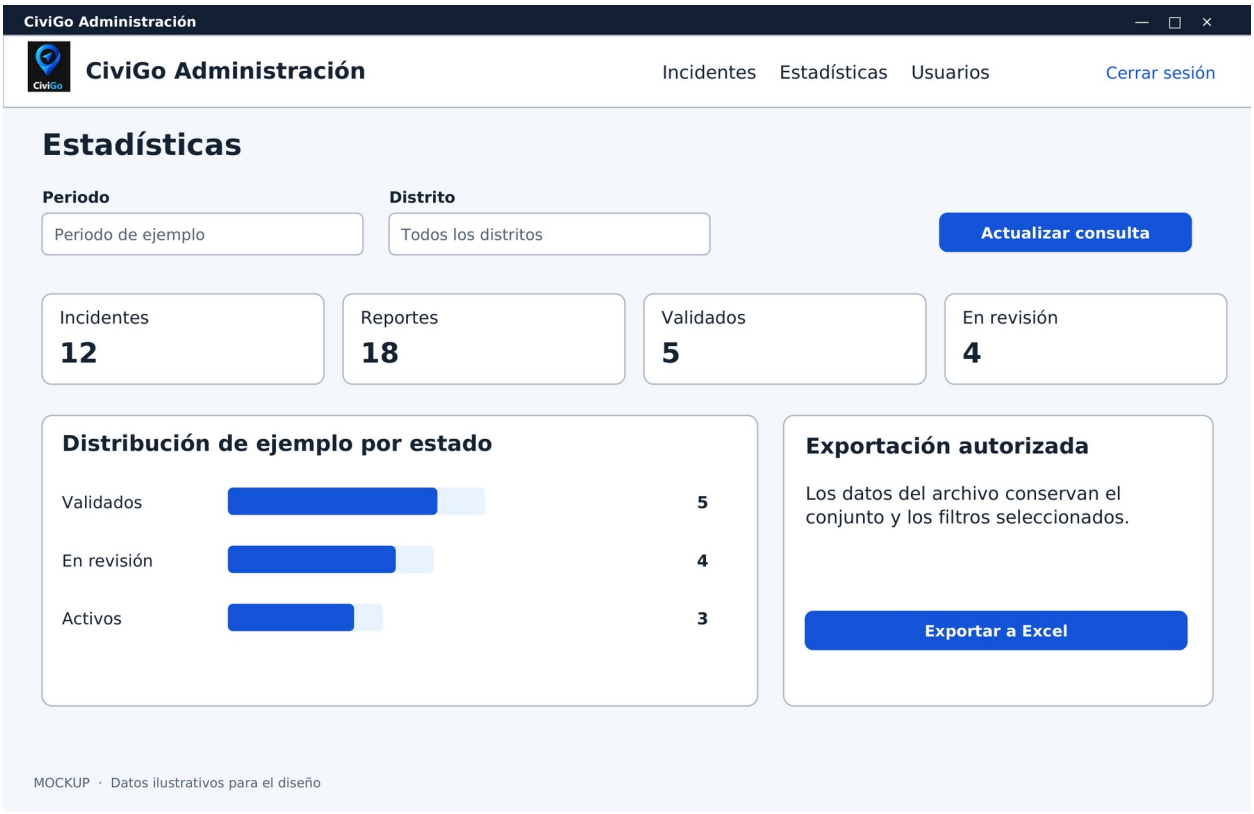


Figura 25 Estadísticas y exportación administrativa

La consulta debe mantener sus filtros y generar únicamente la exportación autorizada. Fuente desktop/src/components/management/StatisticsPanel.tsx.

RF22; CU17. La consulta y el archivo exportado deben conservar el mismo periodo y ámbito. Antes de guardar se debe mantener una sesión administrativa válida. Los valores del gráfico son datos de ejemplo utilizados para evaluar legibilidad y distribución.

Las interfaces deben conservar mensajes distintos para errores de entrada, falta de permiso, ausencia de datos y problemas de conexión. El diseño de la revisión debe mostrar el estado del aporte sin equiparar una sospecha automática con una sanción. En los recorridos, la presentación debe distinguir duración estimada, riesgo registrado y avisos temporales.

Referencias del proyecto

Equipo CiviGo. Repositorio CiviGo, rama main. Revisión 78c500f43ee14011802901c9ba6726515e7c2397. Consulta documental del 26 de octubre de 2026. Cada referencia enlaza al módulo en esta revisión.

[R01] Manifiesto y páginas de la web
frontend/package.json

[R02] Manifiesto y arranque de la API
backend/src/server.js · backend/package.json

[R03] Aplicación administrativa y configuración del paquete
desktop/package.json

[R04] Modelo de persistencia de CiviGo
backend/prisma/schema.prisma

[R05] Acceso, perfil y verificación de correo
backend/src/routes/users.js

[R06] Recepción, evidencias y apelación de reportes
backend/src/routes/reports.js

[R07] Participación y reglas de revisión
backend/src/lib/workflows.js

[R08] Cálculo del riesgo y subdivisión vial
backend/src/lib/risk.js

[R09] Planificación y condiciones de navegación
backend/src/routes/navigation.js

[R10] Operaciones administrativas y autorización
backend/src/routes/admin.js

[R11] Comunidad, notificaciones e incentivos
backend/src/routes/community.js

[R12] Almacenamiento y acceso a archivos
backend/src/lib/storage.js

[R13] Publicación de anuncios y moderación de tráfico
backend/src/routes/announcements.js

[R14] Catálogo y reglas iniciales
backend/src/lib/catalog.js

[R15] Contrato de importación histórica
backend/src/routes/history.js

[R16] Frontera del cliente de escritorio
desktop/electron/main.cjs

[R17] Verificaciones de integración configuradas
.github/workflows/checks.yml

[R18] Historial de migraciones de la base
backend/prisma/migrations

[R19] Carga inicial y datos de demostración
backend/scripts/seed.js

[R20] Diseño de las interfaces web y administrativas
frontend/app/mapa/page.tsx · desktop/src/App.tsx

[R21] Conectores y condiciones de servicios
backend/src/lib/integrations.js

[R22] Estadísticas y conjuntos de exportación
backend/src/lib/admin-statistics.js